Towards Formal Security Proofs of MQOM

小菅 悠久 (Haruhisa Kosuge)[1] and 草川 恵太 (Keita Xagawa)[2]

[1] NTT Social Informatics Laboratories, Japan
hrhs.kosuge@ntt.com
[2] Technology Innovation Institute, UAE
keita.xagawa@tii.ae

March 31, 2026, draft

Abstract. Recent MPC-in-the-Head (MPCitH) signatures increasingly rely on aggressive GGM-tree optimizations to reduce signature size and cost, culminating in *secret-key-root correlated* GGM tree as used in MQOM (NIST PQC Standardization for Additional Signature Round-2, 2024). While this technique yields substantial compression, it introduces a dependency loop in the proof. The transcript we would like to randomize for simulation is generated by expanding a GGM tree from a root that is part of the secret key, so this randomization must be justified via a reduction to the hardness of recovering the secret key. However, the hiding of the secret key relies on masking randomness that is a part of the transcript derived from the same GGM tree. As a result, justifying the randomization requires hiding, while proving hiding requires the randomization, and standard MPCitH proof templates do not apply directly.
We propose and analyze two variants of MQOM and provide the EUF-CMA security proofs. The first variant makes a minor change to salts and replaces blockcipher-based hash functions in the GGM trees with random functions; we then prove its EUF-CMA security in the (quantum) random oracle model under partial-domain one-wayness or slightly stronger one-wayness assumptions. The second variant also makes a minor change to salts and adjusts security parameters to admit a proof under standard one-wayness in the ideal-cipher and random-oracle models. The proof exploits the H-coefficient technique with one-wayness, which might be of independent interest.

Keywords: MPC-in-the-Head signature, Fiat-Shamir transform, secret-key-root correlated GGM tree

Table of Contents

1 Introduction

MPC-in-the-head signatures: One of the standard methods for constructing digital signatures is the *Fiat–Shamir* (FS) transform [FS87], which converts identification (ID) schemes into signature schemes. A prominent paradigm for constructing such ID schemes from arbitrary hard problems is *MPC-in-the-Head* (MPCitH) [IKOS07, GMO16]. Owing to its generality and efficiency, the MPCitH paradigm has undergone rapid development in recent years and has become a central approach for constructing *post-quantum cryptographic* (PQC) signature schemes, which aim to maintain security even against quantum adversaries. The practical relevance of MPCitH-based signatures was further highlighted by the 2022 NIST call for additional digital signature schemes [NIS22], to which nine MPCitH-based proposals were submitted. Among them, six schemes—FAEST [BBB+24], Mirath [AAB+24], MQOM [BBFR24], PERK [ABB+24a], RYDE [ABB+24c], and SDitH [ABB+24b]—advanced to Round 2, demonstrating the maturity and viability of the MPCitH paradigm in practice.

From a design perspective, improving signature size and efficiency has been a major driving force behind recent progress in MPCitH-based schemes. Two prominent variants of MPCitH have emerged in this context: *VOLE-in-the-Head* (VOLEitH) [BBD+23] and *Threshold Computation in the Head* (TCitH) [FR23, FR25].

GGM tree and (secret-key-root) correlated GGM tree: MPC-in-the-Head signature and its variants commonly use all-but-one vector commitments (AVC) [BBD+23], which are often implemented using the GGM tree [GGM86] for efficient and consistent partial disclosure of secret values. Since its structure directly impacts both computational cost and signature size, optimizing GGM-tree techniques is a key research focus.

One promising technique is *half tree technique* to produce *correlated GGM (cGGM) trees*, which is proposed by Guo et al. [GYW+23] in the context of pseudorandom correlation generators and roughly halves the computation of the GGM trees. Several signature schemes already employed the half-GGM tree: ReSolveD [CLY+24], Bui et al. [BCD25], and BACON [WKK+25].

On the cGGM-based AVC, further optimization is proposed: *The secret-key-root cGGM tree*, which we call *scGGM* tree in short, share the same root that is the part of the *secret key* in order to reduce the signature size. SBC [HJ24], MQOM [BBFR24], and rBN++ [KLS25] employed this technique and $\mathsf{SBC}_{\mathsf{VOLE}}$ [HJ25] further extended this technique to the scGGM forest.

Missing Security Proof for MQOM*:* As far as we checked, the MQOM specification does not include a security proof. Moreover, while Kim, Lee, and Son [KLS25] presented a security proof for their proposed signature scheme rBN++, which employs scGGM, Kosuge and Xagawa [KX25] later pointed out a gap in that proof and Kim et al. removed "secret key root" [KLS24, Ver.20251120:150611]. Consequently, at present, there is no available proof technique that can be directly applied to establish the security of MQOM. This leads us to the following research question:

Can MQOM *be proven EUF-CMA secure?*

1.1 Our Contribution

We formally show the EUF-CMA security of MQOM by appropriately modifying its specification in two ways. Our two variants of MQOM (ver.2.1) [BBFR25] are summarized as follows:

- Both variants incorporate the salt `salt` into the input of Hash_4, whose output is used to select the hidden shares in the subsequent rejection-sampling procedure. We only consider the 3-round version, while we can show the security of the 5-round version for the former modification.
- The first variant, denoted MQOM_1, slightly refines the salt-tweaking mechanism to support domain separation by encoding the indices into the last 12 bits of the tweaked salt. Moreover, we need to model the hash function in the scGGM tree as a random oracle, which will require replacing the hash function. These modifications do not affect the key/signature sizes. We can obtain a security proof in the *(quantum) random-oracle model* ((Q)ROM) assuming a stronger variant of the one-wayness, e.g., partial-domain one-wayness.
- The second variant, denoted MQOM_2, is obtained by adjusting the security parameter to match the secret-key size ($\lambda = |x|$); consequently, the key/signature size and computational cost increase accordingly. We can prove its security in the ROM and the *ideal cipher model (ICM)* under the standard onewayness assumption.

To explain the challenges and our solutions, we first review the GGM tree in AVC and its optimizations.

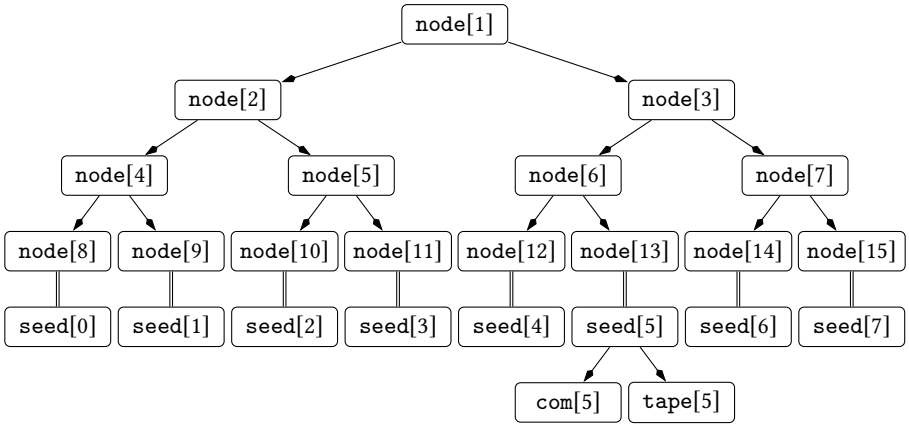

Fig. 1. Example of all-but-one commitment scheme based on the GGM tree with $N = 8$. For $i = 0, \dots, 7$, we let $\mathtt{seed}[i] := \mathtt{node}[N + i]$.

Review of GGM-tree techniques: We briefly review the GGM-tree technique and variants. Suppose that a signer generates τ batches of $N = 2^d$ seeds.

The original GGM tree chooses a root $\mathtt{node}[1] \leftarrow \{0,1\}^\lambda$ at first. For each internal node index $k \in [N]$, the signer derives two children as $(\mathtt{node}[2k], \mathtt{node}[2k+1]) := \mathsf{PRG}(\mathtt{node}[k])$, where $\mathsf{PRG} : \{0,1\}^\lambda \to \{0,1\}^{2\lambda}$. It then obtains the leaf nodes $\mathtt{node}[N], \dots, \mathtt{node}[2N-1]$ and defines $\mathtt{seed}[i] := \mathtt{node}[N+i]$ for all $i \in [N]$. The signer further computes commitments $\mathtt{com}[i] := \mathsf{Com}(\mathtt{seed}[i])$ and random tapes $\mathtt{tape}[i] := \mathsf{XOF}(\mathtt{seed}[i])$ (see Figure 1). In typical MPCitH signatures, the random tapes are used to form secret sharings, say, an N-out-of-N sharing of the secret key x, which is implemented by publishing an offset $\Delta_x := x \oplus \bigoplus_{i \in [N]} \mathtt{tape}[i][0 : |x|]$, where $X[i : j]$ is the substring of X containing bits with indices $i, i+1, \dots, j-1$.

We next review correlated GGM (cGGM) tree [GYW+23]: To reduce the computational cost of the GGM tree, Guo et al. [GYW+23] introduced the use of *circular correlation robust (CCR)* hash functions [CKKZ12] $\mathsf{H}_e : \{0,1\}^\lambda \to \{0,1\}^\lambda$ for $e \in [\tau]$ within the GGM tree. Very roughly speaking, instead of using PRG to derive two child nodes, the e-th GGM tree starts from a random root $\delta \leftarrow \{0,1\}^\lambda$ and defines them as $(\mathtt{node}[2k], \mathtt{node}[2k+1]) := (\mathsf{H}_e(\mathtt{node}[k]), \mathsf{H}_e(\mathtt{node}[k]) \oplus \mathtt{node}[k])$. We note that, by the construction, we have

$$\bigoplus_{i \in [N]} \mathtt{seed}[i] = \bigoplus_{i \in [2^d : 2^{d+1}]} \mathtt{node}[i] = \cdots = \bigoplus_{i \in [2^2 : 2^3]} \mathtt{node}[i] = \mathtt{node}[2] \oplus \mathtt{node}[3] = \delta.$$

Due to this correlation, this GGM tree is dubbed as *correlated* GGM tree. Note that the cGGM trees can share the *same* root $\delta \leftarrow \{0,1\}^\lambda$. In particular, for every $e \in [\tau]$ the leaves satisfy $\bigoplus_{i \in [N]} \mathtt{seed}[e][i] = \delta$, which induces a correlation across all the τ trees.

In *secret-key-root correlated GGM (scGGM) tree* [HJ24, KLS25], the signer sets

$$\delta := x[0 : \lambda] \text{ and } \mathtt{tape}[i] := \mathtt{seed}[i] \,\|\, \mathsf{XOF}(\mathtt{seed}[i]).$$

Since the prefix of each tape equals its seed, the XOR of all tape prefixes is also δ. Thus, when computing the masking offset $\Delta_x := x \oplus \bigoplus_{i \in [N]} \mathtt{tape}[i][0 : |x|]$, the first λ bits always cancel and need not be included in the signature, yielding a reduction of $\tau\lambda$ bits in the signature size.

Informally speaking, MQOM employs the scGGM tree with the Davies–Meyer-like (DM-like) hash $\mathsf{H}_e(s)$.[3]

Techniques for MQOM_1*:* We first present a proof in the ROM, as it captures the core challenge and clarifies how we address it. We consider the following sequence of hybrid games, which differs from the standard EUF-CMA proof template for MPCitH signature schemes.

- Game 0: The original EUF-CMA game.
- Game 1: The signing oracle first samples the hash value and then reprograms Hash_4 to return it on the corresponding input. Since the input to Hash_4 includes a fresh salt $\mathtt{salt}$, the adversary cannot predict this reprogrammed point. Hence, this reprogramming is feasible.

[3] $\mathsf{H}_e(s) := E(e,s) \oplus \sigma(s)$ with a block cipher $E : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda$ and $\sigma(s_l \,\|\, s_r) := (s_l \oplus s_r) \,\|\, s_l$ for $s_l, s_r \in \{0,1\}^{\lambda/2}$ [GKWY20].

– Game 2: The signing oracle computes the hidden index i^* from h at the beginning. This is a pure conceptual change.
– Game 3: We replace the decommitment, $\mathtt{com}[i^*]$, and $\mathtt{tape}[i^*]$ with random ones in the scGGM-tree-based AVC.
– Game 4: We finally replace the secret-key-dependent parts of the signature, e.g., $\Delta_x[\lambda : |x|]$, with random values. In this game, the signing oracle has no need to use the secret key x and an EUF-NMA adversary can simulate this game.

To analyze the difference between Games 2 and 3, a natural strategy is to use a layer-by-layer hybrid argument and to gradually replace the random-function outputs by uniform randomness. However, this is not feasible for the cGGM tree, since δ can be reconstructed from the random-function input at any layer, allowing the adversary to always recover the hidden value that should remain unknown for subsequent randomization. Consequently, we define Bad as the event that the adversary queries a node on one of the hidden paths, equivalently, that it makes a random-oracle query that results in the recovery of δ, and we replace all of the corresponding outputs in a single step.[4] However, in the scGGM tree, $\delta = x[0 : \lambda]$ is part of the secret key and, information-theoretically speaking, this δ is leaked from the public key. This information-theoretic leakage prevents us from using the previous proof techniques [GYW+23, BCD25] to randomize the scGGM tree's output. Furthermore, x is masked using $\mathtt{tape}[i^*]$, which is derived from the same tree outputs that form the transcript. This creates a dependency loop: justifying the randomization of $\mathtt{tape}[i^*]$ requires hiding of the secret key, while proving secret-key hiding relies on the randomness of $\mathtt{tape}[i^*]$.

We therefore postpone bounding $\Pr[\mathsf{Bad}]$ to Game 4, where signature generation no longer uses x. In particular, we show that, after replacing the random-oracle outputs at all inputs corresponding to nodes on the hidden path, Games 2, 3, and 4 remain perfectly indistinguishable. In Game 4, the signing oracle is secret-key free, which allows us to reduce $\Pr[\mathsf{Bad}]$ to partial-domain one-wayness: given the public key $\hat{y}$, recover the partial secret key δ. Conditioned on Bad, there exists a *right* query among the adversary's at most Q queries from which δ can be reconstructed; since the reduction cannot identify it, it guesses a query uniformly at random and succeeds with probability $1/Q$, yielding a multiplicative factor of Q. This completes the EUF-CMA security proof for MQOM_1.

Extension to the QROM proof. In the QROM, we face the tightness issue: unlike in the ROM, measurement-based black-box reductions typically incur a square-root loss [JZM21]. To keep the bound as tight as possible, we therefore aim to make the extracted value δ *verifiable*; namely, once the reduction reconstructs a candidate from a measured oracle query, it should be able to efficiently test whether this candidate is indeed a valid solution. This verification is enabled by working in *domain-extension* one-wayness: the adversary finds a value δ' such that $x' := \mathsf{PRF}(K, \delta')$ is consistent with the public key $\hat{y}$. We then combine the semi-classical one-way-to-hiding (O2H) technique [AHU19] with a carefully chosen puncture set. Since this approach still introduces a multiplicative factor $\sqrt{Q}$, we also consider using the double-sided O2H technique [BHH+19] to eliminate the factor $\sqrt{Q}$ under a stronger version of one-wayness.

Caveats for MQOM *in the ROM+ICM:* While we provide proofs in the ROM/QROM for MQOM_1, these proofs will require replacing the DM-like hash H_e with a monolithic random oracle. Establishing the security of MQOM_1 with the original H_e in the ROM and the *ideal-cipher model (ICM)* remains an interesting open direction, but our strategy for the ROM fails by the following reason: Let us consider the game hops used in the ROM proof for MQOM_1. If the ideal cipher only supported encryption, then we could use the same Bad event and defer bounding $\Pr[\mathsf{Bad}]$ until Game 4, because H_e is indifferentiable from a random oracle (see e.g., [KM07]). However, since the adversary can make decryption queries, we must broaden the definition of Bad by treating the simulator's answers to queries, rather than the queries alone. Thus, the definition of Bad changes when transitioning from Game 3 to Game 4, preventing a direct application of the technique used for MQOM_1. This motivates our second variant, MQOM_2, for which we can bound $\Pr[\mathsf{Bad}]$ even in the presence of decryption queries.

Techniques for MQOM_2*:* Our idea is to apply the well-known H-coefficient technique [Pat09, CS14] to bound the difference between Game 2 and Game 4 *directly*, by defining bad and good transcripts consisting of queries and their corresponding answers. To invoke the H-coefficient technique, we must evaluate the probability that the transcript in Game 4 is bad, which should be negligible, among other requirements. If δ were completely hidden from the adversary, then the evaluation would closely resemble the information-theoretic analysis of

[4] This can be considered as a merger of the proofs in Guo et al. [GYW+23] and Bui et al. [BCD25] and the proof that the random oracle is CCR.

permutation-based CCR hashes [GKWY20]. However, in our setting, the adversary *knows* the public key, which may fully reveal δ if the adversary is unbounded, as we already discussed.

Our key observation is that we can evaluate the bad probability using the *computational* advantage associated with one-wayness. In other words, we can construct a polynomial-time reduction algorithm that simulates Game 4 and recovers δ whenever the adversary produces a bad transcript. To ensure that the reduction successfully recovers δ, we must adjust certain parameters for technical reasons and rely on a heuristic assumption about matrices derived from the adversary's queries and the public key of MQOM. To the best of our knowledge, this is the first result that combines the H-coefficient technique with computational hardness, and we believe that this observation is of independent interest.

The above explanation may suggest that MQOM itself requires no modification. However, backward queries introduce a fundamental obstacle to constructing a one-wayness reduction; therefore, we must adjust some parameters to design a working reduction.

Open problems: Several interesting problems remain: The first important one is showing the security of the original MQOM in the ROM and ICM. The second one is extending the current proof for MQOM_2 to the QROM and the *quantum* ICM (QICM). Currently, we have fewer tools in the QICM than in the QROM, and the extension will require developing such tools for the QICM. The final one is showing the security of other signature schemes, rBN++ [KLS25] and $\mathsf{SBC}_{\mathsf{VOLE}}$ [HJ25].

1.2 Related Works

Study of GGM trees in the MPCitH signatures: Picnic (ver.2.0) [ZCD+19, Section 7.3.1] and BBQ [DDOS19] are the first generation of the MPCitH signatures using the GGM tree with hash-based PRG, e.g., SHAKE. Bui et al. [BCC+24] proposed a construction based on a doubling PRG based on fixed-key block ciphers, $x \mapsto (\pi_0(x) \oplus x, \pi_1(x) \oplus x)$, to compute two child nodes.

On the cGGM tree, Guo et al. [GYW+23] proposed the half-tree technique and cGGM tree using $x \mapsto (\mathsf{H}_e(x), \mathsf{H}_e(x) \oplus x)$. Several signature schemes already employed the cGGM tree with the DM-like hash $\mathsf{H}_e(x) := \pi_e(x) \oplus \sigma(x)$, where σ is a special linear function: ReSolveD [CLY+24], Bui et al. [BCD25], and BACON [WKK+25]. We note that the DM-like hash in the random permutation model or ICM is *distinguishable* from the random oracle [CDMP05, KM07].

The scGGM tree is adopted in SBC [HJ24], rBN++ [KLS25], and MQOM [BBFR24], where MQOM used the DM-like hash H_e. In $\mathsf{SBC}_{\mathsf{VOLE}}$, Huth and Joux [HJ25] proposed the scGGM forest. They employed $\mathsf{H}_e(x) := \pi_{e,0}(x) \oplus \pi_{e,1}(x)$ with two random permutations $\pi_{e,0}$ and $\pi_{e,1}$, which is indifferentiable from the random oracle (see, e.g., [GBJ+23]). Thus, our techniques for MQOM_1 could formally prove the security of $\mathsf{SBC}_{\mathsf{VOLE}}$ in the ROM.

Security proofs for MPCitH signatures based on 3-round protocols: The Fiat–Shamir transform [FS87] converts a 3-round protocol into a signature scheme. Don et al. [DFMS22b] established a tight online-extractability for commit-and-open Σ-protocols by leveraging the notion of extractable QROM, a refinement built upon Zhandry's compressed QRO technique [Zha19]. In a companion work [DFMS22a], they proved *tight* security in the QROM for both a NIZK proof of knowledge and a signature scheme obtained via the FS transform applied to commit-and-open Σ-protocols (e.g., Picnic [CDG+17]); their argument proceeds through the framework of Chung et al. [CFHL21], which itself relies on compressed QRO [Zha19].

1.3 Concurrent Work

Feneuil and Rivain [FR26] concurrently provided security proofs for four variants of MQOM (ver. 2.0). They consider four variants in which Hash_e, Com, and XOF are instantiated by (1) random oracles, (2) Davies–Meyer–like hash functions, (3) counter modes, or (4) Davies–Meyer–like hash functions combined with XOR-of-Permutations (XoP) constructions [GBJ+23, Din24]. The first variant is essentially the same as our MQOM_1, and the second variant is very similar to our MQOM_2, whereas their second variant preserves the security parameter. They then study the simulation security of openings of the scGGM trees and prove it using the H-coefficient technique, as in our proof for MQOM_2. To establish the simulation security and EUF-CMA security of their variants, they introduce two new assumptions. The first is partial-guessing one-wayness, which is essentially equivalent to partial-domain one-wayness. The second is relation-guessing one-wayness, where an adversary is given access to several relations $g : \mathbb{F}^n \to \{0,1\}^\lambda$ and wins if it queries a generated relation g along with $\hat{x}_0$ such that $g(x) = \hat{x}_0$. The first, third, and fourth variants rely on the partial-guessing one-wayness of the key-generation algorithm, while the second variant additionally requires relation-guessing one-wayness.

1.4 Organization

In Section 2, we fix notation and recall the proof techniques we use in the QROM. In Section 3, we describe the (correlated half-tree) all-but-one vector commitment used in MQOM. In Section 4, we review the specification of MQOM and introduce the one-wayness notions for key generation. In Section 5, we prove the EUF-CMA security of MQOM_1 in the ROM and extend the argument to the QROM. Finally, in Section 6, we study the EUF-CMA security of MQOM_2 in the ROM+ICM setting.

Section A reviews the parameter sets of MQOM. Section B and Section C contain the missing definitions and proofs, respectively. Section D provides two warm-up examples illustrating applications of the H-coefficients technique combined with one-wayness. In Section E, we prove the EUF-NMA securities of MQOM_1 in the (Q)ROM and MQOM_2 in the ROM+ICM setting.

2 Preliminaries

The security parameter is denoted by $\lambda \in \mathbb{Z}^+$. We use the standard O-notation. DPT, PPT, and QPT stand for deterministic, probabilistic, and quantum polynomial time, respectively.

For $n \in \mathbb{Z}^+$, we let $[n] := \{0, \dots, n-1\}$. For $n_1, n_2 \in \mathbb{Z}^+$, we let $[n_1 : n_2] := \{n_1, n_1+1, \dots, n_2-1\}$ as Python. Let $\mathcal{X}$ and $\mathcal{Y}$ be two finite sets. $\mathsf{Func}(\mathcal{X}, \mathcal{Y})$ denotes a set of all functions whose domain is $\mathcal{X}$ and codomain is $\mathcal{Y}$.

For a distribution D, we often write "$x \leftarrow D$," which indicates that we take a sample x according to D. For a finite set $\mathcal{S}$, $U(\mathcal{S})$ denotes the uniform distribution over $\mathcal{S}$. We often write "$x \leftarrow \mathcal{S}$" instead of "$x \leftarrow U(\mathcal{S})$." If inp is a string, then "$\mathsf{out} \leftarrow \mathsf{A}^O(\mathsf{inp})$" denotes the output of algorithm A running on input inp with an access to a set of oracles O. If A and oracles are deterministic, then out is a fixed value and we write "$\mathsf{out} := \mathsf{A}^O(\mathsf{inp})$." We also use the notation "$\mathsf{out} := \mathsf{A}(\mathsf{inp}; r)$" to make the randomness r of A explicit.

For a function $f : \{0,1\}^n \to \{0,1\}^m$, a *quantum access* to f is modeled as an oracle access to unitary $O_f : |x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle$. By convention, we will use the notation $\mathsf{A}^{|f\rangle,g}$ to stress A's *quantum* and classical access to f and g, respectively. For a function $f : \mathcal{X} \to \mathcal{Y}$, we denote the procedure reprogramming $f(x)$ with h by $f := f[x \mapsto h]$.

For a non-negative integer $i \in [2^x]$, $\mathsf{bin}_x(i)$ denotes the x-bit representation of the integer i.

2.1 Digital Signature

The model for digital signature schemes is summarized as follows:

Definition 2.1. *A digital signature scheme* DS *consists of the following triple of PPT algorithms* (Gen, Sign, Vrfy)*:*

- $\mathsf{Gen}(1^\lambda) \to (\mathtt{vk}, \mathtt{sk})$*: a key-generation algorithm that, on input* 1^λ*, outputs a pair of keys* $(\mathtt{vk}, \mathtt{sk})$*.* $\mathtt{vk}$ *and* $\mathtt{sk}$ *are verification and signing keys, respectively.*
- $\mathsf{Sign}(\mathtt{sk}, \mathtt{msg}) \to \sigma$*: a signing algorithm that takes as input signing key* $\mathtt{sk}$ *and message* $\mathtt{msg} \in \mathcal{M}$ *and outputs signature* $\sigma \in \mathcal{S}$*.*
- $\mathsf{Vrfy}(\mathtt{vk}, \mathtt{msg}, \sigma) \to 0/1$*: a verification algorithm that takes as input verification key* $\mathtt{vk}$*, message* $\mathtt{msg} \in \mathcal{M}$*, and signature* σ *and outputs its decision* 1 *for acceptance or* 0 *for rejection.*

We require perfect correctness; that is, for any message $\mathtt{msg} \in \mathcal{M}$*, we have*

$$\Pr[(\mathtt{vk}, \mathtt{sk}) \leftarrow \mathsf{Gen}(1^\lambda), \sigma \leftarrow \mathsf{Sign}(\mathtt{sk}, \mathtt{msg}) : \mathsf{Vrfy}(\mathtt{vk}, \mathtt{msg}, \sigma) = 1] = 1$$

for some negligible function δ*.*

Security notions: We review the standard security notions of existential unforgeability under chosen-message attacks (EUF-CMA) and under no-message attacks (EUF-NMA).

Definition 2.2 (EUF-CMA and EUF-NMA security). *Let* $\mathsf{DS} = (\mathsf{Gen}, \mathsf{Sign}, \mathsf{Vrfy})$ *be a digital signature scheme. For any* $\mathcal{A}$*, we define its* euf-cma/euf-nma *advantages against* DS *as* $\mathsf{Adv}^{\text{euf-cma}}_{\mathsf{DS},\mathcal{A}}(\lambda) := \Pr[\mathsf{Expt}^{\text{euf-cma}}_{\mathsf{DS},\mathcal{A}}(1^\lambda) = 1]$ *and* $\mathsf{Adv}^{\text{euf-nma}}_{\mathsf{DS},\mathcal{A}}(\lambda) := \Pr[\mathsf{Expt}^{\text{euf-nma}}_{\mathsf{DS},\mathcal{A}}(1^\lambda) = 1]$*, where* $\mathsf{Expt}^{\text{euf-cma}}_{\mathsf{DS},\mathcal{A}}(1^\lambda)$ *and* $\mathsf{Expt}^{\text{euf-nma}}_{\mathsf{DS},\mathcal{A}}(1^\lambda)$ *are the experiments described in Figure 2. We say that* DS *is EUF-CMA-secure (resp., EUF-NMA-secure) if* $\mathsf{Adv}^{\text{euf-cma}}_{\mathsf{DS},\mathcal{A}}(\lambda)$ *(resp.,* $\mathsf{Adv}^{\text{euf-nma}}_{\mathsf{DS},\mathcal{A}}(\lambda)$*) is negligible in* λ *for any QPT adversary* $\mathcal{A}$*.*

Game $\mathsf{Expt}^{\text{euf-nma}}_{\mathsf{DS},\mathcal{A}}(1^\lambda)$ / $\mathsf{Expt}^{\text{euf-cma}}_{\mathsf{DS},\mathcal{A}}(1^\lambda)$	SIGN(msg)
1 : $\mathcal{Q} := \emptyset$	1 : $\sigma \leftarrow \mathsf{Sign}(\mathtt{sk}, \mathtt{msg})$
2 : $(\mathtt{vk}, \mathtt{sk}) \leftarrow \mathsf{Gen}(1^\lambda)$	2 : $\mathcal{Q} := \mathcal{Q} \cup \{\mathtt{msg}\}$
3 : $(\mathtt{msg}^*, \sigma^*) \leftarrow \mathcal{A}(\mathtt{vk})$ // euf-nma	3 : **return** σ
4 : $(\mathtt{msg}^*, \sigma^*) \leftarrow \mathcal{A}^{\text{SIGN}}(\mathtt{vk})$ // euf-cma	
5 : **if** $\mathtt{msg} \in \mathcal{Q}$ **then return** 0 // euf-cma	
6 : **return** $\mathsf{Vrfy}(\mathtt{vk}, \mathtt{msg}^*, \sigma^*)$	

Fig. 2. Games for EUF-NMA and EUF-CMA security (Definition 2.2).

3 (Correlated) All-but-one Vector Commitment

We formulate a half-tree-based (correlated) all-but-one vector commitment (AVC) scheme as a tuple of three algorithms, (Commit, Open, Recon), where we assume that $N = 2^d$ and let $i^* \in [N]$ be a hidden index. We further let n_b denote the tape length in λ-bit blocks.

- Commit takes salt $\mathtt{salt}$, root seed $\mathtt{rseed} \in \{0,1\}^\lambda$, root $\delta \in \{0,1\}^\lambda$, and tree index $e \in [\tau]$ as input and outputs commitment $\mathtt{com}[e]$, decommitment $\mathtt{decom}[e]$, random seeds $\mathtt{seed}[e] \in (\{0,1\}^\lambda)^N$, and random tapes $\mathtt{tape}[e] \in (\{0,1\}^{n_b\lambda})^N$.
- Open takes $\mathtt{decom}[e]$, e, and hidden leaf index $i^* \in [N]$ as input and outputs an opening information $\mathtt{pdecom}[e]$.
- Recon takes $\mathtt{salt}$, $\mathtt{pdecom}[e]$, e, and i^* as input and outputs reconstructed $\mathtt{com}[e]$, $(\mathtt{seed}[e][i])_{i \neq i^*}$, $(\mathtt{tape}[e][i])_{i \neq i^*}$.

We define two functions to compute a path from the hidden index i^* to the root and a set of siblings of the nodes in the path:

- $\mathsf{path}_j(i^*) := \lfloor (N + i^*)/2^{d-j} \rfloor$, which is the index of the hidden path from i^* in the layer $j \in [1 : d+1]$.
- $\mathsf{copath}_j(i^*) := \lfloor (N + i^*)/2^{d-j} \rfloor \oplus 1$, which is the index of the siblings of the hidden nodes in the layer $j \in [1 : d+1]$.

We describe the cGGM-tree-based AVC scheme in MQOM[5] in Figure 3, which we call $\mathsf{cAVC}_{\mathsf{IC}}$ and $\mathsf{cAVC}_{\mathsf{RO}}$. As a restriction on the indices (i, j) provided as input to TweakSalt, we introduce a set $\mathcal{I} := \{(0,0),(1,0)\} \cup (\{2\} \times [1 : d]) \cup (\{3\} \times [n_b])$.

4 MQOM

Specification: Let $\mathbb{F}$ be a finite field. Let $x \leftarrow \mathbb{F}^n$ be a secret key. The public key consists of m degree-2 polynomials $F = (f_0, \ldots, f_{m-1}) \in (\mathbb{F}[x_0, \ldots, x_{n-1}])^m$ with $f_i : x \mapsto x^\top A_i x + b_i^\top x - y_i$, where $A_i \in \mathbb{F}^{m \times m}$, $b_i \in \mathbb{F}^m$, and $y_i \in \mathbb{F}$, such that $F(x) = 0$. The underlying ID protocol is designed to show the knowledge of x satisfying $F(x) = 0$.

Let $\mathbb{K}$ be an extension field of $\mathbb{F}$ with basis $\beta_0, \ldots, \beta_{\mu-1}$. Let $\phi : (e_0, \ldots, e_{\mu-1}) \in \mathbb{F}^\mu \mapsto \sum_{i \in [\mu]} e_i \cdot \beta_i \in \mathbb{K}$. Let $\hat{m} := m/\mu \in \mathbb{N}$. We expand ϕ in a natural way. We then define

$$\begin{aligned} \hat{A}_0 &:= \phi(A_0, \ldots, A_{\mu-1}), & \ldots, & & \hat{A}_{\hat{m}-1} &:= \phi(A_{m-\mu}, \ldots, A_{m-1}), \\ \hat{b}_0 &:= \phi(b_0, \ldots, b_{\mu-1}), & \ldots, & & \hat{b}_{\hat{m}-1} &:= \phi(b_{m-\mu}, \ldots, b_{m-1}), \\ \hat{y}_0 &:= \phi(y_0, \ldots, y_{\mu-1}), & \ldots, & & \hat{y}_{\hat{m}-1} &:= \phi(y_{m-\mu}, \ldots, y_{m-1}). \end{aligned}$$

From ver.2.1 [BBFR25], MQOM's signer shows that $\hat{F}(x) = (0, \ldots, 0) \in \mathbb{K}^{\hat{m}}$, where $\hat{F} = (f_0, \ldots, f_{\hat{m}-1})$ with

$$\hat{f}_i : x \in \mathbb{F}^n \mapsto x^\top \hat{A}_i x + \hat{b}_i^\top x - \hat{y}_i \in \mathbb{K}, \tag{1}$$

where $(\hat{A}_i, \hat{b}_i)$ are expanded from the seed $\mathtt{mseed}_{\mathsf{eq}}$ via $(\{\hat{A}_i\}, \{\hat{b}_i\})_{i \in [\hat{m}]} := \mathsf{ExpandEquations}(\mathtt{mseed}_{\mathsf{eq}})$. Due to linearity of ϕ, $F(x) = 0$ is equivalent to $\hat{F}(x) = 0$.

We describe MQOM (v2.1) in Figures 4, 5, and 6. Here, $\Omega = \{\omega_0, \ldots, \omega_{N-1}\} \subseteq \mathbb{K}$ is the evaluation domain, and each Hash_i outputs 2λ bits, i.e., has range $\{0,1\}^{2\lambda}$ (see [BBFR24, Section 2.5.5]). Parameter sets are summarized in Section A.

[5] Inside of BLC.Commit.

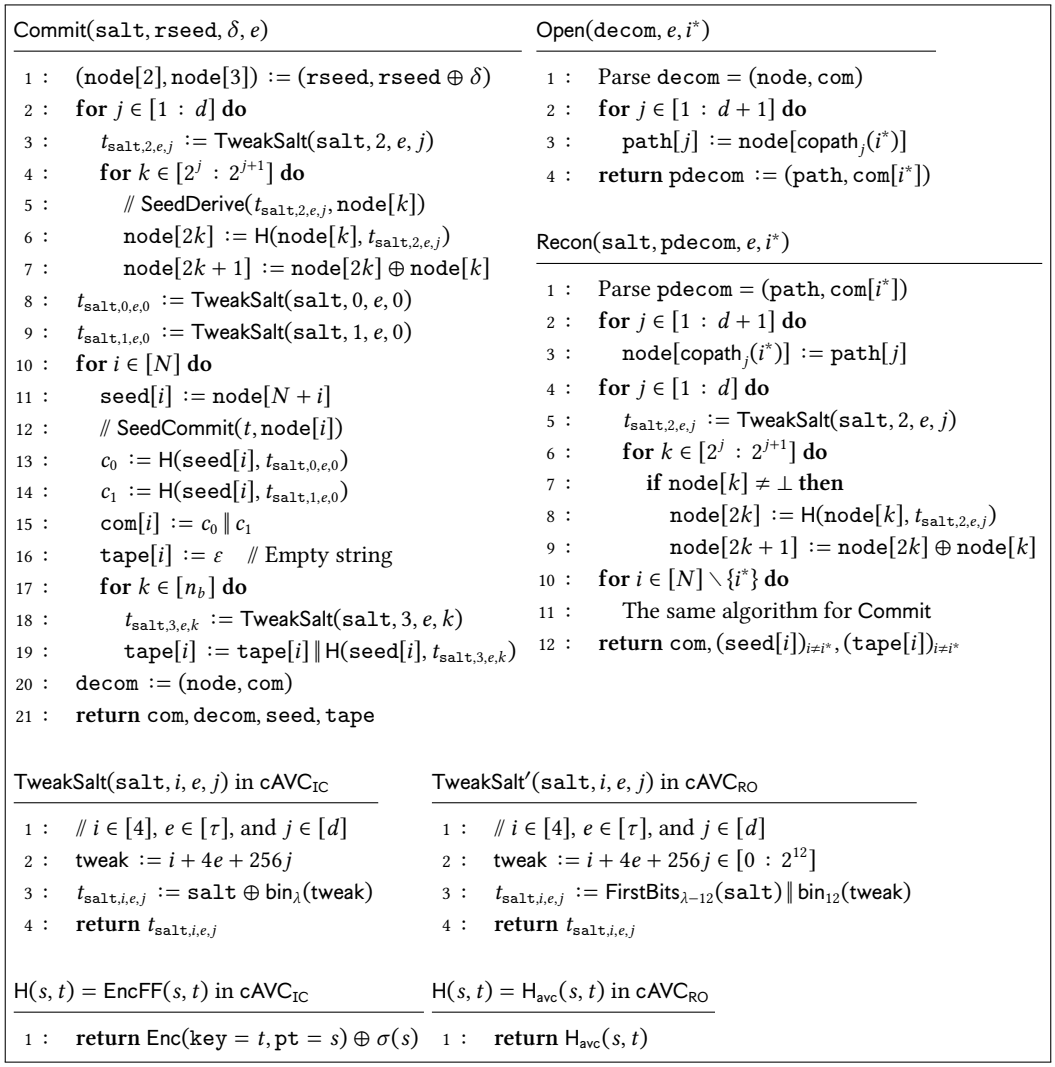

Fig. 3. All-but-one Vector Commitment $\mathsf{cAVC}_{\mathsf{IC}}$ and $\mathsf{cAVC}_{\mathsf{RO}}$ from salted cGGM tree by using an ideal cipher and a random oracle, respectively. In $\mathsf{cAVC}_{\mathsf{IC}}$, $\mathsf{Enc} : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda$ is modeled as an ideal cipher. In $\mathsf{cAVC}_{\mathsf{RO}}$, $\mathsf{H}_{\mathsf{avc}} : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda$ is modeled as a random oracle.

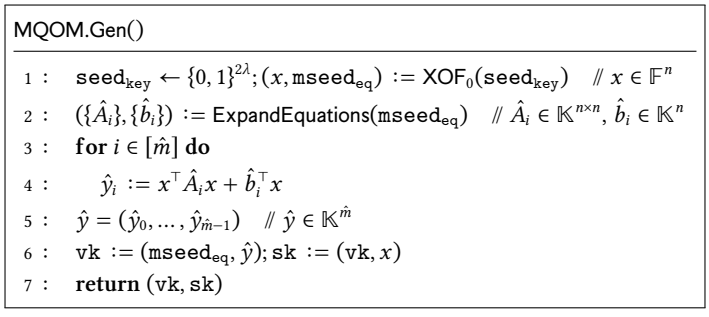

Fig. 4. Key-generation algorithm of MQOM (ver.2.1)

Security notions of key generation: In our security proof for MQOM, we rely on the assumption that the secret key x (or a part of it) cannot be inferred from the public key $(\mathtt{mseed}_{\mathtt{eq}}, \hat{y})$. To formalize this requirement, we introduce a notion of one-wayness of key generation. We begin with the standard notion of one-wayness.

MQOM.Sign($\mathtt{sk}, \mathtt{msg}$)

$(\mathtt{vk}, x) := \mathsf{Parse}(\mathtt{sk}); (\mathtt{mseed}_{\mathtt{eq}}, \hat{y}) := \mathsf{Parse}(\mathtt{vk}); (\{\hat{A}_i\}, \{\hat{b}_i\}) := \mathsf{ExpandEquations}(\mathtt{mseed}_{\mathtt{eq}})$
$\mathtt{mseed} \leftarrow \{0,1\}^{\lambda}; \mathtt{salt} \leftarrow \{0,1\}^{\lambda}$
$h_{\mathtt{msg}} := \mathsf{Hash}_2(\mathtt{msg})$ // $h_{\mathtt{msg}}$ = msg_hash
// $(\mathtt{com}_1, \mathtt{key}, x_0, u_0, u_1) := \mathsf{BLC.Commit}(\mathtt{mseed}_{\mathtt{eq}}, \mathtt{salt}, x)$, where $\mathtt{key} := (\mathtt{node}, \mathtt{com}, \Delta_x^{(1)})$
$(\mathtt{rseed}[0], \ldots, \mathtt{rseed}[\tau-1]) := \mathsf{PRG}(\mathtt{mseed})$
$\delta := \mathsf{FirstBits}_{\lambda}(x)$
for $e \in [\tau]$ **do**
  $(\mathtt{com}[e], \mathtt{decom}[e], \mathtt{seed}[e], \mathtt{tape}[e]) := \mathsf{cAVC.Commit}(\mathtt{salt}, \mathtt{rseed}[e], \delta, e)$
  $h_{\mathtt{com}}[e] := \mathsf{Hash}_6(\mathtt{com}[e])$
  for $i \in [N]$ **do** $(\bar{x}[e][i], \bar{u}[e][i]) := \mathsf{Parse}(\mathtt{seed}[e][i], \mathtt{tape}[e][i])$
  // Compute $P_u(X) = u_0[e] + u_1[e] \cdot X \in (\mathbb{K}[X])^{\eta}$
  $u_0[e] := -\sum_{i \in [N]} \omega_i \cdot \bar{u}[e][i]; u_1[e] := \sum_{i \in [N]} \bar{u}[e][i]$ // $u_0[e], u_1[e] \in \mathbb{K}^{\eta}$
  // Compute $P_x(X) = x_0[e] + x \cdot X \in (\mathbb{K}[X])^n$
  $x_0[e] := -\sum_{i \in [N]} \omega_i \cdot \bar{x}[e][i]$ // $x_0[e] \in \mathbb{K}^n$
  $\Delta_x[e] := x - \sum_{i \in [N]} \bar{x}[e][i]$ // The first λ bits of $\Delta_x[e]$ is 0^{λ}
  $\Delta_x^{(1)}[e] := \mathsf{NextBits}_{\lambda}(\Delta_x[e])$ // Truncate first λ bits of x
$\mathtt{com}_1 := \mathsf{Hash}_7(h_{\mathtt{com}}, \Delta_x^{(1)})$
$\boldsymbol{\Gamma} := \boldsymbol{I}_{\eta} \in \mathbb{K}^{\eta \times \eta}$ // 3-round ver. $\eta = \hat{m} := m/\mu$
or $\boldsymbol{\Gamma} := \mathsf{XOF}_8(\mathtt{com}_1) \in \mathbb{K}^{\eta \times \hat{m}}$ // 5-round ver. $\eta < \hat{m}$
// $(\alpha_0, \alpha_1) := \mathsf{ComputePAlpha}'(\boldsymbol{\Gamma}, x_0, u_0, u_1, x, \{\hat{A}_i\}, \{\hat{b}_i\}, \{\hat{y}_i\})$
for $e \in [\tau]$ **do**
  // $(z_0, z_1) := \mathsf{ComputePz}(x_0[e], x, \{\hat{A}_i\}, \{\hat{b}_i\})$
  for $i \in [\hat{m}]$ **do**
    // Compute $P_{z,i}(X) = z_{0,i} + z_{1,i} \cdot X \in \mathbb{K}[X]$
    $t_0 := \hat{A}_i \cdot x_0[e]; t_1 := \hat{A}_i \cdot x + \hat{b}_i$
    $z_{0,i} := t_0^{\top} \cdot x_0[e]; z_{1,i} := t_0^{\top} \cdot x + t_1^{\top} \cdot x_0[e]$
  $z_0 := (z_{0,0}, \ldots, z_{0,\hat{m}-1}); z_1 := (z_{1,0}, \ldots, z_{1,\hat{m}-1})$
  $\alpha_0[e] := u_0[e] + \boldsymbol{\Gamma} \cdot z_0$ // $\alpha_0[e] \in \mathbb{K}^{\eta}$
  $\alpha_1[e] := u_1[e] + \boldsymbol{\Gamma} \cdot z_1$ // $\alpha_1[e] \in \mathbb{K}^{\eta}$
$\mathtt{com}_2 := \mathsf{Hash}_3(\alpha_0, \alpha_1)$
$h_2 := \mathsf{Hash}_4(\mathtt{vk}, \mathtt{com}_1, \mathtt{com}_2, h_{\mathtt{msg}})$ // h_2 = hash
// $(\mathbf{i}^*, \mathtt{nonce}) := \mathsf{SampleChallenge}(h_2)$
$\mathtt{nonce} := 0$
$(\mathbf{i}^*, v_{\mathrm{grinding}}) := \mathsf{XOF}_5(h_2, \mathtt{nonce})$
while $v_{\mathrm{grinding}} \neq 0^w$ **do**
  $\mathtt{nonce} := \mathtt{nonce} + 1$
  $(\mathbf{i}^*, v_{\mathrm{grinding}}) := \mathsf{XOF}_5(h_2, \mathtt{nonce})$
// $\mathtt{opening} = (\mathtt{pdecom}, \Delta_x^{(1)}) := \mathsf{BLC.Open}(\mathtt{key}, \mathbf{i}^*)$ with $\mathtt{key} = (\mathtt{node}, \mathtt{com}, \Delta_x^{(1)})$
for $e \in [\tau]$ **do**
  // $\mathtt{path}[e] := \mathsf{GGMTree.Open}(\mathtt{node}[e], \mathbf{i}^*[e])$
  $\mathtt{pdecom}[e] := \mathsf{cAVC.Open}(\mathtt{decom}, e, \mathbf{i}^*[e])$
return $\sigma := \mathsf{Serialize}(\mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2, \alpha_1, \mathtt{pdecom}, \Delta_x^{(1)}, \mathtt{nonce})$

Fig. 5. Expanded signing algorithm of MQOM (ver.2.1)

Definition 4.1 (One-wayness). *We say that* MQOM.Gen *is* one-way *if, for any PPT/QPT adversary $\mathcal{A}$, its advantage $\mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}}(\lambda)$ is negligible in λ, where the advantage is defined as follows:*

$$\mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}}(\lambda) := \Pr\left[\begin{matrix}(\mathtt{vk}, \mathtt{sk}) \leftarrow \mathsf{MQOM.Gen}(1^{\lambda}), \\ x' \leftarrow \mathcal{A}(\mathtt{vk})\end{matrix} : \hat{F}(x') = 0\right],$$

where $\mathtt{vk} = (\mathtt{mseed}_{\mathtt{eq}}, \hat{y})$, $\mathtt{sk} = (\mathtt{vk}, x)$, *and* $\hat{F} = \{\hat{f}_i\}$ *is defined as in Equation 1.*

MQOM.Vrfy($\mathtt{vk}, \sigma, \mathtt{msg}$)

$(\mathtt{mseed}_{\mathsf{eq}}, \hat{y}) := \mathsf{Parse}(\mathtt{vk}); (\{\hat{A}_i\}, \{\hat{b}_i\}) := \mathsf{ExpandEquations}(\mathtt{mseed}_{\mathsf{eq}})$
$(\mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2, \alpha_1, \mathtt{pdecom}, \Delta_x^{(1)}, \mathtt{nonce}) := \mathsf{Parse}(\sigma)$
$h_{\mathtt{msg}} := \mathsf{Hash}_2(\mathtt{msg}); h_2 := \mathsf{Hash}_4(\mathtt{vk}, \mathtt{com}_1, \mathtt{com}_2, h_{\mathtt{msg}})$
$(\mathbf{i}^*, v_{\text{grinding}}) := \mathsf{XOF}_5(h_2, \mathtt{nonce})$
if $v_{\text{grinding}} \neq 0^w$ **then return** 0
// $(\mathtt{ret}, x_{\mathtt{eval}}, u_{\mathtt{eval}}) := \mathsf{BLC.Eval}(\mathtt{salt}, \mathtt{com}_1, \mathtt{opening}, \mathbf{i}^*)$,
// where $\mathtt{opening} = (\mathtt{pdecom}, \Delta_x^{(1)})$
for $e \in [\tau]$ **do**
  $(\mathtt{com}[e], (\mathtt{seed}[e][i])_{i \neq \mathbf{i}^*[e]}, (\mathtt{tape}[e][i])_{i \neq \mathbf{i}^*[e]}) := \mathsf{cAVC.Recon}(\mathtt{salt}, \mathtt{pdecom}, e, \mathbf{i}^*[e])$
  $h_{\mathtt{com}}[e] := \mathsf{Hash}_6(\mathtt{com}[e])$
  for $i \in [N] \setminus \{\mathbf{i}^*[e]\}$ **do** $(\bar{x}[e][i], \bar{u}[e][i]) := \mathsf{Parse}(\mathtt{seed}[e][i], \mathtt{tape}[e][i])$
  $\Delta_x[e] := 0^\lambda \,\|\, \Delta_x^{(1)}[e]$
  $x_{\mathtt{eval}}[e] := \omega_{\mathbf{i}^*[e]} \cdot \Delta_x[e] + \sum_{i \neq \mathbf{i}^*[e]} (\omega_{\mathbf{i}^*[e]} - \omega_i) \cdot \bar{x}[e][i]$
  $u_{\mathtt{eval}}[e] := \sum_{i \neq \mathbf{i}^*[e]} (\omega_{\mathbf{i}^*[e]} - \omega_i) \cdot \bar{u}[e][i]$
$\mathtt{com}_1' := \mathsf{Hash}_7(h_{\mathtt{com}}, \Delta_x^{(1)})$
if $\mathtt{com}_1' \neq \mathtt{com}_1$ **then return** 0
// $\alpha_0 := \mathsf{RecomputePAlpha}(\mathtt{com}_1, \alpha_1, x_{\mathtt{eval}}, u_{\mathtt{eval}}, \{\hat{A}_i\}, \{\hat{b}_i\}, \{\hat{y}_i\})$
$\Gamma := I_\eta$ // 3-round ver.
or $\Gamma := \mathsf{XOF}_8(\mathtt{com}_1)$ // 5-round ver.
for $e \in [\tau]$ **do**
  // $z_{\mathtt{eval}} := \mathsf{ComputePzEval}(r, x_{\mathtt{eval}}[e], \{\hat{A}_i\}, \{\hat{b}_i\}, \{\hat{y}_i\})$
  for $i \in [\hat{m}]$ **do**
    // Compute $P_{z,i}(r)$
    $t_{\mathtt{eval}} := \hat{A}_i \cdot x_{\mathtt{eval}}[e] + \hat{b}_i \cdot \omega_{\mathbf{i}^*[e]}$
    $z_{\mathtt{eval},i} := t_{\mathtt{eval}}^\top \cdot x_{\mathtt{eval}}[e] - \hat{y}_i \cdot \omega_{\mathbf{i}^*[e]}^2$
  $z_{\mathtt{eval}} := (z_{\mathtt{eval},0}, \ldots, z_{\mathtt{eval},\hat{m}-1})$
  $\alpha_{\mathtt{eval}} := u_{\mathtt{eval}} + \Gamma \cdot \phi(z_{\mathtt{eval}})$
  $\alpha_0[e] := \alpha_{\mathtt{eval}} - \alpha_1[e] \cdot \omega_{\mathbf{i}^*[e]}$
$\mathtt{com}_2' := \mathsf{Hash}_3(\alpha_0, \alpha_1)$
if $\mathtt{com}_2' \neq \mathtt{com}_2$ **then return** 0
return 1

Fig. 6. Expanded verification algorithm of MQOM (ver.2.1)

We next define the partial-domain one-wayness for the case where x is longer than λ.

Definition 4.2 (Partial-domain one-wayness [FOPS01, FOPS04], adapted). *We say that* MQOM.Gen *is* partial-domain one-way *if, for any PPT/QPT adversary $\mathcal{A}$, its advantage $\mathsf{Adv}^{\text{pdow}}_{\text{Gen},\mathcal{A}}(\lambda)$ is negligible in λ, where the advantage is defined as follows:*

$$\mathsf{Adv}^{\text{pdow}}_{\text{Gen},\mathcal{A}}(\lambda) := \Pr\left[\begin{matrix}(\mathtt{vk}, \mathtt{sk}) \leftarrow \mathsf{MQOM.Gen}(1^\lambda), \\ \delta' \leftarrow \mathcal{A}(\mathtt{vk})\end{matrix} : \delta' = \mathsf{FirstBits}_\lambda(x)\right],$$

where $\mathtt{vk} = (\mathtt{mseed}_{\mathsf{eq}}, \hat{y})$, $\mathtt{sk} = (\mathtt{vk}, x)$.

We also consider the following variants of one-wayness, since the standard notion is not directly applicable to our MQOM security proofs: in the partial-domain one-wayness reduction, the simulator lacks an efficient way to check whether it is the correct solution and we want to introduce the way to check the validity of the candidate solution.

Definition 4.3 (Domain-extension one-wayness). *Let* MQOM.Gen$'$ *denote a modified key-generation algorithm for* MQOM*, where the first λ bits of x are randomly chosen as δ, $K \leftarrow \{0,1\}^\lambda$, and $x = \mathsf{PRF}(K, \delta)$. We say that*

$\mathsf{MQOM.Gen}'$ *is* domain-extension one-way *if, for any PPT/QPT adversary* $\mathcal{A}$*, its advantage* $\mathsf{Adv}^{\mathrm{pdow}}_{\mathsf{MQOM.Gen}',\mathcal{A}}(\lambda)$ *is negligible in* λ*, where the advantage is defined as follows:*

$$\mathsf{Adv}^{\mathrm{deow}}_{\mathsf{Gen}',\mathcal{A}}(\lambda) := \Pr\left[\begin{matrix}(\mathtt{vk}', \mathtt{sk}) \leftarrow \mathsf{MQOM.Gen}'(1^\lambda), \\ \delta' \leftarrow \mathcal{A}(\mathtt{vk}')\end{matrix} : \hat{F}(\mathsf{PRF}(K, \delta')) = 0\right],$$

where $\mathtt{vk}' = (\mathtt{mseed}_{\mathsf{eq}}, \hat{y}, K)$, $\mathtt{sk} = (\mathtt{vk}, x)$*, and* $\hat{F} = \{\hat{f}_i\}$ *is defined as in Equation 1.*

In the domain-extension setting, the reduction can efficiently verify a candidate δ' and hence single out a correct solution among potentially many preimages. However, for our QROM proof we also need to single out a unique solution among these candidates. This motivates introducing the following variant, which provides just enough auxiliary information to make a single solution efficiently identifiable.

Definition 4.4 (Domain-extension/auxiliary-input one-wayness). *Let* $\mathsf{H}_{\mathsf{aux}} : \mathbb{F}^n \to \{0,1\}^\lambda$ *be a random function. We say that* $\mathsf{MQOM.Gen}'$ *(as defined in Definition 4.3) is* domain-extension/auxiliary-input one-way *if, for any PPT/QPT adversary* $\mathcal{A}$*, its advantage* $\mathsf{Adv}^{\mathrm{daow}}_{(\mathsf{Gen}',\mathsf{H}_{\mathsf{aux}}),\mathcal{A}}(\lambda)$ *is negligible in* λ*, where the advantage is defined as follows:*

$$\mathsf{Adv}^{\mathrm{daow}}_{(\mathsf{Gen}',\mathsf{H}_{\mathsf{aux}}),\mathcal{A}}(\lambda) := \Pr\left[\begin{matrix}(\mathtt{vk}', \mathtt{sk}) \leftarrow \mathsf{MQOM.Gen}'(1^\lambda), \\ z := \mathsf{H}_{\mathsf{aux}}(x), \delta' \leftarrow \mathcal{A}^{|\mathsf{H}_{\mathsf{aux}}\rangle}(\mathtt{vk}', z)\end{matrix} : \begin{matrix}\hat{F}(\mathsf{PRF}(K, \delta')) = 0 \\ \wedge \mathsf{H}_{\mathsf{aux}}(\mathsf{PRF}(K, \delta')) = z\end{matrix}\right],$$

where $\mathtt{vk}' = (\mathtt{mseed}_{\mathsf{eq}}, \hat{y}, K)$, $\mathtt{sk} = (\mathtt{vk}, x)$*, and* $\hat{F} = \{\hat{f}_i\}$ *is defined as in Equation 1.*

5 EUF-CMA Security of MQOM_1 in the ROM/QROM

5.1 Modification on MQOM in MQOM_1

To simplify the security proof, we make the following modifications to MQOM in MQOM_1:

- We consider the 3-round version of MQOM.
- The input to Hash_4 (which computes h_2) includes the salt $\mathtt{salt}$.
- To make simple domain separation[6] feasible in $\mathsf{H}_{\mathsf{avc}}$, we replace $\mathsf{TweakSalt}$ with $\mathsf{TweakSalt}'$ defined as in Figure 3.

5.2 EUF-CMA Security of MQOM_1 in the ROM

We give the first formal security proof of the MQOM_1, modeling EncEF as a classical random oracle $\mathsf{H}_{\mathsf{avc}}$.

Theorem 5.1 (EUF-NMA to EUF-CMA of MQOM_1 in ROM). *Let* $\mathsf{H}_{\mathsf{avc}}$, Hash_2, *and* Hash_4 *be random oracles. For a QPT adversary* $\mathcal{A}$ *that queries the oracles* X *at most* Q_X *times, there exist QPT adversaries* $\mathcal{A}_{\mathrm{nma}}$, $\mathcal{A}_{\mathrm{pdow}}$, *and* $\mathcal{A}_{\mathrm{prg}}$ *such that*

$$\begin{aligned}\mathsf{Adv}^{\text{euf-cma}}_{\mathsf{MQOM}_1,\mathcal{A}}(\lambda) \le \mathsf{Adv}^{\text{euf-nma}}_{\mathsf{MQOM}_1,\mathcal{A}_{\mathrm{nma}}}(\lambda) + Q_{\mathsf{H}_{\mathsf{avc}}} \cdot \mathsf{Adv}^{\mathrm{pdow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{pdow}}}(\lambda) + Q_{\mathsf{Sign}} \cdot \mathsf{Adv}^{\mathrm{prg}}_{\mathsf{PRG},\mathcal{A}_{\mathrm{prg}}}(\lambda) \\ + Q^2_{\mathsf{Sign}}/2^{\lambda-12} + Q_{\mathsf{Sign}}(Q_{\mathsf{Sign}} + Q_{\mathsf{Hash}_4})/2^\lambda + Q^2_{\mathsf{Hash}_2}/2^{2\lambda}.\end{aligned}$$

The running times of $\mathcal{A}_{\mathrm{nma}}$, $\mathcal{A}_{\mathrm{pdow}}$, *and* $\mathcal{A}_{\mathrm{prg}}$ *are approximately that of* $\mathcal{A}$*, and the number of queries of* $\mathcal{A}_{\mathrm{nma}}$ *is the same as the one of* $\mathcal{A}$*.*

Moreover, there exist QPT adversaries $\mathcal{A}_{\mathrm{deow}}$ *and* $\mathcal{A}_{\mathrm{prf}}$ *such that the above bound also holds after replacing the term* $Q_{\mathsf{H}_{\mathsf{avc}}} \cdot \mathsf{Adv}^{\mathrm{pdow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{pdow}}}(\lambda)$ *with* $\mathsf{Adv}^{\mathrm{deow}}_{\mathsf{Gen}',\mathcal{A}_{\mathrm{deow}}}(\lambda) + 2 \cdot \mathsf{Adv}^{\mathrm{pr}}_{\mathsf{PRF},\mathcal{A}_{\mathrm{prf}}}(\lambda)$*.*

Proof. Following the game-based proof technique of [KX25], we construct a sequence of games $\mathsf{G}_0, \mathsf{G}_1, \ldots, \mathsf{G}_4$ and denote by W_i the winning event in G_i. (See Figure 7 for the detailed game descriptions.)

- Game 0: The original EUF-CMA security game.

[6] We use domain separation only in the EUF-NMA security proof.

G_0-G_4

$Q := \emptyset$
$Q_{\mathsf{salt}} := \emptyset$ // $\mathsf{G}_{0.2}$-G_4
$(\mathtt{vk}, \mathtt{sk}) \leftarrow \mathsf{Gen}(1^\lambda)$
for $q \in [Q_{\mathsf{Sign}}]$ **do** // G_2-G_4
  $\mathtt{mseed}^{(q)} \leftarrow \{0,1\}^\lambda$ // G_2
  $\mathtt{salt}^{(q)} \leftarrow \{0,1\}^\lambda$ // G_2-G_4
  if $\mathsf{FirstBits}_{\lambda-12}(\mathtt{salt}^{(q)}) \in Q_{\mathsf{salt}}$ **then abort**; $Q_{\mathsf{salt}} := Q_{\mathsf{salt}} \cup \{\mathsf{FirstBits}_{\lambda-12}(\mathtt{salt}^{(q)})\}$ // G_2-G_4
  $(\mathtt{rseed}^{(q)}[0], \ldots, \mathtt{rseed}^{(q)}[\tau-1]) := \mathsf{PRG}(\mathtt{mseed}^{(q)}, \tau\lambda)$ // G_2
  $(\mathtt{rseed}^{(q)}[0], \ldots, \mathtt{rseed}^{(q)}[\tau-1]) \leftarrow (\{0,1\}^\lambda)^\tau$ // $\mathsf{G}_{2.1}$-G_4
  $h_2^{(q)} \leftarrow \{0,1\}^{2\lambda}$ // G_2-G_4
  $(\mathbf{i}^{*(q)}, \mathtt{nonce}^{(q)}) := \mathsf{SampleChallenge}(h_2^{(q)})$ // G_2-G_4
  for $e \in [\tau]$ **do** // G_2-G_4
    $(\mathtt{com}^{(q)}[e], \mathtt{decom}^{(q)}[e], \mathtt{seed}^{(q)}[e], \mathtt{tape}^{(q)}[e])$
      $:= \mathsf{cAVC}_{\mathsf{RO}}.\mathsf{Commit}(\mathtt{salt}^{(q)}, \mathtt{rseed}^{(q)}[e], x, e)$ // G_2-$\mathsf{G}_{2.2}$
      $:= \mathsf{cAVC}'_{\mathsf{RO}}.\mathsf{Commit}(\mathtt{salt}^{(q)}, \mathtt{rseed}^{(q)}[e], x, e, \mathbf{i}^{*(q)}[e])$ // G_3-G_4
    $(\mathtt{node}^{(q)}[e], \mathtt{com}^{(q)}[e]) := \mathsf{Parse}(\mathtt{decom}^{(q)}[e])$
$O := \{\mathsf{H}_{\mathsf{avc}}, \mathsf{Hash}_2, \mathsf{Hash}_4\}$
$(\mathtt{msg}^*, \sigma^*) \leftarrow \mathcal{A}^{\mathrm{Sign},|O\rangle}(\mathtt{vk})$ // See Figure 8 for Sign
if $\mathtt{msg}^* \in Q$ **then return** 0
if $\exists q : \mathsf{Hash}_2(\mathtt{msg}^*) = \mathsf{Hash}_2(\mathtt{msg}^{(q)})$ **then abort** // $\mathsf{G}_{0.1}$-G_4
return $\mathsf{V}(\mathtt{vk}, \mathtt{msg}^*, \sigma^*)$

Fig. 7. Games for proving EUF-NMA $\Rightarrow$ EUF-CMA of MQOM_1

- Game 0.1: In the modification of Game 0.1, we introduce an abort condition tied to the forgery message $\mathtt{msg}^*$: the game aborts if there exists a signing-query message $\mathtt{msg}^{(q)}$ such that $\mathsf{Hash}_2(\mathtt{msg}^*) = \mathsf{Hash}_2(\mathtt{msg}^{(q)})$. Since $\mathtt{salt}$ is not included in the Hash_2's input, this probability can only be bounded by the adversary's collision-finding probability. Therefore, we have $|\Pr[\mathsf{G}_0 : W_0] - \Pr[\mathsf{G}_{0.1} : W_{0.1}]| \leq Q^2_{\mathsf{Hash}_2}/2^{2\lambda}$.
- Game 0.2: The game aborts if there is a collision among the values $\mathsf{FirstBits}_{\lambda-12}(\mathtt{salt})$ used across signing queries. The collision probability is bounded by $Q^2_{\mathsf{Sign}}/2^{\lambda-12}$; therefore, $|\Pr[\mathsf{G}_{0.1} : W_{0.1}] - \Pr[\mathsf{G}_{0.2} : W_{0.2}]| \leq Q^2_{\mathsf{Sign}}/2^{\lambda-12}$.
- Game 1: In this game, the signing oracle begins by choosing h_2 uniformly at random and reprogramming it as the output of Hash_4. Since the computation of h_2 includes the salt $\mathtt{salt}$, any input relevant to this reprogramming is unpredictable to the adversary before the oracle evaluates Hash_4 on that input. Therefore, unless the adversary has previously queried the corresponding input to Hash_4, its view remains unchanged. Therefore, we have
$$\left|\Pr[\mathsf{G}_{0.2} : W_{0.2}] - \Pr[\mathsf{G}_1 : W_1]\right| \leq Q_{\mathsf{Sign}}(Q_{\mathsf{Sign}} + Q_{\mathsf{Hash}_4})/2^\lambda.$$
- Game 2: We modify the signing oracle so that it computes $(\mathbf{i}^*, \mathtt{nonce})$ at the beginning of the signing oracle. Since h_2 is chosen uniformly at random and $(\mathbf{i}^*, \mathtt{nonce})$ is deterministically derived from h_2, this modification is purely conceptual and does not change the adversary's view. Thus, $\Pr[\mathsf{G}_1 : W_1] = \Pr[\mathsf{G}_2 : W_2]$.
- Game 2.1: We replace the value $\mathtt{rseed}$ output by $\mathsf{PRG}(\mathtt{mseed})$ with a uniformly random $\mathtt{rseed}$. Using a standard hybrid argument over the Q_{Sign} signing queries, this step reduces to the security of PRG (which is keyed by the hidden seed $\mathtt{mseed}$). Hence, $\left|\Pr[\mathsf{G}_2 : W_2] - \Pr[\mathsf{G}_{2.1} : W_{2.1}]\right| \leq Q_{\mathsf{Sign}} \cdot \mathsf{Adv}^{\mathrm{prg}}_{\mathsf{PRG},\mathcal{A}_{\mathrm{prg}}}(\lambda)$.
- Game 2.2: The signing oracle computes α_1 in a converse manner. Specifically, it first computes α_0 and $\alpha_{\mathtt{eval}}$, and then sets $\alpha_1[e] := r^{-1} \cdot (\alpha_{\mathtt{eval}} - \alpha_0[e])$, where $r = \omega_{\mathbf{i}^*[e]}$ is defined as in the verification algorithm. This modification allows the signing oracle to discard the values $u_1[e]$ and z_1. Since this is purely a conceptual change, we have $\Pr[\mathsf{G}_{2.1} : W_{2.1}] = \Pr[\mathsf{G}_{2.2} : W_{2.2}]$.
- Game 2.3: We replace $\mathsf{H}_{\mathsf{avc}}$ with a modified version $\mathsf{H}'_{\mathsf{avc}}$. We first define a predicate P_δ for $\delta \in \{0,1\}^\lambda$ as follows:
$$\mathsf{P}_\delta(\mathtt{node}, t_{\mathtt{salt}}, q, e, i, j) := \left(\mathtt{node} = \delta \oplus \bigoplus_{j' \in [1:\ell+1]} \mathtt{node}^{(q)}[e][\mathsf{copath}_{j'}(i^*)]\right) \wedge (t_{\mathtt{salt}} = \mathsf{TweakSalt}'(\mathtt{salt}^{(q)}, i, e, j)), \tag{2}$$

$\textsc{Sign}(\mathtt{msg}^{(q)})$

1 : $(\mathtt{vk}, x) := \mathsf{Parse}(\mathtt{sk}); (\mathtt{mseed}_{\mathsf{eq}}, y) := \mathsf{Parse}(\mathtt{vk}), (\{A_i\}, \{b_i\}) := \mathsf{ExpandEquations}(\mathtt{mseed}_{\mathsf{eq}})$
2 : $\mathtt{mseed}^{(q)} \leftarrow \{0,1\}^{\lambda}; \mathtt{salt}^{(q)} \leftarrow \{0,1\}^{\lambda}$ // G_0-G_1
3 : **if** $\mathsf{FirstBits}_{\lambda-12}(\mathtt{salt}^{(q)}) \in Q_{\mathrm{salt}}$ **then abort**; $Q_{\mathrm{salt}} := Q_{\mathrm{salt}} \cup \{\mathsf{FirstBits}_{\lambda-12}(\mathtt{salt}^{(q)})\}$ // $\mathsf{G}_{0.2}$-G_1
4 : $h_{\mathtt{msg}} := \mathsf{Hash}_2(\mathtt{msg}^{(q)})$
5 : $(\mathtt{rseed}^{(q)}[0], \ldots, \mathtt{rseed}^{(q)}[\tau-1]) := \mathsf{PRG}(\mathtt{mseed}^{(q)}, \tau\lambda)$ // G_0-G_1
6 : $\delta := \mathsf{FirstBits}_{\lambda}(x)$
7 : **for** $e \in [\tau]$ **do**
8 : $(\mathtt{com}^{(q)}[e], \mathtt{decom}^{(q)}[e], \mathtt{seed}^{(q)}[e], \mathtt{tape}^{(q)}[e]) = \mathsf{cAVC}_{\mathsf{RO}}.\mathsf{Commit}(\mathtt{salt}^{(q)}, \mathtt{rseed}_e^{(q)}, \delta, e)$ // G_0-G_1
9 : $(\mathtt{node}^{(q)}[e], \mathtt{com}^{(q)}[e]) := \mathsf{Parse}(\mathtt{decom}^{(q)}[e])$ // G_0-G_1
10 : $h_{\mathtt{com}}[e] := \mathsf{Hash}_6(\mathtt{com}^{(q)}[e])$
11 : **for** $i \in [N]$ **do** $(\bar{x}[e][i], \bar{u}[e][i]) := (\mathtt{seed}^{(q)}[e][i], \mathtt{tape}^{(q)}[e][i])$
12 : $r := \omega_{\mathtt{i}^{*(q)}[e]}$ // $\mathsf{G}_{2.2}$-G_4
13 : $u_0[e] := -\sum_{i\in[N]} \omega_i \cdot \bar{u}[e][i]$ // G_0-$\mathsf{G}_{3.1}$
14 : $u_1[e] := \sum_{i\in[N]} \bar{u}[e][i]$ // G_0-$\mathsf{G}_{2.1}$
15 : $u_{\mathtt{eval}}[e] := \sum_{i\neq \mathtt{i}^{*(q)}[e]} (r-\omega_i) \cdot \bar{u}[e][i]$ // $\mathsf{G}_{2.2}$-G_4
16 : $x_0[e] := -\sum_{i\in[N]} \omega_i \cdot \bar{x}[e][i]$
17 : $\Delta_x[e] := x - \sum_{i\in[N]} \bar{x}[e][i]; \Delta_x^{(1)}[e] := (\Delta_x[e])[\lambda:]$ // G_0-$\mathsf{G}_{3.1}$
18 : $\Delta_x^{(1)}[e] \leftarrow \{0,1\}^{|x|_2-\lambda}$ // G_4
19 : $x_{\mathtt{eval}}[e] := \sum_{i\neq \mathtt{i}^{*(q)}[e]} (r-\omega_i) \cdot \bar{x}[e][i]$ // $\mathsf{G}_{2.2}$-G_4
20 : $\mathtt{com}_1 := \mathsf{Hash}_7(h_{\mathtt{com}}, \Delta_x^{(1)}); \boldsymbol{\Gamma} := \boldsymbol{I}_\eta$
21 : **for** $e \in [\tau]$ **do**
22 : $r := \omega_{\mathtt{i}^{*(q)}[e]}$ // $\mathsf{G}_{2.2}$-G_4
23 : **for** $i \in [\hat{m}]$ **do**
24 : $t_0 := \hat{A}_i \cdot x_0[e]$
25 : $t_1 := \hat{A}_i \cdot x + b_i$ // G_0-$\mathsf{G}_{2.1}$
26 : $z_{0,i} := t_0^T \hat{A}_i \cdot x_0[e]$
27 : $z_{1,i} := t_0^T \cdot x + t_1^T \cdot x_0[e]$ // G_0-$\mathsf{G}_{2.1}$
28 : $z_{\mathtt{eval},i} = x_{\mathtt{eval}}[e]^\top \cdot \hat{A}_i \cdot x_{\mathtt{eval}}[e] + \hat{b}_i^\top \cdot x_{\mathtt{eval}}[e] \cdot r - \hat{y}_i \cdot r^2$ // $\mathsf{G}_{2.2}$-G_4
29 : $z_0 = (z_{0,0}, \ldots, z_{0,\hat{m}-1})$
30 : $z_{\mathtt{eval}} = (z_{\mathtt{eval},0}, \ldots, z_{\mathtt{eval},\hat{m}-1})$ // $\mathsf{G}_{2.2}$-G_4
31 : $\alpha_0[e] := u_0[e] + \boldsymbol{\Gamma} \cdot \phi(z_0)$ // G_0-$\mathsf{G}_{3.1}$
32 : $\alpha_0[e] \leftarrow \mathbb{K}^\eta$ // G_4
33 : $\alpha_1[e] := u_1[e] + \boldsymbol{\Gamma} \cdot \phi(z_1)$ // G_0-$\mathsf{G}_{2.1}$
34 : $\alpha_{\mathtt{eval}}[e] := u_{\mathtt{eval}} + \boldsymbol{\Gamma} \cdot \phi(z_{\mathtt{eval}}); \alpha_1[e] := r^{-1} \cdot (\alpha_{\mathtt{eval}}[e] - \alpha_0[e])$ // $\mathsf{G}_{2.2}$-G_4
35 : $\mathtt{com}_2 := \mathsf{Hash}_3(\alpha_0, \alpha_1)$
36 : $h_2^{(q)} := \mathsf{Hash}_4(\mathtt{vk}, \mathtt{salt}^{(q)}, \mathtt{com}_1, \mathtt{com}_2, h_{\mathtt{msg}})$ // G_0-$\mathsf{G}_{0.1}$
37 : $h_2^{(q)} \leftarrow \{0,1\}^{2\lambda}$ // G_1
38 : **for** $e \in [\tau]$ **do**
39 : $(\mathtt{i}^{*(q)}, \mathtt{nonce}^{(q)}) := \mathsf{SampleChallenge}(h_2^{(q)})$ // G_0-G_1
40 : **for** $e \in [\tau]$ **do**
41 : $\mathtt{pdecom}[e] := \mathsf{cAVC}_{\mathsf{RO}}.\mathsf{Open}(\mathtt{decom}^{(q)}, e, \mathtt{i}^{*(q)}[e])$ // G_0-$\mathsf{G}_{2.3}$
42 : $\mathtt{pdecom}[e] \leftarrow \{0,1\}^{(d+1)\lambda}$ // G_3-G_4
43 : $\mathtt{opening} := (\mathtt{pdecom}, \Delta_x^{(1)})$
44 : $\sigma^{(q)} := \mathsf{Serialize}(\mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2, \alpha_1, \mathtt{opening}, \mathtt{nonce}^{(q)})$
45 : $\mathsf{Hash}_4 := \mathsf{Hash}_4[(\mathtt{vk}, \mathtt{salt}^{(q)}, \mathtt{com}_1, \mathtt{com}_2, h_{\mathtt{msg}}) \mapsto h_2^{(q)}]$ // G_1-G_4
46 : $Q := Q \cup \{\mathtt{msg}^{(q)}\}$
47 : **return** $\sigma^{(q)}$

Fig. 8. Signing oracle of Figure 7

$\mathsf{cAVC}'_{\mathsf{RO}}.\mathsf{Commit}(\mathtt{salt}, \mathtt{rseed}, \delta, e, \mathbf{i}^*[e])$

$\mathtt{hidden} := \mathsf{Parents}(\mathbf{i}^*[e])$
$(\mathtt{node}[2], \mathtt{node}[3]) := (\mathtt{rseed}, \mathtt{rseed} \oplus \delta)$
if $2 \in \mathtt{hidden}$ **then** $(\mathtt{node}[2], \mathtt{node}[3]) := (\mathtt{node}[3], \mathtt{node}[2])$ // $\mathsf{G}_{3.1}$-G_4
for $j \in [1 : d]$ **do**
  $t_{\mathtt{salt},2,e,j} := \mathsf{TweakSalt}'(\mathtt{salt}, 2, e, j)$
  for $k \in [2^j : 2^{j+1}]$ **do**
    if $k \in \mathtt{hidden}$ **then** $\mathtt{node}[2k] \leftarrow \{0,1\}^\lambda$
    else then $\mathtt{node}[2k] := \mathsf{H}_{\mathsf{avc}}(\mathtt{node}[k], t_{\mathtt{salt},2,e,j})$
    $\mathtt{node}[2k+1] := \mathtt{node}[2k] \oplus \mathtt{node}[k]$
  if $2k \in \mathtt{hidden}$ **then** $(\mathtt{node}[2k], \mathtt{node}[2k+1]) := (\mathtt{node}[2k+1], \mathtt{node}[2k])$ // $\mathsf{G}_{3.1}$-G_4
$t_{\mathtt{salt},0,e,0} := \mathsf{TweakSalt}'(\mathtt{salt}, 0, e, 0)$; $t_{\mathtt{salt},1,e,0} := \mathsf{TweakSalt}'(\mathtt{salt}, 1, e, 0)$
for $i \in [N]$ **do**
  $\mathtt{seed}[i] := \mathtt{node}[N+i]$
  if $i = \mathbf{i}^*[e]$ **then**
    $\mathtt{com}[i] \leftarrow \{0,1\}^\lambda$
    $\mathtt{tape}[i] \leftarrow (\{0,1\}^\lambda)^{n_b}$
  else then
    $\mathtt{com}[i] := \mathsf{H}_{\mathsf{avc}}(\mathtt{seed}[i], t_{\mathtt{salt},0,e,0}) \| \mathsf{H}_{\mathsf{avc}}(\mathtt{seed}[i], t_{\mathtt{salt},1,e,0})$
    for $k \in [n_b]$ **do**
      $t_{\mathtt{salt},3,e,k} := \mathsf{TweakSalt}'(\mathtt{salt}, 3, e, k)$
      $\mathtt{tape}[i] := \mathtt{tape}[i] \| \mathsf{H}_{\mathsf{avc}}(\mathtt{seed}[i], t_{\mathtt{salt},3,e,k})$
$\mathtt{decom} := (\mathtt{node}, \mathtt{com})$
return $\mathtt{com}, \mathtt{decom}, \mathtt{seed}, \mathtt{tape}$

Fig. 9. $\mathsf{cAVC}'_{\mathsf{RO}}.\mathsf{Commit}$ used in Games G_3–G_4

where $i^* := \mathbf{i}^{*(q)}[e]$ and $\ell := j$ if $i = 2$ and d otherwise. On inputs belonging to the set $\mathcal{S}$ defined below, the output is independently resampled uniformly at random, while $\mathsf{H}'_{\mathsf{avc}}(\mathtt{node}, t_{\mathtt{salt}}) = \mathsf{H}_{\mathsf{avc}}(\mathtt{node}, t_{\mathtt{salt}})$ holds for $(\mathtt{node}, t_{\mathtt{salt}}) \notin \mathcal{S}$.

$$\mathcal{S} := \left\{ (\mathtt{node}, t_{\mathtt{salt}}) : \begin{array}{l} \exists q \in [Q_{\mathsf{Sign}}],\ \exists e \in [\tau],\ \exists (i,j) \in \mathcal{I} \\ \text{s.t. } \mathsf{P}_\delta(\mathtt{node}, t_{\mathtt{salt}}, q, e, i, j) = 1 \end{array} \right\},$$

where the definitions of the functions and related notation are in Section 3.

In this game hop, we aim to replace $\mathsf{H}_{\mathsf{avc}}$ with $\mathsf{H}'_{\mathsf{avc}}$ on all inputs corresponding to nodes on the hidden path. Recall that $t_{\mathtt{salt}}$ is a tweaked salt value: by checking whether $t_{\mathtt{salt}} = \mathsf{TweakSalt}'(\mathtt{salt}^{(q)}, i, e, j)$ for some signing query, we identify the indices (q, e), while the selector i specifies the context (tree expansion for $i = 2$, commitment generation for $i \in \{0,1\}$, and tape generation for $i = 3$). These indices fix the relevant path information, so that XOR-ing the queried $\mathtt{node}$ with the associated copath nodes yields the offset δ whenever $\mathtt{node}$ is a value actually used in the signing query; therefore, $\mathcal{S}$ indeed captures the hidden-path query points.

We define the event Bad as the event that the adversary queries $\mathsf{H}_{\mathsf{avc}}$ on $\mathcal{S}$. When Bad does not occur, the replacement to $\mathsf{H}'_{\mathsf{avc}}$ is feasible. Thus, we have

$$|\Pr[\mathsf{G}_{2.2} : W_{2.2}] - \Pr[\mathsf{G}_{2.3} : W_{2.3}]| \le \Pr[\mathsf{G}_{2.3} : \mathsf{Bad}].$$

We postpone bounding $\Pr[\mathsf{G}_{2.3} : \mathsf{Bad}]$ until after presenting the full sequence of game hops.

– Game 3: We replace $\mathsf{cAVC}_{\mathsf{RO}}.\mathsf{Commit}$ with a modified version $\mathsf{cAVC}'_{\mathsf{RO}}.\mathsf{Commit}$, which independently samples $\mathtt{node}[2k]$ for all $k \in \mathtt{hidden}$, as well as $\mathtt{com}[\mathbf{i}^*[e]]$ and $\mathtt{tape}[\mathbf{i}^*[e]]$ at random.

We show that Games 2.3 and 3 are identical; the only subtlety is that $\mathcal{S}$ is defined via $\mathtt{node}$ that we modify in this game hop. The key observation is that in the preceding hop the oracle is already replaced so that the adversary interacts with $\mathsf{H}'_{\mathsf{avc}}$, which returns independently uniform values on inputs in $\mathcal{S}$. We therefore fix in advance a table of independent uniform values for the $\mathcal{S}$-points and couple the two games by using this same table in both: in Game 2.3, whenever $\mathsf{cAVC}_{\mathsf{RO}}.\mathsf{Commit}$ invokes $\mathsf{H}_{\mathsf{avc}}$ on a point in $\mathcal{S}$, it receives the

corresponding table entry, while in Game 3, $\mathsf{cAVC}'_{\mathsf{RO}}.\mathsf{Commit}$ uses the same entries. Under this coupling, the resulting transcripts are identical, implying $\Pr[\mathsf{G}_{2.3} : W_{2.3}] = \Pr[\mathsf{G}_3 : W_3]$ and $\Pr[\mathsf{G}_{2.3} : \mathsf{Bad}] = \Pr[\mathsf{G}_3 : \mathsf{Bad}]$.

– Game 3.1: In this game, we modify the way node values are assigned. Originally, the left node is set to a random value, and the right node is computed as the XOR of a random value and its parent node. However, if the left node belongs to the hidden path, we switch this relationship. Consequently, the node on the opening path is always assigned a random value, while the node on the hidden path is always computed as the XOR of a random value and its parent node.

  The adversary cannot detect that this swap has been performed. This is because, in Game 3, the value on the opening path is masked by the adjacent hidden path value, making it statistically indistinguishable from a random value. Therefore, we have $\Pr[\mathsf{G}_3 : W_3] = \Pr[\mathsf{G}_{3.1} : W_{3.1}]$ and $\Pr[\mathsf{G}_3 : \mathsf{Bad}] = \Pr[\mathsf{G}_{3.1} : \mathsf{Bad}]$.

– Game 4: The signing oracle replaces $\Delta_x^{(1)}[e]$ and $\alpha_0[e]$ with random ones and forget $u_0[e]$ for all $e \in [\tau]$. Since $\Delta_x^{(1)}[e]$ and $u_0[e]$ are masked with the hidden random value $\mathtt{tape}[\mathbf{i}^*[e]]$, they can be statistically randomized. In addition, $\alpha_0[e]$ is masked with $u_0[e]$; therefore it is also randomized. Hence, there is no difference between Games 3.1 and 4: $\Pr[\mathsf{G}_{3.1} : W_{3.1}] = \Pr[\mathsf{G}_4 : W_4]$ and $\Pr[\mathsf{G}_{3.1} : \mathsf{Bad}] = \Pr[\mathsf{G}_4 : \mathsf{Bad}]$.

We show that $\Pr[\mathsf{G}_4 : W_4] \le \mathsf{Adv}^{\text{euf-nma}}_{\mathsf{MQOM}_1,\mathcal{A}_{\text{nma}}}(\lambda)$. In Game 4, $\mathcal{A}_{\text{nma}}$ efficiently simulates the signing oracle by sampling h_2, $\mathtt{pdecom}$, $\Delta_x^{(1)}$, and α_0 as in the game, and answers random-oracle queries using its oracle access; in particular, it programs Hash_4 by overwriting each input queried internally during signing with an independent uniform value. This programming does not affect verification of the final forgery, since verification queries Hash_4 only on the input determined by $(\mathtt{msg}^*, \sigma^*)$, while any previously signed message is rejected in the EUF-CMA game, and we abort if $\mathsf{Hash}_2(\mathtt{msg}^*) = \mathsf{Hash}_2(\mathtt{msg}^{(q)})$ for some signing query $\mathtt{msg}^{(q)}$ (so $h_{\mathtt{msg}^*}$ is fresh). Therefore, whenever $\mathcal{A}$ wins in G_4 and no abort occurs, $\mathcal{A}_{\text{nma}}$ wins the EUF-NMA experiment, proving the reduction.

Then, we give a bound on $\left|\Pr[\mathsf{G}_{2.2} : W_{2.2}] - \Pr[\mathsf{G}_{2.3} : W_{2.3}]\right|$.

Lemma 5.1. *There exists a partial-domain one-way adversary $\mathcal{A}_{\text{pdow}}$ such that*

$$\left|\Pr[\mathsf{G}_{2.2} : W_{2.2}] - \Pr[\mathsf{G}_{2.3} : W_{2.3}]\right| \le Q_{\mathsf{H}_{\mathsf{avc}}} \cdot \mathsf{Adv}^{\text{pdow}}_{\mathsf{Gen},\mathcal{A}_{\text{pdow}}}(\lambda).$$

Proof. We construct a partial-domain one-way adversary $\mathcal{A}_{\text{pdow}}$ that simulates Game 4 and attempts to recover δ from the random-oracle query responsible for Bad. $\mathcal{A}_{\text{pdow}}$ samples an index $k \leftarrow [Q_{\mathsf{H}_{\mathsf{avc}}}]$ uniformly at random and focuses on the k-th random-oracle query $(\mathtt{node}, t)$. It computes a candidate δ' as follows: if $\exists (q, i, e, j)$, $t = \mathsf{TweakSalt}'(\mathtt{salt}^{(q)}, i, e, j)$, then it computes $i^* = \mathbf{i}^{*(q)}[e]$, $\ell = \mathsf{Layer}(i, j)$, and $\delta' := \mathtt{node} \oplus \bigoplus_{j' \in [1:\ell+1]} \mathtt{node}^{(q)}[e][\mathsf{copath}_{j'}(i^*)]$. If Bad occurs, then there exists at least one random-oracle query from which the correct δ can be reconstructed. However, $\mathcal{A}_{\text{pdow}}$ cannot verify whether its computed δ' is correct. Consequently, conditioned on Bad, $\mathcal{A}_{\text{pdow}}$ identifies the correct query with probability $1/Q_{\mathsf{H}_{\mathsf{avc}}}$. Thus, $\Pr[\mathsf{G}_{2.3} : \mathsf{Bad}] = \Pr[\mathsf{G}_4 : \mathsf{Bad}] \le Q_{\mathsf{H}_{\mathsf{avc}}} \cdot \mathsf{Adv}^{\text{pdow}}_{\mathsf{Gen},\mathcal{A}_{\text{pdow}}}(\lambda)$, which completes the proof. □

We also have the following bound.

Lemma 5.2. *There exists a domain-extension one-way adversary $\mathcal{A}_{\text{deow}}$ such that*

$$\left|\Pr[\mathsf{G}_{2.2} : W_{2.2}] - \Pr[\mathsf{G}_{2.3} : W_{2.3}]\right| \le \mathsf{Adv}^{\text{deow}}_{\mathsf{Gen}',\mathcal{A}_{\text{deow}}}(\lambda) + 2 \cdot \mathsf{Adv}^{\text{pr}}_{\mathsf{PRF},\mathcal{A}_{\text{prf}}}(\lambda).$$

Proof. We first modify the key generation procedure as follows. Instead of sampling x uniformly at random, the challenger samples δ and K, and sets $x := \delta \| \mathsf{PRF}(K, \delta)$. We use a prime to denote the games obtained by applying the above modification to the key generation procedure. Since this modification is carried out without revealing K the adversary, it can be justified by a standard PRF-hybrid argument. Therefore, any game can be transformed into this variant at a loss of $\mathsf{Adv}^{\text{pr}}_{\mathsf{PRF},\mathcal{A}_{\text{prf}}}(\lambda)$. Thus, we have

$$\begin{aligned}\left|\Pr[\mathsf{G}_{2.2} : W_{2.2}] - \Pr[\mathsf{G}_{2.3} : W_{2.3}]\right| \le{}& \left|\Pr[\mathsf{G}'_{2.2} : W'_{2.2}] - \Pr[\mathsf{G}'_{2.3} : W'_{2.3}]\right| \\ &+ 2 \cdot \mathsf{Adv}^{\text{pr}}_{\mathsf{PRF},\mathcal{A}_{\text{prf}}}(\lambda).\end{aligned}$$

We next present a reduction to the domain-extension one-wayness, which yields a tight bound. For tightness, $\mathcal{A}_{\text{deow}}$ must be able to efficiently identify a random-oracle query that outputs some δ' satisfying $\hat{F}(\delta' \| \mathsf{PRF}(K, \delta')) = 0$. To this end, we replace $\mathcal{S}$ with the following set $\mathcal{S}'$, for which membership can be efficiently checked by $\mathcal{A}_{\text{deow}}$.

$$\mathcal{S}' = \left\{ (\mathtt{node}, t_{\mathtt{salt}}) \;:\; \begin{array}{c} \exists q \in [Q_{\mathsf{Sign}}],\ \exists e \in [\tau],\ \exists (i,j) \in \mathcal{I} \\ \text{s.t. } \mathsf{P}_{\delta'}(\mathtt{node}, t_{\mathtt{salt}}, q, e, i, j) = 1 \\ \text{and } \hat{F}\big(\delta' \| \mathsf{PRF}(K, \delta')\big) = 0 \end{array} \right\},$$

where $\mathsf{P}_{\delta'}$ is defined in Equation 2. Note that in the domain-extension one-wayness experiment of Definition 4.3, the verification key $\mathtt{vk}'$ additionally contains K, and hence $\mathcal{A}_{\mathrm{deow}}$ can evaluate $\mathsf{PRF}(K,\cdot)$ and test whether the computed δ' satisfies $\hat{F}(\delta' \| \mathsf{PRF}(K,\delta')) = 0$ (where $\mathtt{vk}' = (\mathtt{mseed}_{\mathtt{eq}}, \hat{y}, K)$).

As in Lemma 5.1, we use the fact that $\Pr[\mathsf{G}'_{2.3} : \mathsf{Bad}] = \Pr[\mathsf{G}'_4 : \mathsf{Bad}]$ holds. To bound $\Pr[\mathsf{G}'_4 : \mathsf{Bad}]$, we construct $\mathcal{A}_{\mathrm{deow}}$ that simulates G'_4 and, upon a random-oracle query $(\mathtt{node}, t)$, computes a candidate δ' by XORing $\mathtt{node}$ and copath nodes. Whenever Bad occurs, there exists at least one random-oracle query from which a valid solution can be recovered, and $\mathcal{A}_{\mathrm{deow}}$ can identify such a query using the public key $(\mathtt{mseed}_{\mathtt{eq}}, \hat{y})$ and output the corresponding value. Thus, conditioned on Bad, $\mathcal{A}_{\mathrm{deow}}$ succeeds with probability 1; hence $\Pr[\mathsf{G}'_4 : \mathsf{Bad}] \leq \mathsf{Adv}^{\mathrm{deow}}_{\mathsf{Gen}',\mathcal{A}_{\mathrm{deow}}}(\lambda)$, which completes the proof. □

Having bounded the loss of advantage across all game hops as outlined above, we conclude the proof of this theorem. □

5.3 EUF-CMA Security of MQOM_1 in the QROM

A key difference between the ROM and the QROM is that reductions to one-wayness cannot, in general, be made tight. In particular, any measurement-based reduction incurs a square-root loss [JZM21]. This motivates us to pursue reductions that are as tight as possible. To this end, we avoid relying on partial-domain one-wayness in the QROM.

Theorem 5.2 (EUF-NMA to EUF-CMA of MQOM_1 in the QROM). *Let $\mathsf{H}_{\mathsf{avc}}$, Hash_2, and Hash_4 be random oracles. For a QPT adversary $\mathcal{A}$ that queries the oracles X at most Q_X times, there exist QPT adversaries $\mathcal{A}_{\mathrm{nma}}$, $\mathcal{A}_{\mathrm{deow}}$, $\mathcal{A}_{\mathrm{prg}}$, and $\mathcal{A}_{\mathrm{prf}}$ such that*

$$\begin{aligned}\mathsf{Adv}^{\text{euf-cma}}_{\mathsf{MQOM}_1,\mathcal{A}}(\lambda) \leq\ & \mathsf{Adv}^{\text{euf-nma}}_{\mathsf{MQOM}_1,\mathcal{A}_{\mathrm{nma}}}(\lambda) + 2\sqrt{(Q+1)\mathsf{Adv}^{\mathrm{deow}}_{\mathsf{Gen}',\mathcal{A}_{\mathrm{deow}}}(\lambda)} \\ & + Q_{\mathsf{Sign}}\mathsf{Adv}^{\mathrm{prg}}_{\mathsf{PRG},\mathcal{A}_{\mathrm{prg}}}(\lambda) + 2\cdot\mathsf{Adv}^{\mathrm{pr}}_{\mathsf{PRF},\mathcal{A}_{\mathrm{prf}}}(\lambda) \\ & + (3/2)Q_{\mathsf{Sign}}\sqrt{(Q_{\mathsf{Sign}}+Q_{\mathsf{Hash}_4})/2^{\lambda}} + Q^2_{\mathsf{Hash}_2}/2^{\lambda} + Q^2_{\mathsf{Sign}}/2^{\lambda-12}.\end{aligned}$$

The running times of $\mathcal{A}_{\mathrm{nma}}$, $\mathcal{A}_{\mathrm{deow}}$, $\mathcal{A}_{\mathrm{prg}}$, and $\mathcal{A}_{\mathrm{prf}}$ are approximately the same as that of $\mathcal{A}$. The number of oracle queries made by $\mathcal{A}_{\mathrm{nma}}$ is identical to that made by $\mathcal{A}$, and $\mathcal{A}_{\mathrm{prf}}$ makes only a single oracle query.

Moreover, there exists a QPT adversary $\mathcal{A}'_{\mathrm{ow}}$ such that the above bound also holds after replacing the term $2\sqrt{(Q+1)\mathsf{Adv}^{\mathrm{deow}}_{\mathsf{Gen}',\mathcal{A}_{\mathrm{deow}}}(\lambda)}$ with $2\sqrt{\mathsf{Adv}^{\mathrm{daow}}_{(\mathsf{Gen}',\mathsf{H}_{\mathrm{aux}}),\mathcal{A}'_{\mathrm{ow}}}(\lambda)} + \mathbb{E}[Q^2_{\mathrm{sol}}]/2^{\lambda-1}$, where Q_{sol} denotes the number of domain elements x that map to the same codomain value $\hat{y}$, and the expectation $\mathbb{E}[Q^2_{\mathrm{sol}}]$ is taken over the distribution induced by the key-generation algorithm.

Proof. The proof largely follows Theorem 5.1, with the main challenge arising from differences between ROM and QROM. Specifically, these differences manifest in bounding $|\Pr[\mathsf{G}_{0.2} : W_{0.2}] - \Pr[\mathsf{G}_1 : W_1]|$ and $|\Pr[\mathsf{G}_{2.2} : W_{2.2}] - \Pr[\mathsf{G}_{2.3} : W_{2.3}]|$. While the former case is relatively straightforward due to the availability of techniques for handling QROM, the latter case is significantly more difficult.

First, we address the difference between Games 0.2 and 1.

Lemma 5.3. *We have*

$$|\Pr[\mathsf{G}_{0.2} : W_{0.2}] - \Pr[\mathsf{G}_1 : W_1]| \leq (3/2)Q_{\mathsf{Sign}}\sqrt{(Q_{\mathsf{Sign}}+Q_{\mathsf{Hash}_4})/2^{\lambda}}.$$

Proof. Using the adaptive reprogramming technique [GHHM21] (see Lemma B.1), we construct an adversary that distinguishes between the following two cases: (i) h_2 is computed as $\mathsf{Hash}_4(\mathtt{vk}, \mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2, h_{\mathtt{msg}})$, and (ii) h_2 is sampled uniformly at random and the random oracle is reprogrammed at the point $(\mathtt{vk}, \mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2, h_{\mathtt{msg}})$ accordingly. To simulate the signing oracle, the adversary submits $h_{\mathtt{msg}}$ to Repro, which holds $(\mathtt{vk}, \mathtt{sk})$, samples $\mathtt{salt}$, and computes $(\mathtt{vk}, \mathtt{com}_1, \mathtt{com}_2)$ following exactly the same procedure as the real signing oracle. The oracle then returns $((\mathtt{vk}, \mathtt{com}_1, \mathtt{com}_2), (\mathtt{salt}, \alpha_1, \mathtt{pdecom}, \Delta^{(1)}_x, \mathtt{nonce}))$, where $(\mathtt{salt}, \alpha_1, \mathtt{decom}, \Delta^{(1)}_x, \mathtt{nonce})$ is treated as side information. Using this output, the adversary completes the computation of the signature. Consequently, when $b = 0$ the adversary's simulation is identical to Game 0.1, and when $b = 1$ it is identical to Game 1. The adversary's distinguishing advantage therefore yields the claimed bound via Lemma B.1. □

We now bound the distinguishing advantage between Games 2.2 and 2.3 in the QROM.

Lemma 5.4. *There exist a domain-extension one-way adversary $\mathcal{A}_{\mathrm{deow}}$ and a PRF adversary $\mathcal{A}_{\mathrm{prf}}$ satisfying*

$$|\Pr[\mathsf{G}_{2.2} : W_{2.2}] - \Pr[\mathsf{G}_{2.3} : W_{2.3}]| \leq 2\sqrt{(Q+1)\mathsf{Adv}^{\mathrm{deow}}_{\mathsf{Gen}',\mathcal{A}_{\mathrm{deow}}}(\lambda)} + 2 \cdot \mathsf{Adv}^{\mathrm{pr}}_{\mathsf{PRF},\mathcal{A}_{\mathrm{prf}}}(\lambda).$$

Proof. As in the proof of Lemma 5.2, we have

$$\left|\Pr[\mathsf{G}_{2.2} : W_{2.2}] - \Pr[\mathsf{G}_{2.3} : W_{2.3}]\right| \leq \left|\Pr[\mathsf{G}'_{2.2} : W'_{2.2}] - \Pr[\mathsf{G}'_{2.3} : W'_{2.3}]\right| + 2 \cdot \mathsf{Adv}^{\mathrm{pr}}_{\mathsf{PRF},\mathcal{A}_{\mathrm{prf}}}(\lambda).$$

Then, we use the semi-classical O2H lemma [AHU19] (see Lemma B.2) to randomize outputs of random functions by puncturing $\mathcal{S}'$ defined in Lemma 5.2. From Lemma B.2, we have

$$\left|\Pr[\mathsf{G}'_{2.2} : W'_{2.2}] - \Pr[\mathsf{G}'_{2.3} : W'_{2.3}]\right| \leq 2\sqrt{(Q+1)\Pr[\mathsf{G}'_{2.3} : \mathsf{Find}]},$$

where Find denotes the event that the simulator's measurement of the adversary's query for punctured oracle reveals a puncturing point inside $\mathcal{S}'$.

We now proceed to bound $\Pr[\mathsf{G}'_{2.3} : \mathsf{Find}]$. As in Theorem 5.1, we perform a sequence of game hops so that the final game can be simulated without the secret key. Since the arguments of Theorem 5.1 remain valid in the quantum setting (irrelevant to classical/quantum access to random oracles), we have $\Pr[\mathsf{G}'_{2.3} : \mathsf{Find}] = \Pr[\mathsf{G}'_4 : \mathsf{Find}]$. The domain-extension one-wayness adversary $\mathcal{A}_{\mathrm{deow}}$ can efficiently simulate Game 4. Moreover, $\mathcal{A}_{\mathrm{deow}}$ can implement the punctured oracle, since membership in the puncture set $\mathcal{S}'$ can be decided efficiently from the public key. Finally, when the event Find occurs, $\mathcal{A}_{\mathrm{deow}}$ obtains from the corresponding query a value δ' satisfying $\hat{F}\big(\delta' \,\|\, \mathsf{PRF}(K,\delta')\big) = \hat{y}$, and thus wins its own game. Therefore, we obtain the reduction $\Pr[\mathsf{G}'_4 : \mathsf{Find}] \leq \mathsf{Adv}^{\mathrm{deow}}_{\mathsf{Gen}',\mathcal{A}_{\mathrm{deow}}}(\lambda)$, which completes the proof of the lemma. □

We present an alternative reduction under the assumption of domain-extension/auxiliary-input one-wayness Definition 4.4, which reduces the multiplicative factor in the one-wayness advantage; see the proof of Section C.1.

Lemma 5.5. *There exist a domtain-extension/auxiliary-input one-way adversary $\mathcal{A}_{\mathrm{daow}}$ and a PRF adversary $\mathcal{A}_{\mathrm{pr}}$ satisfying*

$$|\Pr[\mathsf{G}_{2.2} : W_{2.2}] - \Pr[\mathsf{G}_3 : W_3]| \leq 2\sqrt{\mathsf{Adv}^{\mathrm{daow}}_{(\mathsf{Gen}',\mathsf{H}_{\mathrm{aux}}),\mathcal{A}_{\mathrm{daow}}}(\lambda)} + \mathsf{Adv}^{\mathrm{pr}}_{\mathsf{PRF},\mathcal{A}_{\mathrm{prf}}}(\lambda) + \mathbb{E}[Q_{\mathrm{sol}}^2]/2^{\lambda-1}.$$

The same bounds as in Theorem 5.1 apply to the remaining parts, as they are independent of quantum access to the random oracle. Therefore, the theorem follows. □

Remark 5.1. Adopting the techniques used in the sEUF-CMA security proof of Mirath [KX25, Theorem 5.2], we can extend Theorems 5.1 and 5.2 to the strong EUF-CMA security proof.

6 EUF-CMA Security of MQOM_2 in the ROM+ICM

We first recall the H-coefficient technique and then consider two warm-up examples that involve the computational assumptions.

H-coefficient technique: We employ the H-coefficient technique [Pat09, CS14, JN18] to upper-bound the distinguisher's advantage between the real and ideal experiments with oracle sets O_{real} and O_{ideal}. An interaction transcript, denoted by ω, consists of the ordered list of query–answer pairs. We consider deterministic distinguishers. Let Θ_{real} (resp. Θ_{ideal}) denote the transcript random variable in the real (resp. ideal) world. A transcript ω is *attainable* if $\Pr[\Theta_{\mathrm{ideal}} = \omega] > 0$.

Theorem 6.1 (H-coefficient Technique). *We consider the two sets of oracles O_{real} and O_{ideal}. Let $\mathcal{D}$ be a deterministic distinguisher. Let Ω be a set of all attainable transcripts made by the distinguisher $\mathcal{D}$. We then consider a partition Bad and Good of Ω, that is, $\Omega = \mathsf{Bad} \sqcup \mathsf{Good}$. Suppose that there exist non-negative ϵ_{good} and ϵ_{bad} such that if for all $\omega \in \mathsf{Good}$, it holds that $\frac{\Pr[\Theta_{\mathrm{real}}=\omega]}{\Pr[\Theta_{\mathrm{ideal}}=\omega]} \geq 1 - \epsilon_{\mathrm{good}}$ and $\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}] \leq \epsilon_{\mathrm{bad}}$. Then, we have*

$$|\Pr[\mathcal{D}^{O_{\mathrm{real}}} = 1] - \Pr[\mathcal{D}^{O_{\mathrm{ideal}}} = 1]| \leq \epsilon_{\mathrm{good}} + \epsilon_{\mathrm{bad}}.$$

Section D gives warm-up applications of the H-coefficient technique with onewayness. Section D.1 shows the computational switching lemma for a random permutation π and a random function ρ, where the distinguisher is given $(f, f(x^*), \pi(x^*))$ or $(f, f(x^*), y^*)$ with random y^*. Section D.2 shows the tweaked CCR (TCCR) property of EncFF with hidden shift x^* where the distinguisher is given $(f, f(x^*))$.

6.1 Modification to MQOM

We modify MQOM in MQOM_2 as follows:

- We consider the 3-round version of MQOM.
- The input to Hash_4 includes the salt `salt`.
- Let $\mathbb{F} = \mathrm{GF}(2^k)$, $\mathbb{K} = \mathrm{GF}(2^{k\mu})$, and $\lambda = kn = k\eta\mu$. This modification results in $\hat{m} = \eta$, $|x| = \lambda$, and $n_b = 1$. For example, we let $\lambda = 160$ (instead of $\lambda = 128$) for MQOM2-L1-gf2-short-3r in Table 1, where $\mathbb{F} = \mathrm{GF}(2)$, $n = 160$, $\mathbb{K} = \mathrm{GF}(2^{16})$, and $\hat{m} = \eta = 10$. For another example, we let $\lambda = 224$ for MQOM2-L1-gf16-short-3r, where $\mathbb{F} = \mathrm{GF}(16)$, $n = 56$, $\mathbb{K} = \mathrm{GF}(16^4)$, and $\hat{m} = \eta = 14$.
- We explicitly consider linear conversions $\mathsf{B2F} : \{0,1\}^\lambda \to \mathbb{F}^n$ and $\mathsf{B2K} : \{0,1\}^\lambda \to \mathbb{K}^\eta$, which are isomorphisms by our assumption and adopted from those in [BBFR25, Section 2.3]. We have $\phi \circ \mathsf{Bit2F} = \mathsf{Bit2K}$. We denote those linear inversions as $\mathsf{F2B} : \mathbb{F}^n \to \{0,1\}^\lambda$ and $\mathsf{K2B} : \mathbb{K}^\eta \to \{0,1\}^\lambda$. Due to linearity, we have $\mathsf{F2B}(x_0) \oplus \mathsf{F2B}(x_1) = \mathsf{F2B}(x_0 + x_1)$.

MQOM employs a linear orthomorphism σ in their EncFF, which is defined as $\sigma(s_l \| s_r) := (s_l \oplus s_r) \| s_l$ and $\bar{\sigma}(s_l \| s_r) = ((s_l \oplus s_r) \| s_l) \oplus (s_l \| s_r) = s_r \| (s_l \oplus s_r)$, where $s_l, s_r \in \{0,1\}^{\lambda/2}$. We note that they are efficiently invertible. Assuming that λ, n, and η are even, the composed function

$$\Psi := \mathsf{B2K} \circ \sigma \circ \mathsf{F2B} : \mathbb{F}^n \to \mathbb{K}^\eta \tag{3}$$

is also linear. This function can be written $\Psi(s) := M_\Psi \cdot s$, where $M_\Psi \in \mathbb{K}^{\eta \times n}$ and s is treated as a vector in $\mathbb{K}^n$.

We show the EUF-NMA security for MQOM_2 in the ROM+ICM in Section E.4 by modifying that for MQOM_1 in the (Q)ROM.

6.2 EUF-CMA Security

Theorem 6.2 (EUF-NMA to EUF-CMA of MQOM_2 in the ROM+ICM). *Let Hash_4 be a random oracle and let E be an ideal cipher. For a QPT adversary $\mathcal{A}$ that queries the oracles X at most Q_X times, there exist QPT adversaries $\mathcal{A}_{\mathrm{prg}}$, $\mathcal{A}_{\mathrm{ow}}$, and $\mathcal{A}_{\mathrm{nma}}$ such that*

$$\begin{aligned}\mathsf{Adv}^{\text{euf-cma}}_{\mathsf{MQOM}_2,\mathcal{A}}(\lambda) \le \mathsf{Adv}^{\text{euf-nma}}_{\mathsf{MQOM}_2,\mathcal{A}_{\mathrm{nma}}}(\lambda) + \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda) + Q_{\mathsf{Sign}} \cdot \mathsf{Adv}^{\mathrm{prg}}_{\mathsf{PRG},\mathcal{A}_{\mathrm{prg}}}(\lambda) \\ + Q^2_{\mathsf{Sign}}/2^{\lambda-12} + Q_{\mathsf{Sign}}(Q_{\mathsf{Sign}} + Q_{\mathsf{Hash}_4})/2^\lambda + Q^2_{\mathsf{Hash}_2}/2^{2\lambda}.\end{aligned}$$

The running times of $\mathcal{A}_{\mathrm{prg}}$, $\mathcal{A}_{\mathrm{ow}}$, and $\mathcal{A}_{\mathrm{nma}}$ are that of $\mathcal{A}$ plus $\mathsf{poly}(Q,\lambda)$ with $Q = Q_{\mathsf{Sign}} + Q_{\mathsf{Hash}_4}$.

Proof. We first observe that the integer $\mathsf{sel} + 4e + 256j$ for $\mathsf{sel} \in [4]$, $e \in [\tau]$, and $j \in [d]$ or $j \in [n_b]$ is at most $256 \cdot 12$ in the all parameter sets. Hence, we can therefore conclude that $\mathsf{tweak} = \mathsf{bin}_\lambda(\mathsf{sel} + 4e + 256j)$ modifies only the last 12 bits of `salt`.

We define a sequence of games, closely following and slightly adapting the proof of Theorem 5.1. Let G_i denote the i-th game and let W_i be its winning event. We use the same game definitions for Games $0, \dots, 2.2$. The definition diverges from Game 2.3.

- Game 2.3: The signing oracle first computes `pdecom` (as in [GYW+23, CLY+24, BCD25]), derives the opened seeds and tapes using $\mathsf{cAVC}_{\mathrm{IC}}.\mathsf{Recon}$, and then derives the hidden seeds and tapes using the binary format $\chi := \mathsf{F2B}(x)$ of the secret key x. This is a purely conceptual change, and therefore $\Pr[\mathsf{G}_{2.2} : W_{2.2}] = \Pr[\mathsf{G}_{2.3} : W_{2.3}]$.
- Game 3: We finally replace the signing oracle with a simulator that generates a random `pdecom`, derives the opened seeds and tapes using $\mathsf{cAVC}_{\mathrm{IC}}.\mathsf{Recon}$, and samples each $\alpha_0[e]$ uniformly at random. The difference between Game 2.3 and Game 3 is bounded by the one-wayness of Gen as shown in Corollary 6.1, which is proved via the H-coefficient technique.

In G_3, the signing oracle no longer needs to use the secret key x. Therefore, an EUF-NMA adversary $\mathcal{A}_{\mathrm{nma}}$ can simulate the signing oracle in G_3, and we have $\Pr[\mathsf{G}_3 : W_3] \le \mathsf{Adv}^{\text{euf-nma}}_{\mathsf{MQOM}_2,\mathcal{A}_{\mathrm{nma}}}(\lambda)$ as in the proof of Theorem 5.1. □

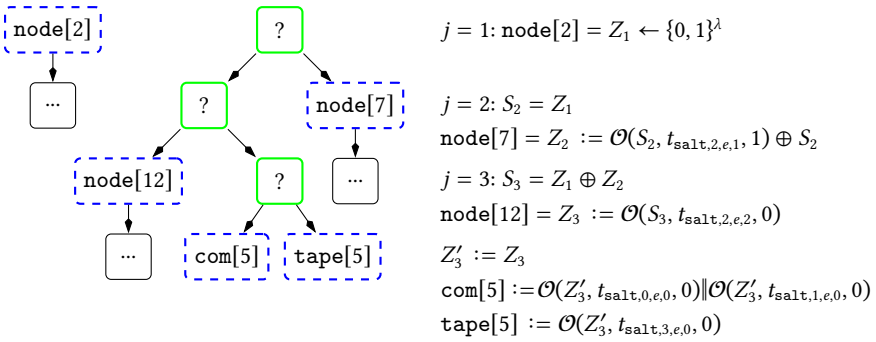

Fig. 10. How to use $\mathcal{O} = \mathcal{O}_x^{\mathrm{TCCR}}$ or random function ρ for $N = 8$ and $i^* = 5$ in [GYW+23, CLY+24, BCD25]. Green nodes $\mathtt{node}[3]$, $\mathtt{node}[6]$, and $\mathtt{node}[13] = \mathtt{seed}[5]$ are hidden.

6.3 Bounding Game 2.3 and Game 3

Review of cGGM trees: To prepare the notions and definitions of the transcripts, we first recall how to simulate the cGGM tree's security game by using a (T)CCR hash or a random one [GYW+23, CLY+24, BCD25]. We modify the notation in Bui et al. [BCD25] to fit it into our context, the IC-based AVC in MQOM_2. Let x be a secret key and $\chi := \mathsf{F2B}(x)$. Recall that, in the real world, $\mathcal{O}_\chi^{\mathrm{TCCR}}(s,t,b) = \mathsf{EncFF}(s \oplus \chi, t) \oplus b\chi = E(t, s \oplus \chi) \oplus \sigma(s \oplus \chi) \oplus b\chi$.

To show that the cGGM tree is pseudorandom, the reduction algorithm computes co-pathes, commitments, and tapes by using $\mathcal{O} = \mathcal{O}_\chi^{\mathrm{TCCR}}$ or a random function ρ as follows:

1. At first, it chooses $Z_1 \leftarrow \{0,1\}^\lambda$, wihch is corresponding to $\mathtt{node}[\mathsf{copath}_1(i^*)]$.
2. For $j = 2, \ldots, d$, it defines the following values:

$$S_j := \bigoplus_{\ell \in [1:j]} Z_\ell,$$

$$Z_j := \begin{cases} \mathcal{O}(S_j, t_{\mathtt{salt},2,e,j-1}, 0) & \text{if } \mathsf{copath}_j(i^*) \text{ is left node} \\ \mathcal{O}(S_j, t_{\mathtt{salt},2,e,j-1}, 1) \oplus S_j & \text{otherwise,} \end{cases}$$

$$Z'_j := \begin{cases} Z_j & \text{if } \mathsf{copath}_j(i^*) \text{ is left node} \\ Z_j \oplus S_j & \text{otherwise.} \end{cases}$$

3. It then sets $\mathtt{node}[\mathsf{copath}_j(i^*)] := Z_j$ for $j = 1, \ldots, d$, which consits of $\mathtt{path}$.
4. It finally computes $\mathtt{com}[i^*] := \mathcal{O}(Z'_d, t_{\mathtt{salt},0,e,0}, 0) \,\|\, \mathcal{O}(Z'_d, t_{\mathtt{salt},1,e,0}, 0)$ and $\mathtt{tape}[i^*] := \mathcal{O}(Z'_d, t_{\mathtt{salt},3,e,0}, 0)$.

The graphical explanation is in Figure 10.

Main proof: By using those notations, we define two games Real and Ideal as in Figure 11, where they roughly correspond to Game 2.3 and Game 3, respectively. We show the computational indistinguishability between the two games Real and Ideal by using the H-coefficient technique.

Lemma 6.1. *Let $\mathcal{A}$ be a PPT distinguisher. There exists an adversary $\mathcal{A}_{\mathrm{ow}}$ satisfying*

$$|\Pr[\mathsf{Real} = 1] - \Pr[\mathsf{Ideal} = 1]| \le \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda).$$

The running time of $\mathcal{A}_{\mathrm{ow}}$ is that of $\mathcal{A}$ plus $\mathsf{poly}(Q, \lambda)$.

As a direct implication, we have the following corollary giving the upper bound of the difference between $\mathsf{G}_{2.3}$ and G_3. We omit the proof of the corollary, since the reduction is straightforward.

Corollary 6.1. *Let $\mathcal{A}$ be a PPT distinguisher. There exists an adversary $\mathcal{A}_{\mathrm{ow}}$ satisfying*

$$|\Pr[\mathsf{G}_{2.3} : W_{2.3}] - \Pr[\mathsf{G}_3 : W_3]| \le \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda).$$

The running time of $\mathcal{A}_{\mathrm{ow}}$ is that of $\mathcal{A}$ plus $\mathsf{poly}(Q, \lambda)$.

Proof (Proof of Lemma 6.1). Let us prepare the notations for transcripts: The oracle stores the following informations to $\mathcal{Q}_{\mathrm{node}}$, $\mathcal{Q}_{\mathrm{com}}$, and $\mathcal{Q}_{\mathrm{alpha}}$, which store Z_j, c_i, and α_0, respectively. The oracle $E^\pm$ stores $(t, s, y) \in \mathcal{Q}_E$ if E^+ returns y on a query (t, s) or E^- returns s on a query (t, y).

Real / Ideal

$\mathcal{Q} := \emptyset$
$\mathcal{Q}_{\mathsf{salt}} := \emptyset$
$(\mathtt{vk}, \mathtt{sk}) \leftarrow \mathsf{Gen}(1^\lambda)$
$(\mathtt{vk}, x) := \mathsf{Parse}(\mathtt{sk})$
$(\mathtt{mseed}_{\mathsf{eq}}, \{\hat{y}_i\}) := \mathsf{Parse}(\mathtt{vk})$
$(\{\hat{A}_i\}, \{\hat{b}_i\}) := \mathsf{ExpandEquations}(\mathtt{mseed}_{\mathsf{eq}})$
$O := \{E^{\pm}, \mathcal{O}_{\mathrm{real}}\}$ // Real
$O := \{E^{\pm}, \mathcal{O}_{\mathrm{ideal}}\}$ // Ideal
$b \leftarrow \mathcal{A}^{O}(\mathtt{vk})$
return b

$\mathcal{O}_{\mathrm{real}}(\mathbf{i}^*)$

$\mathtt{salt} \leftarrow \{0,1\}^\lambda$
if $\exists \mathsf{FirstBits}_{\lambda-12}(\mathtt{salt}) \in \mathcal{Q}_{\mathsf{salt}}$ **then abort**
$\mathcal{Q}_{\mathsf{salt}} := \mathcal{Q}_{\mathsf{salt}} \cup \{\mathsf{FirstBits}_{\lambda-12}(\mathtt{salt})\}$
for $e \in [\tau]$ **do**
  $i^* := \mathbf{i}^*[e]$
  $Z_1 \leftarrow \{0,1\}^\lambda$
  for $j = 2, \ldots, d$ **do**
    $b := \mathsf{copath}_j(i^*) \bmod 2$
    $S_j := \bigoplus_{\ell \in [1:j]} Z_j$
    $Z_j := E(t_{\mathtt{salt},2,e,j-1}, S_j \oplus \chi) \oplus \sigma(S_j \oplus \chi) \oplus b \cdot \chi \oplus b \cdot S_j$
    $Z'_j := Z_j \oplus b \cdot S_j$
  $c_0 := E(t_{\mathtt{salt},0,e,0}, Z'_d \oplus \chi) \oplus \sigma(Z'_d \oplus \chi)$
  $c_1 := E(t_{\mathtt{salt},1,e,0}, Z'_d \oplus \chi) \oplus \sigma(Z'_d \oplus \chi)$
  $\mathtt{com}[e][i^*] := c_0 \,\|\, c_1$
  $\mathtt{tape}[e][i^*] := E(t_{\mathtt{salt},3,e,0}, Z'_d \oplus \chi) \oplus \sigma(Z'_d \oplus \chi)$
  $\mathtt{pdecom}[e] := (Z_1, Z_2, \ldots, Z_d, \mathtt{com}[e][i^*])$
for $e \in [\tau]$ **do**
  $(\mathtt{com}[e], \mathtt{seed}[e], \mathtt{tape}[e])$
    $:= \mathsf{cAVC}_{\mathrm{IC}}.\mathsf{Recon}(\mathtt{salt}, \mathtt{pdecom}[e], e, \mathbf{i}^*[e])$
  $\mathtt{seed}[e][\mathbf{i}^*[e]] := \chi \oplus \bigoplus_{i \neq \mathbf{i}^*[e]} \mathtt{seed}[e][i]$
  for $i \in [N]$ **do**
    $\bar{x}[e][i] := \mathsf{B2F}(\mathtt{seed}[e][i]) \in \mathbb{F}^n$
    $\bar{u}[e][i] := \mathsf{B2K}(\mathtt{tape}[e][i]) \in \mathbb{K}^\eta$
  $u_0[e] := -\sum_{i \in [N]} \omega_i \cdot \bar{u}[e][i] \in \mathbb{K}^\eta$
  $x_0[e] := -\sum_{i \in [N]} \omega_i \cdot \bar{x}[e][i] \in \mathbb{K}^n$
  // NOTE: We have $x = \sum_{i \in [N]} \bar{x}[e][i]$
for $e \in [\tau]$ **do**
  for $i \in [\hat{m}]$ **do**
    $z_{0,i} := x_0[e]^\top \cdot \hat{A}_i \cdot x_0[e] \in \mathbb{K}$
  $z_0 := (z_{0,0}, \ldots, z_{0,\hat{m}-1}) \in \mathbb{K}^{\hat{m}} = \mathbb{K}^\eta$
  $\alpha_0[e] := u_0[e] + z_0 \in \mathbb{K}^\eta$
return $(\mathtt{salt}, \mathtt{pdecom}, \alpha_0)$

$\mathcal{O}_{\mathrm{ideal}}(\mathbf{i}^*)$

$\mathtt{salt} \leftarrow \{0,1\}^\lambda$
if $\exists \mathsf{FirstBits}_{\lambda-12}(\mathtt{salt}) \in \mathcal{Q}_{\mathsf{salt}}$ **then abort**
$\mathcal{Q}_{\mathsf{salt}} := \mathcal{Q}_{\mathsf{salt}} \cup \{\mathsf{FirstBits}_{\lambda-12}(\mathtt{salt})\}$
for $e \in [\tau]$ **do**
  $i^* := \mathbf{i}^*[e]$
  $Z_1 \leftarrow \{0,1\}^\lambda$
  for $j = 2, \ldots, d$ **do**
    $b := \mathsf{copath}_j(i^*) \bmod 2$
    $S_j := \bigoplus_{\ell \in [1:j]} Z_j$
    $Z_j \leftarrow \{0,1\}^\lambda$
    $Z'_j := Z_j \oplus b \cdot S_j$
  $c_0 \leftarrow \{0,1\}^\lambda$
  $c_1 \leftarrow \{0,1\}^\lambda$
  $\mathtt{com}[e][\mathbf{i}^*[e]] := c_0 \,\|\, c_1$
  $\mathtt{pdecom}[e] := (Z_1, Z_2, \ldots, Z_d, \mathtt{com}[e][\mathbf{i}^*[e]])$
for $e \in [\tau]$ **do**
  $(\mathtt{com}[e], \mathtt{seed}[e], \mathtt{tape}[e])$
    $:= \mathsf{cAVC}_{\mathrm{IC}}.\mathsf{Recon}(\mathtt{salt}, \mathtt{pdecom}[e], e, \mathbf{i}^*[e])$
for $e \in [\tau]$ **do**
  $\alpha_0[e] \leftarrow \mathbb{K}^\eta$
return $(\mathtt{salt}, \mathtt{pdecom}, \alpha_0)$

Fig. 11. Games for bounding Game 2.3 and Game 3

Bad transcripts with respect to nodes and commitments: On q-th signing query, $\mathcal{Q}_{\mathrm{node}}$ stores, for $j = 2, \dots, d$, either $(q, \mathtt{salt}, e, i^*, j, 0, S_j, t_{\mathtt{salt},2,e,j-1}, Z_j)$ if $\mathsf{copath}_j(i^*)$ is left node or $(q, \mathtt{salt}, e, i^*, j, 1, S_j, t_{\mathtt{salt},2,e,j-1}, Z_j)$ if $\mathsf{copath}_j(i^*)$ is right node. $\mathcal{Q}_{\mathrm{com}}$ stores $(q, \mathtt{salt}, e, i^*, Z'_d, t_{\mathtt{salt},k,e,0}, Z''_k)$ for $k = 0, 1$.

The bad transcripts with respect to nodes and commitments are defined as follows:

Definition 6.1. *We define the following four sets of bad transcripts:*

- $\mathsf{Bad}_{1,\mathrm{node},F}$: *There exist* $(*, *, *, *, *, b, s, t, z) \in \mathcal{Q}_{\mathrm{node}}$ *and* $(t, s \oplus \chi, *) \in \mathcal{Q}_E$.
- $\mathsf{Bad}_{1,\mathrm{node},B}$: *There exist* $(*, *, *, *, *, b, s, t, z) \in \mathcal{Q}_{\mathrm{node}}$ *and* $(t, *, \sigma(s \oplus \chi) \oplus b\chi \oplus z)$ *in* $\mathcal{Q}_E$.
- $\mathsf{Bad}_{1,\mathrm{com},F}$: *There exist* $(*, *, *, *, s, t, z) \in \mathcal{Q}_{\mathrm{com}}$ *and* $(t, s \oplus \chi, *)$ *in* $\mathcal{Q}_E$.
- $\mathsf{Bad}_{1,\mathrm{com},B}$: *There exist* $(*, *, *, *, s, t, z) \in \mathcal{Q}_{\mathrm{com}}$ *and* $(t, *, \sigma(s \oplus \chi) \oplus z)$ *in* $\mathcal{Q}_E$.

Bad transcripts with respect to α_0: Fix q and e. Let $i^* := \mathbf{i}^*[e]$. Let $t := t_{\mathtt{salt},3,e,0}$. For ease of notation, we define $\hat{A}(s) := (s^\top \cdot \hat{A}_0 \cdot s, \dots, s^\top \cdot \hat{A}_{\hat{m}-1} \cdot s) \in \mathbb{K}^{\hat{m}} = \mathbb{K}^\eta$ for $s \in \mathbb{K}^n$.

In the real experiment, we have $\alpha_0[e] := u_0[e] + z_0 \in \mathbb{K}^\eta$, where $x_0[e] := -\sum_i \omega_i \cdot \bar{x}[e][i] \in \mathbb{K}^n$, $u_0[e] := -\sum_i \omega_i \cdot \bar{u}[e][i] \in \mathbb{K}^\eta$, and $z_0 = (z_{0,0}, \dots, z_{0,\hat{m}-1})$ with $z_{0,i} := \hat{A}_i(x_0[e]) \in \mathbb{K}$. Recall that, by our modification, we have $\bar{x}[e][i] = \mathsf{B2F}(\mathtt{seed}[e][i])$, $\bar{u}[e][i] = \mathsf{B2K}(\mathtt{tape}[e][i])$, and $x = \sum_i \bar{x}[e][i]$. For ease of notation, we let

$$\tilde{x}_0 := -\sum_{i \neq i^*} \omega_i \cdot \bar{x}[e][i] = x_0[e] + \omega_{i^*} \cdot \bar{x}[e][i^*] \in \mathbb{K}^n, \tag{4}$$

$$\tilde{u}_0 := -\sum_{i \neq i^*} \omega_i \cdot \bar{u}[e][i] = u_0[e] + \omega_{i^*} \cdot \bar{u}[e][i^*] \in \mathbb{K}^\eta. \tag{5}$$

In addition, we define

$$\tilde{x}_1 := \sum_{i \neq i^*} \bar{x}[e][i] = x - \bar{x}[e][i^*] \in \mathbb{F}^n, \tag{6}$$

$$\tilde{\chi}_1 := \mathsf{F2B}(\tilde{x}_1) = \mathsf{F2B}(x) \oplus \mathsf{F2B}(\bar{x}[e][i^*]) = \chi \oplus \mathtt{seed}[e][i^*]. \tag{7}$$

We note that Z'_d in the previous paragraph is $\tilde{\chi}_1$. Hence, we have

$$\begin{aligned}
\alpha_0[e] &= u_0[e] + \hat{A}(x_0[e]) \\
&= \tilde{u}_0 - \omega_{i^*} \cdot \bar{u}[e][i^*] + \hat{A}\big(\tilde{x}_0 - \omega_{i^*} \cdot \bar{x}[e][i]\big) \\
&= \tilde{u}_0 - \omega_{i^*} \cdot \mathsf{B2K}(\mathtt{tape}[e][i^*]) + \hat{A}(\tilde{x}_0 - \omega_{i^*} \cdot (x - \tilde{x}_1)) \\
&= \tilde{u}_0 - \omega_{i^*} \cdot \mathsf{B2K}\big(E(t, \chi \oplus \tilde{\chi}_1) \oplus \sigma(\chi \oplus \tilde{\chi}_1)\big) + \hat{A}(\omega_{i^*} \cdot x - (\omega_{i^*} \cdot \tilde{x}_1 + \tilde{x}_0)),
\end{aligned} \tag{8}$$

where the second equation uses the definitions of $\tilde{u}_0$ (Equation 5) and $\tilde{x}_0$ (Equation 4), the third equation uses the definition of $\bar{u}[e][i^*]$ and $\tilde{x}_1$ (Equation 6), and the forth equation uses the definition of $\mathtt{tape}[e][i^*] = E(t, \mathtt{seed}[e][i^*]) \oplus \sigma(\mathtt{seed}[e][i^*])$ with $\mathtt{seed}[e][i^*] = \chi \oplus \tilde{\chi}_1$ (Equation 7) and $\hat{A}(-s) = \hat{A}(s)$.

Notice that we have $\hat{y} = \hat{A}(x) + \hat{B} \cdot x$, where $\hat{B} \in \mathbb{K}^{\hat{m} \times n}$ and its i-th row is $\hat{b}_i^\top \in \mathbb{K}^{1 \times n}$. For simplicity of notation, let $\dot{x} := \tilde{x}_1 + \omega_{i^*}^{-1} \tilde{x}_0 \in \mathbb{K}^n$. For each $i \in [\hat{m}]$, we have

$$\begin{aligned}
[\hat{A}(x - \dot{x}) - \hat{y}]_i &= (x - \dot{x})^\top \cdot \hat{A}_i \cdot (x - \dot{x}) - x^\top \cdot \hat{A}_i \cdot x - \hat{b}_i^\top \cdot x \\
&= (-\dot{x}^\top \hat{A}_i^\top - \dot{x}^\top \hat{A}_i - \hat{b}_i^\top) \cdot x + \dot{x}^\top \hat{A}_i \dot{x}.
\end{aligned} \tag{9}$$

Thus, we can write

$$\begin{aligned}
\hat{A}(\omega_{i^*} \cdot x - (\omega_{i^*} \cdot \tilde{x}_1 + \tilde{x}_0)) - \omega_{i^*}^2 \hat{y} &= \omega_{i^*}^2 \cdot \hat{A}(x - \dot{x}) - \omega_{i^*}^2 \cdot \hat{y} = \omega_{i^*}^2 \cdot \big(\hat{A}(x - \dot{x}) - \hat{y}\big) \\
&= \omega_{i^*}^2 \cdot (B \cdot x + c),
\end{aligned} \tag{10}$$

where $B \in \mathbb{K}^{\hat{m} \times n}$ and $c \in \mathbb{K}^{\hat{m}}$ are defined as in Equation 9. Rewriting Equation 8, we have

$$\begin{aligned}
&\omega_{i^*}^{-1} \cdot (\alpha_0[e] - \tilde{u}_0) - \omega_{i^*} \hat{y} && (11) \\
&= -\mathsf{B2K}(E(t, \chi \oplus \tilde{\chi}_1) \oplus \sigma(\chi \oplus \tilde{\chi}_1)) + \omega_{i^*}^{-1} \cdot \big(\hat{A}(\omega_{i^*} \cdot x - (\omega_{i^*} \cdot \tilde{x}_1 + \tilde{x}_0)) - \omega_{i^*}^2 \cdot \hat{y}\big) && \text{(from eq.(8))} \\
&= -\mathsf{B2K}(E(t, \chi \oplus \tilde{\chi}_1)) - \mathsf{B2K}(\sigma(\mathsf{F2B}(x))) - \mathsf{B2K}(\sigma(\mathsf{F2B}(\tilde{x}_1))) + \omega_{i^*} \cdot (B \cdot x + c) && \text{(from eq.(10))} \\
&= -\mathsf{B2K}(E(t, \chi \oplus \mathsf{F2B}(\tilde{x}_1))) + (\omega_{i^*} B - M_\Psi) \cdot x - M_\Psi \tilde{x}_1 + \omega_{i^*} c. && \text{(from eq.(3))}
\end{aligned}$$

In other words, we have

$$\mathsf{B2K}\big(E(t, \chi \oplus \mathsf{F2B}(\tilde{x}_1))\big) = -\omega_{i^*}^{-1} \cdot (\alpha_0[e] - \tilde{u}_0) + \omega_{i^*}\hat{y} + (\omega_{i^*}B - M_\Psi) \cdot x - M_\Psi \tilde{x}_1 + \omega_{i^*}c.$$

Wrapping up, the oracle stores $(q, \mathtt{salt}, e, i^*, \tilde{x}_1, \tilde{u}_0, B, c, t = t_{\mathtt{salt},3,e,0}, \alpha_0[e])$ to query set $\mathcal{Q}_{\mathrm{alpha}}$.

Now, we consider the following bad sets of transcripts:

Definition 6.2. *We define the following four sets of bad transcripts:*

- $\mathsf{Bad}_{1,\mathrm{alpha},F}$*: There exist* $(*, *, *, *, \tilde{x}_1, \tilde{u}_0, B, c, t, \alpha_0[e]) \in \mathcal{Q}_{\mathrm{alpha}}$ *and* $(t, \chi \oplus \mathsf{F2B}(\tilde{x}_1), *) \in \mathcal{Q}_E$.
- $\mathsf{Bad}_{1,\mathrm{alpha},B}$*: There exist* $(*, *, *, i^*, \tilde{x}_1, \tilde{u}_0, B, c, t, \alpha_0[e]) \in \mathcal{Q}_{\mathrm{alpha}}$ *and* $\big(t, *, \mathsf{K2B}(-\omega_{i^*}^{-1} \cdot (\alpha_0[e] - \tilde{u}_0) + \omega_{i^*}\hat{y} + (\omega_{i^*}B - M_\Psi) \cdot x - M_\Psi\tilde{x}_1 + \omega_{i^*}c)\big) \in \mathcal{Q}_E$.

Because of technical reasons, we need to conjecture that the following holds:

Definition 6.3 (Conjecture). *Let* $\dot{x} := \tilde{x}_1 + \omega_{i^*}^{-1}\tilde{x}_0$. *Let* $B \in \mathbb{K}^{\hat{m}\times n}$, *where the* i*-th row is* $b_i^\top := -\dot{x}^\top \cdot \hat{A}_i^\top - \dot{x}^\top \cdot \hat{A}_i - \hat{b}_i^\top$. *We conjecture that* $\mathrm{rank}_{\mathbb{F}}\big(\mathsf{K2F}(\omega_{i^*}B - M_\Psi)\big) = n - O(1)$ *for all queries.*

Bad transcripts: impossible collisions in $E(t, \cdot)$*:* Let us consider the impossible event in Ideal that causes a collision in $E(t, \cdot)$ virtually. We need to consider collisions between the computations of nodes, coms, and alphas.

- Collision between nodes: There are two distinct entries $(*, *, *, *, j, b, s, t, z), (*, *, *, *, j', b', s', t', z') \in \mathcal{Q}_{\mathrm{node}}$ satisfying $t = t'$ and $\sigma(s \oplus \chi) \oplus b \cdot (s \oplus \chi) \oplus z = \sigma(s' \oplus \chi) \oplus b' \cdot (s' \oplus \chi) \oplus z'$. However, since we exclude the collision of $\mathtt{salt}$, if $j \neq j'$, then $t \neq t'$, and this event never happens.
- Collision between node and com: This does not occur since we exclude the collision of t's between nodes and coms.
- Collision between node and alpha: This never happens because we exclude collisions between t's across nodes and alphas.
- Collision between coms: There are two distinct entries $(q, \mathtt{salt}, e, i^*, s, t, z), (q, \mathtt{salt}, e, i^*, s', t', z') \in \mathcal{Q}_{\mathrm{com}}$ satisfying $t = t'$ and $\sigma(s \oplus \chi) \oplus z = \sigma(s' \oplus \chi) \oplus z'$. Since we eliminate the collision of $\mathtt{salt}$, when we fix $(q, \mathtt{salt}, e, i^*)$, we have only two queries $(q, \mathtt{salt}, e, i^*, s, t_{\mathtt{salt},0,e,0}, z)$ and $(q, \mathtt{salt}, e, i^*, s', t_{\mathtt{salt},1,e,0}, z')$ in $\mathcal{Q}_{\mathrm{com}}$ and $t_{\mathtt{salt},0,e,0} \neq t_{\mathtt{salt},1,e,0}$. Thus, this event never happens.
- Collision between com and alpha: This never happens since we exclude the collision of t's between com and alpha.
- Collision between alphas: This also does not happen.

Wrapping up, we have no need to consider the output collision since we eliminate the collision of $t_{\mathtt{salt},k,e,j}$.

Bounding $\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}]$*:* We define $\mathsf{Bad} := \mathsf{Bad}_{1,\mathrm{node},F} \cup \mathsf{Bad}_{1,\mathrm{node},B} \cup \mathsf{Bad}_{1,\mathrm{com},F} \cup \mathsf{Bad}_{1,\mathrm{com},B} \cup \mathsf{Bad}_{1,\mathrm{alpha},F} \cup \mathsf{Bad}_{1,\mathrm{alpha},B}$. Let us give an upperbound on $\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}]$. We give a reduction algorithm $\mathcal{A}_{\mathrm{ow}}$ that simulates the ideal experiment as follows:

1. Receive $\mathtt{vk}$ from its challenger and compute $\hat{F}$.
2. Fix a random tape of $\mathcal{A}$ and run it as a deterministic distinguisher. Simulate the ideal experiment and maintain $\mathcal{Q}_{\mathrm{node}}$, $\mathcal{Q}_{\mathrm{com}}$, $\mathcal{Q}_{\mathrm{alpha}}$, and $\mathcal{Q}_E$.
3. If the game does not abort, then find x' as follows:
   - $\mathsf{Bad}_{1,\mathrm{node},F}$: For each $t = t_{\mathtt{salt},2,e,j-1}$, search $(*, *, *, *, *, b, s, t, z) \in \mathcal{Q}_{\mathrm{node}}$ and $(t, s', *) \in \mathcal{Q}_E$. Check if $x' := \mathsf{B2F}(s \oplus s')$ satisfies $\hat{F}(x') = 0$ or not. If so, output x'.
   - $\mathsf{Bad}_{1,\mathrm{node},B}$: For each $t = t_{\mathtt{salt},2,e,j-1}$, search $(*, *, *, *, *, b, s, t, z) \in \mathcal{Q}_{\mathrm{node}}$ and $(t, *, y) \in \mathcal{Q}_E$. If $b = 0$, check if $x' := \mathsf{B2F}(\sigma^{-1}(y \oplus z) \oplus s)$ satisfies $\hat{F}(x') = 0$ or not. If so, output x'. If $b = 1$, check if $x' := \mathsf{B2F}(\bar{\sigma}^{-1}(y \oplus z \oplus \sigma(s)))$ satisfies $\hat{F}(x') = 0$ or not. If so, output x'.
   - $\mathsf{Bad}_{1,\mathrm{com},F}$: For each $t = t_{\mathtt{salt},i,e,0}$, search $(*, *, *, *, s, t, z) \in \mathcal{Q}_{\mathrm{com}}$ and $(t, s', *) \in \mathcal{Q}_E$. If $\hat{F}(s \oplus s') = 0$, then output $x' := s \oplus s'$.
   - $\mathsf{Bad}_{1,\mathrm{com},B}$: For each $t = t_{\mathtt{salt},i,e,0}$, search $(*, *, *, *, s, t, z) \in \mathcal{Q}_{\mathrm{com}}$ and $(t, *, y) \in \mathcal{Q}_E$ and check if $x' := \sigma^{-1}(y \oplus z) \oplus s$ satisfies $\hat{F}(x') = 0$. If so, output x'.
   - $\mathsf{Bad}_{1,\mathrm{alpha},F}$: For each $t = t_{\mathtt{salt},3,e,0}$, search $(*, *, *, i^*, \tilde{x}_1, \tilde{u}_0, B, c, t, \alpha_0[e]) \in \mathcal{Q}_{\mathrm{alpha}}$ and $(t, s', *) \in \mathcal{Q}_E$. If $\hat{F}\big(\mathsf{B2F}(s' \oplus \mathsf{F2B}(\tilde{x}_1))\big) = 0$, then output $x' := s' \oplus \tilde{x}_1$.

– $\mathsf{Bad}_{1,\mathrm{alpha},B}$: For each $t = t_{\mathtt{salt},3,e,0}$, search $(*,*,*,i^*,\tilde{x}_1,\tilde{u}_0,B,c,t,\alpha_0[e]) \in \mathcal{Q}_{\mathrm{alpha}}$ and $(t,*,y) \in \mathcal{Q}_E$. Find $x' \in \mathbb{F}^n$ satisfying

$$(\omega_{i^*}B - M_\Psi)\cdot x' = \mathsf{B2K}(y) + \omega_{i^*}^{-1}\cdot(\alpha_0[e] - \tilde{u}_0) - \omega_{i^*}\hat{y} + M_\Psi\tilde{x}_1 - \omega_{i^*}c$$

and check if $\hat{F}(x') = 0$. If so, output x'. From our conjecture Definition 6.3, $\mathrm{rank}_{\mathbb{F}}\left(\mathsf{K2F}(\omega_{i^*}B - M_\Psi)\right)$ is sufficiently high and we can find such x'.

Thus, assuming that our conjecture (Definition 6.3) holds, we have $\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}] \le \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda)$.

Likelihood ratio on good transcripts: We then show that, for any $\omega \in \mathsf{Good} := \Omega \setminus \mathsf{Bad}$, $\frac{\Pr[\Theta_{\mathrm{real}}=\omega]}{\Pr[\Theta_{\mathrm{ideal}}=\omega]} \ge 1$ holds.

Let us fix $\omega \in \mathsf{Good}$. In the ideal experiment, we choose $\mathtt{pdecom}$ and α_0 uniformly at random, which are independent of E. Thus, we have

$$\begin{aligned}\Pr[\Theta_{\mathrm{ideal}} = \omega] &= \Pr_{\mathsf{Ideal}}[\mathcal{Q}_{\mathrm{node}},\mathcal{Q}_{\mathrm{com}},\mathcal{Q}_E] = \Pr_{\mathsf{Ideal}}[\mathcal{Q}_{\mathrm{node}},\mathcal{Q}_{\mathrm{com}} \mid \mathcal{Q}_E]\cdot\Pr_{\mathsf{Ideal}}[\mathcal{Q}_E]\\ &= \Pr_E[\mathcal{Q}_E]\cdot\left(\frac{1}{2^{\lambda(d+2)}}\frac{1}{2^{k\eta}}\right)^{\tau\cdot Q_O}.\end{aligned}$$

On the other hand, since we have restrictions on ideal ciphers in the real experiment, we have

$$\begin{aligned}\Pr[\Theta_{\mathrm{real}} = \omega] &= \Pr_{\mathsf{Real}}[\mathcal{Q}_{\mathrm{node}},\mathcal{Q}_{\mathrm{com}},\mathcal{Q}_E] = \Pr_{\mathsf{Real}}[\mathcal{Q}_{\mathrm{node}},\mathcal{Q}_{\mathrm{com}} \mid \mathcal{Q}_E]\cdot\Pr_{\mathsf{Real}}[\mathcal{Q}_E]\\ &\ge \Pr_E[\mathcal{Q}_E]\cdot\left(\frac{1}{2^{\lambda(d+2)}}\frac{1}{2^{k\eta}}\right)^{\tau\cdot Q_O}.\end{aligned}$$

Thus, for any $\omega \in \mathsf{Good}$, we obtain $\frac{\Pr[\Theta_{\mathrm{real}}=\omega]}{\Pr[\Theta_{\mathrm{ideal}}=\omega]} \ge 1$. This implies $\epsilon_{\mathrm{good}} = 0$.

Conclusion via the H-coefficient technique: Applying the H-coefficient technique (Theorem 6.1), we obtain that there exists a PPT machine $\mathcal{A}_{\mathrm{ow}}$ satisfying

$$\begin{aligned}|\Pr[\mathsf{Real}=1] - \Pr[\mathsf{Ideal}=1]| &\le \epsilon_{\mathrm{good}} + \epsilon_{\mathrm{bad}}\\ &\le 0 + \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda)\end{aligned}$$

as we wanted. □

Acknowledgment

Parts of the manuscript were edited with the assistance of generative AI tools (ChatGPT, OpenAI; model: GPT-5.2 Thinking) for language polishing (e.g., phrasing, grammar, and clarity) and another tool (Gemini 3, Google) for making figures. All AI-assisted suggestions were reviewed and revised by the authors, who take full responsibility for the correctness and integrity of the manuscript.

References

AAB+24. Gor Adj, Nicolas Aragon, Stefano Barbero, Magali Bardet, Emmanuele Bellini, Loïc Bidoux, Jesús-Javier Chi-Domínguez, Victor Dyseryn, Andre Esser, Thibauld Feneuil, Philippe Gaborit, Romaric Neveu, Matthieu Rivain, Luis Rivera-Zamarripa, Carlo Sanna, Jean-Pierre Tillich, Javier Verbel, and Floyd Zweydinger. Mirath (merger of MIRA/MiRitH). Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures. 2

ABB+24a. Najwa Aaraj, Slim Bettaieb, Loïc Bidoux, Alessandro Budroni, Victor Dyseryn, Andre Esser, Thibauld Feneuil, Philippe Gaborit, Mukul Kulkarni, Victor Mateu, Marco Palumbi, Lucas Perin, Matthieu Rivain, Jean-Pierre Tillich, and Keita Xagawa. PERK. Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures. 2

ABB+24b. Carlos Aguilar Melchor, Slim Bettaieb, Loïc Bidoux, Thibauld Feneuil, Philippe Gaborit, Nicolas Gama, Shay Gueron, James Howe, Andreas Hülsing, David Joseph, Antoine Joux, Mukul Kulkarni, Edoardo Persichetti, Tovohery H. Randrianarisoa, Matthieu Rivain, and Dongze Yue. SDitH — Syndrome Decoding in the Head. Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures. 2

ABB+24c. Nicolas Aragon, Magali Bardet, Loïc Bidoux, Jesús-Javier Chi-Domínguez, Victor Dyseryn, Thibauld Feneuil, Philippe Gaborit, Antoine Joux, Romaric Neveu, Matthieu Rivain, Jean-Pierre Tillich, and Adrien Vinçotte. RYDE. Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures. 2

AHJ+23. Carlos Aguilar Melchor, Andreas Hülsing, David Joseph, Christian Majenz, Eyal Ronen, and Dongze Yue. SDitH in the QROM. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VII*, volume 14444 of *LNCS*, pages 317–350. Springer, Singapore, December 2023. 39

AHU19. Andris Ambainis, Mike Hamburg, and Dominique Unruh. Quantum security proofs using semi-classical oracles. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 269–295. Springer, Cham, August 2019. 4, 17, 28

BBB+24. Carsten Baum, Lennart Braun, Ward Beullens, Cyprien Delpech de Saint Guilhem, Michael Klooß, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. FAEST. Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures. 2

BBD+23. Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 581–615. Springer, Cham, August 2023. 2

BBFR24. Ryad Benadjila, Charles Bouillaguet, Thibauld Feneuil, and Matthieu Rivain. MQOM — MQ on my Mind. Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures. 2, 5, 7, 32

BBFR25. Ryad Benadjila, Charles Bouillaguet, Thibauld Feneuil, and Matthieu Rivain. The mqom specification version 2.1 (released on 2025/09/22). Website, September 2025. 2, 7, 18, 27

BCC+24. Dung Bui, Eliana Carozza, Geoffroy Couteau, Dahmun Goudarzi, and Antoine Joux. Faster signatures from MPC-in-the-head. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part I*, volume 15484 of *LNCS*, pages 396–428. Springer, Singapore, December 2024. 5

BCD25. Dung Bui, Kelong Cong, and Cyprien Delpech de Saint Guilhem. Faster VOLEitH signatures from all-but-one vector commitment and half-tree. In Willy Susilo and Josef Pieprzyk, editors, *ACISP 25, Part I*, volume 15658 of *LNCS*, pages 205–223. Springer, Singapore, July 2025. 2, 4, 5, 18, 19

BHH+19. Nina Bindel, Mike Hamburg, Kathrin Hövelmanns, Andreas Hülsing, and Edoardo Persichetti. Tighter proofs of CCA security in the quantum random oracle model. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019, Part II*, volume 11892 of *LNCS*, pages 61–90. Springer, Cham, December 2019. 4, 28, 29

CDG+17. Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, and Greg Zaverucha. Post-quantum zero-knowledge and signatures from symmetric-key primitives. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1825–1842. ACM Press, October / November 2017. 5

CDMP05. Jean-Sébastien Coron, Yevgeniy Dodis, Cécile Malinaud, and Prashant Puniya. Merkle-Damgård revisited: How to construct a hash function. In Victor Shoup, editor, *CRYPTO 2005*, volume 3621 of *LNCS*, pages 430–448. Springer, Berlin, Heidelberg, August 2005. 5

CFHL21. Kai-Min Chung, Serge Fehr, Yu-Hsuan Huang, and Tai-Ning Liao. On the compressed-oracle technique, and post-quantum security of proofs of sequential work. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 598–629. Springer, Cham, October 2021. 5, 36, 37, 38, 41

CKKZ12. Seung Geol Choi, Jonathan Katz, Ranjit Kumaresan, and Hong-Sheng Zhou. On the security of the "free-XOR" technique. In Ronald Cramer, editor, *TCC 2012*, volume 7194 of *LNCS*, pages 39–53. Springer, Berlin, Heidelberg, March 2012. 3

CLY+24. Hongrui Cui, Hanlin Liu, Di Yan, Kang Yang, Yu Yu, and Kaiyi Zhang. ReSolveD: Shorter signatures from regular syndrome decoding and VOLE-in-the-head. In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part I*, volume 14601 of *LNCS*, pages 229–258. Springer, Cham, April 2024. 2, 5, 18, 19

CS14. Shan Chen and John P. Steinberger. Tight security bounds for key-alternating ciphers. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 327–350. Springer, Berlin, Heidelberg, May 2014. 4, 17

DDOS19. Cyprien Delpech de Saint Guilhem, Lauren De Meyer, Emmanuela Orsini, and Nigel P. Smart. BBQ: Using AES in picnic signatures. In Kenneth G. Paterson and Douglas Stebila, editors, *SAC 2019*, volume 11959 of *LNCS*, pages 669–692. Springer, Cham, August 2019. 5

DFMS22a. Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Efficient NIZKs and signatures from commit-and-open protocols in the QROM. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 729–757. Springer, Cham, August 2022. 5, 35, 36, 37, 40

DFMS22b. Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Online-extractability in the quantum random-oracle model. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 677–706. Springer, Cham, May / June 2022. 5, 35, 36, 39

Din24. Itai Dinur. Tight indistinguishability bounds for the XOR of independent random permutations by Fourier analysis. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 33–62. Springer, Cham, May 2024. 5

FOPS01. Eiichiro Fujisaki, Tatsuaki Okamoto, David Pointcheval, and Jacques Stern. RSA-OAEP is secure under the RSA assumption. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 260–274. Springer, Berlin, Heidelberg, August 2001. 10

FOPS04. Eiichiro Fujisaki, Tatsuaki Okamoto, David Pointcheval, and Jacques Stern. RSA-OAEP is secure under the RSA assumption. *Journal of Cryptology*, 17(2):81–104, March 2004. 10

FR23. Thibauld Feneuil and Matthieu Rivain. Threshold linear secret sharing to the rescue of MPC-in-the-head. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part I*, volume 14438 of *LNCS*, pages 441–473. Springer, Singapore, December 2023. 2

FR25. Thibauld Feneuil and Matthieu Rivain. Threshold computation in the head: Improved framework for post-quantum signatures and zero-knowledge arguments. *Journal of Cryptology*, 38(3):28, July 2025. 2

FR26. Thibauld Feneuil and Matthieu Rivain. On the security of MPC-in-the-head signatures with correlated GGM trees. Cryptology ePrint Archive, Paper 2025/1411, 2026. 5

FS87. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO'86*, volume 263 of *LNCS*, pages 186–194. Springer, Berlin, Heidelberg, August 1987. 2, 5

GBJ+23. Aldo Gunsing, Ritam Bhaumik, Ashwin Jha, Bart Mennink, and Yaobin Shen. Revisiting the indifferentiability of the sum of permutations. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part III*, volume 14083 of *LNCS*, pages 628–660. Springer, Cham, August 2023. 5

GGM86. Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, October 1986. 2

GHHM21. Alex B. Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Tight adaptive reprogramming in the QROM. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 637–667. Springer, Cham, December 2021. 16, 28

GKW+20. Chun Guo, Jonathan Katz, Xiao Wang, Chenkai Weng, and Yu Yu. Better concrete security for half-gates garbling (in the multi-instance setting). In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 793–822. Springer, Cham, August 2020. 32, 33, 34

GKWY20. Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. Efficient and secure multiparty computation from fixed-key block ciphers. In *2020 IEEE Symposium on Security and Privacy*, pages 825–841. IEEE Computer Society Press, May 2020. 3, 5, 32, 33, 34

GMO16. Irene Giacomelli, Jesper Madsen, and Claudio Orlandi. ZKBoo: Faster zero-knowledge for Boolean circuits. In Thorsten Holz and Stefan Savage, editors, *USENIX Security 2016*, pages 1069–1083. USENIX Association, August 2016. 2

GYW+23. Xiaojie Guo, Kang Yang, Xiao Wang, Wenhao Zhang, Xiang Xie, Jiang Zhang, and Zheli Liu. Half-tree: Halving the cost of tree expansion in COT and DPF. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part I*, volume 14004 of *LNCS*, pages 330–362. Springer, Cham, April 2023. 2, 3, 4, 5, 18, 19

HJ24. Janik Huth and Antoine Joux. MPC in the head using the subfield bilinear collision problem. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 39–70. Springer, Cham, August 2024. 2, 3, 5

HJ25. Janik Huth and Antoine Joux. VOLE-in-the-head signatures from subfield bilinear collisions. In *ASIACRYPT 2025, Part IV*, pages 3–34, 2025. 2, 5

IKOS07. Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge from secure multiparty computation. In David S. Johnson and Uriel Feige, editors, *39th ACM STOC*, pages 21–30. ACM Press, June 2007. 2

JN18. Ashwin Jha and Mridul Nandi. A survey on applications of H-technique: Revisiting security analysis of PRP and PRF. Cryptology ePrint Archive, Report 2018/1130, 2018. 17

JZM21. Haodong Jiang, Zhenfeng Zhang, and Zhi Ma. On the non-tightness of measurement-based reductions for key encapsulation mechanism in the quantum random oracle model. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 487–517. Springer, Cham, December 2021. 4, 16

KLS24. Seongkwang Kim, Byeonghak Lee, and Mincheol Son. Relaxed vector commitment for shorter signatures. Cryptology ePrint Archive, Report 2024/1004, 2024. 2

KLS25. Seongkwang Kim, Byeonghak Lee, and Mincheol Son. Relaxed vector commitment for shorter signatures. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part IV*, volume 15604 of *LNCS*, pages 427–455. Springer, Cham, May 2025. 2, 3, 5

KM07. Hidenori Kuwakado and Masakatu Morii. Indifferentiability of single-block-length and rate-1 compression functions. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, E90-A(10):2301–2308, 2007. 4, 5

KX25. Haruhisa Kosuge and Keita Xagawa. New security proofs of MPC-in-the-head signatures in the quantum random oracle model. Cryptology ePrint Archive, Report 2025/1999, 2025. 2, 11, 17, 37, 39, 40

NIS22. NIST. Call for additional digital signature schemes for the post-quantum cryptography standardization process, October 2022. 2

Pat09. Jacques Patarin. The "coefficients H" technique (invited talk). In Roberto Maria Avanzi, Liam Keliher, and Francesco Sica, editors, *SAC 2008*, volume 5381 of *LNCS*, pages 328–345. Springer, Berlin, Heidelberg, August 2009. 4, 17

WKK+25. Yalan Wang, Bryan Kumara, Harsh Kasyap, Liqun Chen, Sumanta Sarkar, Christopher J.P. Newton, Carsten Maple, and Ugur Ilker Atmaca. BACON: An improved vector commitment construction with applications to signatures. Cryptology ePrint Archive, Report 2025/1411, 2025. 2, 5

ZCD+19. Greg Zaverucha, Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, Jonathan Katz, Xiao Wang, and Vladmir Kolesnikov. Picnic. Technical report, National Institute of Standards and Technology, 2019. available at https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-2-submissions. 5

Zha19. Mark Zhandry. How to record quantum queries, and applications to quantum indifferentiability. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 239–268. Springer, Cham, August 2019. 5

Supporting Materials

A Parameter Sets of MQOM

Table 1. Parameter Sets of MQOM (v.2.1) [BBFR25]

Parameter Sets	NIST	MQ Param.		Proof System Param.				
	Security	$\lvert\mathbb{F}\rvert$	$m = n$	τ	N	μ	η	w
MQOM2-L1-gf2-short-3r/5r	Cat. I	2	160	12	2048	16	10 / 8	8
MQOM2-L1-gf2-fast-3r/5r	Cat. I	2	160	17	256	8	20 / 16	9
MQOM2-L1-gf16-short-3r/5r	Cat. I	16	56	12	2048	4	14 / 8	8
MQOM2-L1-gf16-fast-3r/5r	Cat. I	16	56	17	256	2	28 / 16	9
MQOM2-L1-gf256-short-3r/5r	Cat. I	256	48	12	2048	2	24 / 8	8
MQOM2-L1-gf256-fast-3r/5r	Cat. I	256	48	17	256	1	48 / 16	9
MQOM2-L3-gf2-short-3r/5r	Cat. III	2	240	18	2048	16	15 / 12	12
MQOM2-L3-gf2-fast-3r/5r	Cat. III	2	240	27	256	8	30 / 24	3
MQOM2-L3-gf16-short-3r/5r	Cat. III	16	84	18	2048	4	21 / 12	12
MQOM2-L3-gf16-fast-3r/5r	Cat. III	16	84	27	256	2	42 / 24	3
MQOM2-L3-gf256-short-3r/5r	Cat. III	256	72	18	2048	2	36 / 12	12
MQOM2-L3-gf256-fast-3r/5r	Cat. III	256	72	27	256	1	72 / 24	3
MQOM2-L5-gf2-short-3r/5r	Cat. V	2	320	25	2048	16	20 / 16	6
MQOM2-L5-gf2-fast-3r/5r	Cat. V	2	320	36	256	8	40 / 32	4
MQOM2-L5-gf16-short-3r/5r	Cat. V	16	116	25	2048	4	29 / 16	6
MQOM2-L5-gf16-fast-3r/5r	Cat. V	16	116	36	256	2	58 / 32	4
MQOM2-L5-gf256-short-3r/5r	Cat. V	256	96	25	2048	2	48 / 16	6
MQOM2-L5-gf256-fast-3r/5r	Cat. V	256	96	36	256	1	96 / 32	4

B Missing Definitions and Lemmas

B.1 3/5-Pass Identification

We consider 3-pass and 5-pass identification (ID) schemes. We only treat *public-coin* ID schemes; that is, the verifier chooses i-th challenge uniformly at random from the challenge set $\mathcal{C}_i$. The syntax follows:

Definition B.1 ($(2n+1)$-pass identification). *A $(2n+1)$-pass ID scheme* ID *consists of the following tuple of PPT algorithms* $(\mathsf{Gen}, \mathsf{P}_1, \mathcal{C}_1, \ldots \mathsf{P}_n, \mathcal{C}_n, \mathsf{P}_{n+1}, \mathsf{V})$*:*

- $\mathsf{Gen}(1^\lambda) \to (\mathtt{vk}, \mathtt{sk})$*: a key-generation algorithm that takes 1^λ as input, where λ is the security parameter, and outputs a pair of keys* $(\mathtt{vk}, \mathtt{sk})$*.* $\mathtt{vk}$ *and* $\mathtt{sk}$ *are public verification and signing keys, respectively.*
- $\mathsf{P}_1(\mathtt{sk}; \rho) \to (a_1, \mathtt{state}_1)$*: a first prover algorithm that takes signing key* $\mathtt{sk}$ *and randomness ρ as input, and outputs the first message a_1 and state* $\mathtt{state}_1$*.*
- $\mathsf{P}_i(c_{i-1}, \mathtt{state}_{i-1}) \to (a_i, \mathtt{state}_i)$*: an i-th prover algorithm for $i = 2, \ldots, n$ that takes the $(i-1)$-th challenge $c_{i-1} \in \mathcal{C}_{i-1}$ and state $\mathtt{state}_{i-1}$ as input, and outputs the i-th message a_i and state $\mathtt{state}_i$.*
- $\mathsf{P}_{n+1}(c_n, \mathtt{state}_n) \to z/\perp$*: the last prover algorithm that takes the n-th challenge $c_n \in \mathcal{C}_n$ and state $\mathtt{state}_n$ as input, and outputs the last message z or $\perp$.*
- $\mathsf{V}(\mathtt{vk}, a_1, c_1, \ldots, a_n, c_n, z) \to 0/1$*: a verification algorithm that takes verification key* $\mathtt{vk}$ *and the transcript $a_1, c_1, \ldots, a_n, c_n, z$ as input and outputs its decision,* 1 *(true) or* 0 *(false).*

We assume perfect correctness; a verifier always outputs 1 *for an arbitrary honestly-generated key and transcript. If $n = 1$, then we will drop some subscripts and write a scheme as* $(\mathsf{Gen}, \mathsf{P}_1, \mathcal{C}, \mathsf{P}_2, \mathsf{V})$ *and a transcript as* (a, c, z)*.*

B.2 Other Security Notions

We consider the following version of PRF security.

Definition B.2 (PRF). *Let* $\mathsf{PRF} : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$ *be a function. We define its advantage as*

$$\mathsf{Adv}^{\mathrm{prf}}_{\mathsf{PRF},\mathcal{A}}(\lambda) := \left| \Pr_{k\leftarrow\mathcal{K}}\left[1 \leftarrow \mathcal{A}^{\mathsf{F}_0}()\right] - \Pr_{\mathsf{F}_1\leftarrow\mathsf{Func}(\mathcal{X},\mathcal{Y})}\left[1 \leftarrow \mathcal{A}^{\mathsf{F}_1}()\right] \right|,$$

where $\mathsf{F}_0(x)$ *returns* $\mathsf{PRF}(k,x)$*. We say that* PRF *is pseudorandom if* $\mathsf{Adv}^{\mathrm{prf}}_{\mathsf{PRF},\mathcal{A}}(\lambda)$ *is negligible for any PPT/QPT adversary* $\mathcal{A}$*.*

Definition B.3 (PRG). *Let* $\mathsf{PRG} : \mathcal{K} \to \mathcal{R}$ *be a function. We define its advantage as*

$$\mathsf{Adv}^{\mathrm{prg}}_{\mathsf{PRG},\mathcal{A}}(\lambda) := \left| \Pr_{k\leftarrow\mathcal{K}}[1 \leftarrow \mathcal{A}(\mathsf{PRG}(k))] - \Pr_{r\leftarrow\mathcal{R}}[1 \leftarrow \mathcal{A}(r)] \right|.$$

We say that PRG *is* pseudorandom *if* $\mathsf{Adv}^{\mathrm{prg}}_{\mathsf{PRG},\mathcal{A}}(\lambda)$ *is negligible for any PPT/QPT adversary* $\mathcal{A}$*.*

B.3 Proof Techniques in QROM

Adaptive Reprogramming [GHHM21]: It is hard to distinguish whether the random oracle is reprogrammed or not if the min-entropy of the reprogrammed point is sufficiently high.

Lemma B.1 ([GHHM21, Theorem 1]). *Let* $\mathcal{X}_1$*,* $\mathcal{X}_2$*,and* $\mathcal{Y}$ *be finite sets, and* $\mathrm{O}_0 = \mathrm{O}_1 \leftarrow \mathsf{Func}(\mathcal{X}_1 \times \mathcal{X}_2, \mathcal{Y})$*. We define an oracle* Repro *as follows: on input* $x_2 \in \mathcal{X}_2$*, it samples* $(x_1, \mathsf{side}) \leftarrow D$ *and* $y \leftarrow \mathcal{Y}$*, reprograms only* O_1 *as* $\mathrm{O}_1[(x_1,x_2) \mapsto y]$*, and returns* (x_1, side)*. We let* D_i *be a distribution that the adversary* $\mathcal{A}$ *queries to* Repro *in the* i*-th query. Suppose that* $\mathcal{A}$ *makes* R *queries to* Repro *and* q *quantum queries to* $|\mathrm{O}_b\rangle$*. Then, the distinguishing advantage of* $\mathcal{A}$ *is bounded by*

$$\left|\Pr\left[1 \leftarrow \mathcal{A}^{|\mathrm{O}_0\rangle,\mathrm{Repro}}()\right] - \Pr\left[1 \leftarrow \mathcal{A}^{|\mathrm{O}_1\rangle,\mathrm{Repro}}()\right]\right| \le \frac{3}{2}\sum_{i\in[R]}\sqrt{q p^{(i)}_{\mathsf{max}}},$$

where we define $p^{(i)}_{\mathsf{max}} := \mathbb{E}\max_{\hat{x}_1}\Pr_{(x_1,\mathsf{side})\leftarrow D_i}[x_1 = \hat{x}_1]$ *with the expectation taken over* $\mathcal{A}$*'s behavior up to the* i*-th query.*

Semi-classical O2H Technique [AHU19]: We define *punctured oracle* following [BHH+19].

Definition B.4 (Punctured Oracle [BHH+19, Definition 1]). *Let* $\mathcal{S} \subset \mathcal{X}$ *be a set and* $\mathsf{f}_{\mathcal{S}} : \mathcal{X} \to \{0,1\}$ *be a predicate that returns* 1 *if and only if* $x \in \mathcal{S}$*.* Punctured oracle $\mathsf{H}\backslash\mathcal{S}$ *(*H *punctured by* $\mathcal{S}$*) of* $\mathsf{H} : \mathcal{X} \to \mathcal{Y}$ *runs as follows: on input* x*, computes whether* $x \in \mathcal{S}$ *in an auxiliary qubit* $|\mathsf{f}_{\mathcal{S}}(x)\rangle$*, measures* $|\mathsf{f}_{\mathcal{S}}(x)\rangle$*, runs* $\mathsf{H}(x)$*, and returns the result. Let* Find *be an event that any of measurements of* $|\mathsf{f}_{\mathcal{S}}(x)\rangle$ *returns* 1*.*

Given $\sum_x \alpha_x |x\rangle|y\rangle$, the punctured oracle $\mathsf{H}\backslash\mathcal{S}$ computes $\sum_x \alpha_x |x\rangle|y\rangle|\mathsf{f}_{\mathcal{S}}(x)\rangle$ and measures the third register. If the result is 0, then the query is transformed to $\sum_{x\notin\mathcal{S}} \alpha_x |x\rangle|y\rangle|0\rangle$ and the punctured oracle $\mathsf{H}\backslash\mathcal{S}$ returns $\sum_{x\notin\mathcal{S}} \alpha_x |x\rangle|y \oplus \mathsf{H}(x)\rangle$ to the adversary. If the result is 1 (and thus, $\mathsf{Find} = \top$ holds), $\mathsf{H}\backslash\mathcal{S}$ returns $\sum_{x\in\mathcal{S}} \alpha_x |x\rangle|y \oplus \mathsf{H}(x)\rangle$ to the adversary.

The following lemma provides a bound on the change in advantage caused by puncturing the oracle, based on the probability that $\mathsf{Find} = \top$.

Lemma B.2 (Semi-classical O2H Technique [AHU19, Theorem 1], simplified). *Let* $\mathcal{S} \subset \mathcal{X}$ *be random. Let* $\mathsf{H}, \mathsf{H}' : \mathcal{X} \to \mathcal{Y}$ *be random functions satisfying* $\forall x \in \mathcal{X}$*,* $\mathsf{H}(x) = \mathsf{H}'(x)$*. Let* z *be a random bitstring. (The tuple* $(\mathcal{S}, \mathsf{H}, \mathsf{H}', z)$ *is taken from an arbitrary joint distribution.) Let* $\mathcal{A}$ *be an oracle quantum adversary issuing at most* q *(quantum) queries, and let* E *be an arbitrary classical event. Then we have*

$$\left|\Pr\left[\mathcal{A}^{|\mathsf{H}\rangle}(z) : \mathsf{E}\right] - \Pr\left[\mathcal{A}^{|\mathsf{H}'\rangle}(z) : \mathsf{E}\right]\right| \le 2\sqrt{(q+1)\Pr\left[\mathcal{A}^{|\mathsf{H}'\backslash\mathcal{S}\rangle}(z) : \mathsf{Find} = \top\right]}.$$

Double-sided O2H Technique [BHH+19]: For a special case, we have the following tighter technique in the O2H.

Lemma B.3 (Double-sided O2H Technique [BHH+19, Lemma 5]). *Let $S \subset \mathcal{X}$ be random. Let $\mathsf{H}, \mathsf{H}' : \mathcal{X} \to \mathcal{Y}$ be random functions such that $\forall x \in \mathcal{X}$, $\mathsf{H}(x) = \mathsf{H}'(x)$. Let z be a random bitstring. The tuple $(\mathsf{H}, \mathsf{H}', S, z)$ may have an arbitrary joint distribution. Let $\mathcal{A}^{|\mathsf{H}\rangle}$ be a quantum oracle algorithm, let $f : \mathcal{X} \to \mathcal{W} \subseteq \{0,1\}^n$ be any function, and let $f(S)$ denote the image of S under f. Let E be an arbitrary classical event. We define a quantum oracle algorithm $\mathcal{B}^{|\mathsf{H}\rangle,|\mathsf{H}'\rangle}(z)$ that runs in about the same time as $\mathcal{A}$ and simulates $\mathcal{A}$, but whenever $\mathcal{A}$ makes a query to H, the algorithm $\mathcal{B}$ queries both H and H' and additionally evaluates f twice. If $f(S) = \{w^*\}$, then $\mathcal{B}$ outputs only either w^* or $\perp$, and*

$$\left|\Pr\left[\mathcal{A}^{|\mathsf{H}\rangle}(z) : \mathsf{E}\right] - \Pr\left[\mathcal{A}^{|\mathsf{H}'\rangle}(z) : \mathsf{E}\right]\right| \leq 2\sqrt{\Pr\left[\mathcal{B}^{|\mathsf{H}\rangle,|\mathsf{H}'\rangle}(z) \in f(S)\right]}.$$

C Missing Proofs

C.1 Proof of Lemma 5.5

Lemma 5.5 (restated). *There exist a domain-extension/auxiliary-input one-way adversary $\mathcal{A}_{\mathrm{ow}}$ and a PRF adversary $\mathcal{A}_{\mathrm{pr}}$ satisfying*

$$|\Pr[\mathsf{G}_{2.2} : W_{2.2}] - \Pr[\mathsf{G}_3 : W_3]| \leq 2\sqrt{\mathsf{Adv}^{\mathrm{daow}}_{(\mathsf{Gen}',\mathsf{H}_{\mathsf{aux}}),\mathcal{A}_{\mathrm{ow}}}(\lambda)} + \mathsf{Adv}^{\mathrm{pr}}_{\mathsf{PRF},\mathcal{A}_{\mathrm{prf}}}(\lambda) + \mathbb{E}[Q_{\mathrm{sol}}^2]/2^{\lambda-1}.$$

Proof. In addition to modifying x as $\delta \| \mathsf{PRF}(K, \delta)$, we introduce the following abort condition: the game aborts if there exists an $x' \neq x$ such that $\hat{F}(x') = \hat{y}$ and $\mathsf{H}_{\mathsf{aux}}(x') = \mathsf{H}_{\mathsf{aux}}(x)$. If there are Q_{sol} solutions x' to $\hat{F}(x') = \hat{y}$, then the probability of the collision event $\mathsf{H}_{\mathsf{aux}}(x') = \mathsf{H}_{\mathsf{aux}}(x)$ is bounded by $Q_{\mathrm{sol}}^2/2^{\lambda}$. Since our security bound is taken in the average case, we may take the expectation over the key-generation distribution, which results in an additional term of $\mathbb{E}[Q_{\mathrm{sol}}^2]/2^{\lambda}$. We use a double prime to denote the games obtained by applying both of the above modifications.

$$|\Pr[\mathsf{G}_{2.2} : W_{2.2}] - \Pr[\mathsf{G}_{2.3} : W_{2.3}]| \leq \left|\Pr[\mathsf{G}''_{2.2} : W_{2.2}] - \Pr[\mathsf{G}''_{2.3} : W_{2.3}]\right| + 2 \cdot \mathsf{Adv}^{\mathrm{pr}}_{\mathsf{PRF},\mathcal{A}_{\mathrm{prf}}}(\lambda) + \mathbb{E}[Q_{sol}^2]/2^{\lambda-1}.$$

We now turn to the main part of this claim. We replace the random function accessible to the adversary. Specifically, we randomize only the portion corresponding to S'' defined as:

$$S'' = \left\{ (\mathtt{node}, t_{\mathtt{salt}}) : \begin{array}{l} \exists q \in [Q_{\mathsf{Sign}}],\ \exists e \in [\tau],\ \exists (i,j) \in \mathcal{I} \\ \text{s.t. } \mathsf{P}_{\delta'}(\mathtt{node}, t_{\mathtt{salt}}, q, e, i, j) = 1, \\ \hat{F}\left(\delta' \| \mathsf{PRF}(K, \delta')\right) = 0, \\ \text{and } \mathsf{H}_{\mathsf{aux}}(\delta' \| \mathsf{PRF}(K, \delta')) = \mathsf{H}_{\mathsf{aux}}(x) \end{array} \right\}, \tag{12}$$

where $\mathsf{P}_{\delta'}$ is defined in Equation 2.

As a condition for applying DS-O2H, we define a function f over S'' such that $f(S'')$ returns a single element δ corresponding to the secret key. Specifically, f operates as follows: for each candidate set of nodes corresponding to the same layer, it computes δ' as defined in Equation 12 and $x' = \mathsf{PRF}(K, \delta')$, and returns δ' if it satisfies $\hat{F}(x') = \hat{y}$ and $\mathsf{H}_{\mathsf{aux}}(x') = \mathsf{H}_{\mathsf{aux}}(x)$. Since we already eliminated the case where there exists an $x' \neq x$ that still satisfies $\hat{F}(x') = \hat{y}$ and $\mathsf{H}_{\mathsf{aux}}(x') = \mathsf{H}_{\mathsf{aux}}(x)$, we ensure that $f(S'') = \{\delta\}$ holds.

Then, we bound the difference in advantages. Let $\mathsf{H}'_{\mathsf{avc}}$ be the modified oracle in this proof. Consider an adversary $\mathcal{B}^{|\mathsf{H}_{\mathsf{avc}}\rangle,|\mathsf{H}'_{\mathsf{avc}}\rangle}(\hat{y}, \mathsf{H}_{\mathsf{aux}}(x))$ that attempts to search for S''. This adversary is given $\hat{y}$ and $\mathsf{H}_{\mathsf{aux}}(x)$, and can recognize whether the value of a queried node can recover x, thereby enabling the construction of f. For the first random function $\mathsf{H}_{\mathsf{avc}}$, we use a standard random function. For the second function $\mathsf{H}'_{\mathsf{avc}}$, it outputs identically to $\mathsf{H}_{\mathsf{avc}}$ as long as f returns $\perp$; otherwise, if f does not return $\perp$, $\mathsf{H}'_{\mathsf{avc}}$ outputs the initially chosen random value. Let Ext be an event that $x' \leftarrow \mathcal{B}^{|\mathsf{H}_{\mathsf{avc}}\rangle,|\mathsf{H}'_{\mathsf{avc}}\rangle}(\hat{y}, \mathsf{H}_{\mathsf{aux}}(x))$ and $x' \in f(S'')$. Then, we have

$$\left|\Pr[\mathsf{G}''_{2.2} : W''_{2.2}] - \Pr[\mathsf{G}''_{2.3} : W''_{2.3}]\right| \leq 2\sqrt{\Pr[\mathsf{G}_{2.3} : \mathsf{Ext}]}.$$

We show that

$$\Pr[\mathsf{G}''_{2.3} : \mathsf{Ext}] \le \mathsf{Adv}^{\mathrm{daow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda). \tag{13}$$

As in the proof of Lemma 5.4, we have $\Pr[\mathsf{G}''_{2.3} : \mathsf{Ext}] = \Pr[\mathsf{G}''_4 : \mathsf{Ext}]$. Therefore, we construct an adversary $\mathcal{A}_{\mathrm{ow}}$ that runs $\mathcal{B}^{|\mathsf{H}_{\mathsf{avc}}\rangle,|\mathsf{H}'_{\mathsf{avc}}\rangle}(\hat{y}, \mathsf{H}_{\mathsf{Gen}}(x))$ in G''_4. The oracles $(\mathsf{H}_{\mathsf{avc}}, \mathsf{H}'_{\mathsf{avc}})$ are simulated as follows. Since the function f satisfying $f(S'') = \{\delta\}$ can be efficiently computed from $(\mathtt{mseed}_{\mathsf{eq}}, \hat{y}, K, \mathsf{H}_{\mathsf{Gen}}(x))$, the adversary can determine whether a queried input belongs to the punctured set S''. Let s denote the rounded input to the oracles. We define $\mathsf{H}_{\mathsf{avc}}$ and $\mathsf{H}'_{\mathsf{avc}}$ so that $\mathsf{H}_{\mathsf{avc}}(s) = \mathsf{H}'_{\mathsf{avc}}(s)$ whenever $f(s) = \bot$, and $\mathsf{H}_{\mathsf{avc}}(s) \neq \mathsf{H}'_{\mathsf{avc}}(s)$ whenever $f(s) \neq \bot$. By construction, the two oracles differ only on the punctured set S'' and remain identical on all other inputs. This enables an efficient simulation of $\mathcal{B}^{|\mathsf{H}_{\mathsf{avc}}\rangle,|\mathsf{H}'_{\mathsf{avc}}\rangle}(\hat{y}, \mathsf{H}_{\mathsf{Gen}}(x))$. Consequently, any adversary that triggers the event Ext can be transformed into an adversary $\mathcal{A}_{\mathrm{ow}}$ that breaks domain-extension/auxiliary-input one-wayness, which proves Equation 13. □

D Examples of H-Coefficeints with Onewayness

D.1 Warmup 1: RP/RF Swithcing Lemma with Onewayness

We begin with the simplest example: the indistinguishability of a random permutation π and a random function ρ with *computational onewayness*. Let Gen be an instance generator outputting $(f, f(x^*), x^*)$, where $f : \{0,1\}^\lambda \to \mathcal{Y}$ and $x^* \in \{0,1\}^\lambda$. We define the onewayness of this instance generator as follows:

Definition D.1. *We say that* Gen *is one-way if for any adversary* $\mathcal{A}$*, its advantage* $\mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}}(\lambda)$ *is negligible in* 1^λ*, where the advantage is defined as follows:*

$$\mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}}(\lambda) := \Pr[(f, f(x^*), x^*) \leftarrow \mathsf{Gen}(1^\lambda), x' \leftarrow \mathcal{A}(f, f(x^*)) : f(x') = f(x^*)].$$

Let $\pi : \{0,1\}^\lambda \to \{0,1\}^\lambda$ be a random permutation. Let $\rho : \{0,1\}^\lambda \to \{0,1\}^\lambda$ be a random function. We define two experiments as follows:

- Real: In the real experiment, the challenger generates $(f, f(x^*), x^*) \leftarrow \mathsf{Gen}(1^\lambda)$, runs $\mathcal{A}^\pi(f, f(x^*), \pi(x^*))$, and outputs $\mathcal{A}$'s output.
- Ideal: In the ideal experiment, the challenger generates $(f, f(x^*), x^*) \leftarrow \mathsf{Gen}(1^\lambda)$, chooses $y^* \leftarrow \mathcal{Y}$, runs $\mathcal{A}^\rho(f, f(x^*), y^*)$, and outputs $\mathcal{A}$'s output.

Theorem D.1. *Let* $\mathcal{A}$ *be a* deterministic *distinguisher making at most* Q *oracle queries. There exists an adversary* $\mathcal{A}_{\mathrm{ow}}$ *satisfying*

$$\left| \Pr_{\pi,\mathsf{Gen}}[\mathsf{Real} = 1] - \Pr_{\rho,\mathsf{Gen},y^*}[\mathsf{Ideal} = 1] \right| \le \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda) + Q(Q+1) \cdot 2^{-(\lambda+1)}.$$

The running time of $\mathcal{A}_{\mathrm{ow}}$ *is that of* $\mathcal{A}$ *plus* $Q \cdot \mathsf{poly}(\lambda)$.

By averaging over the coins of a PPT adversary $\mathcal{A}$, we obtain the following corollary:

Corollary D.1. *Let* $\mathcal{A}$ *be a PPT distinguisher making at most* Q *oracle queries. Then there exists a PPT adversary* $\mathcal{A}_{\mathrm{ow}}$ *such that*

$$\left| \Pr_{\pi,\mathsf{Gen},\mathcal{A}}[\mathsf{Real} = 1] - \Pr_{\rho,\mathsf{Gen},y^*,\mathcal{A}}[\mathsf{Ideal} = 1] \right| \le \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda) + Q(Q+1) \cdot 2^{-(\lambda+1)}.$$

Moreover, the running time of $\mathcal{A}_{\mathrm{ow}}$ *is that of* $\mathcal{A}$ *plus* $Q \cdot \mathsf{poly}(\lambda)$.

Proof (Proof of Theorem). Let us consider a transcript $\omega' = \left((x^{(q)}, y^{(q)})\right)_{q=1,\dots,Q}$ generated by the interaction between $\mathcal{A}$ and its oracle π (resp. ρ), where $x^{(q)}$ denotes the q-th query made by $\mathcal{A}$ and $y^{(q)}$ denotes the corresponding oracle response. We assume that all queries $x^{(1)}, \dots, x^{(Q)}$ are distinct.

We further extend ω' by appending a special pair $(x^{(0)}, y^{(0)})$. In the real experiment, we set $(x^{(0)}, y^{(0)}) := (x^*, \pi(x^*))$, where x^* is queried by the challenger. In the ideal experiment, we set $(x^{(0)}, y^{(0)}) := (\bot, y^*)$, where $\bot$ is a distinguished symbol indicating that this pair is appended artificially and where y^* is sampled independently and uniformly at random from $\{0,1\}^\lambda$. We denote the resulting extended transcript by ω.

Let Θ_{real} and Θ_{ideal} be random variables representing the extended transcript ω in the real and ideal experiments, respectively.

We define the following three subsets of Ω corresponding to bad transcripts:

– Bad_1: There exist $1 \le q < q' \le Q$ such that $y^{(q)} = y^{(q')}$ (i.e., a collision among $y^{(1)}, \dots, y^{(Q)}$).
– Bad_2: There exists $q \in \{1, \dots, Q\}$ such that $x^{(q)} = x^*$ (i.e., $(x^*, *)$ appears in ω').
– Bad_3: There exists $q \in \{1, \dots, Q\}$ such that $y^{(q)} = y^*$ (i.e., $(*, y^*)$ appears in ω').

We define

$$\mathsf{Bad} := \mathsf{Bad}_1 \cup \mathsf{Bad}_2 \cup \mathsf{Bad}_3.$$

It is straightforward to bound the probabilities of these bad events in the ideal experiment:

– $\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}_1]$: Since $y^{(1)}, \dots, y^{(Q)}$ are sampled independently and uniformly at random from $\{0,1\}^\lambda$, we have

$$\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}_1] = 1 - \prod_{i=0}^{Q-1} \left(1 - \frac{i}{2^\lambda}\right) \le \frac{Q(Q-1)}{2^{\lambda+1}}.$$

– $\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}_2]$: This event implies that $\mathcal{A}$ recovers a preimage of $f(x^*)$. Therefore, we can construct a one-wayness adversary $\mathcal{A}_{\mathrm{ow}}$ against Gen as follows: given $(f, f(x^*))$, it runs $\mathcal{A}$ as a subroutine, simulates the ideal experiment for $\mathcal{A}$'s oracle queries, and outputs $\mathcal{A}$'s query x' if $f(x') = f(x^*)$. Hence,

$$\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}_2] \le \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda).$$

– $\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}_3]$: Since y^* is sampled independently and uniformly at random from $\{0,1\}^\lambda$, we have

$$\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}_3] = 1 - \left(1 - \frac{1}{2^\lambda}\right)^Q \le \frac{Q}{2^\lambda}.$$

Next, we compute the likelihood ratio for good transcripts, $\frac{\Pr[\Theta_{\mathrm{real}}=\omega]}{\Pr[\Theta_{\mathrm{ideal}}=\omega]}$. Since x^* never appears among $\mathcal{A}$'s queries on *good* transcripts, we can identify the artificially appended symbol $\perp$ with the fixed label x^* when comparing transcript probabilities. Fix any transcript $\omega \in \mathsf{Good}$, where $\omega = \left((x^{(q)}, y^{(q)})\right)_{q=0,\dots,Q}$. In particular, $\omega \notin \mathsf{Bad}$ implies that (i) $y^{(1)}, \dots, y^{(Q)}$ are pairwise distinct, (ii) $x^* \notin \{x^{(1)}, \dots, x^{(Q)}\}$, and (iii) $y^* \notin \{y^{(1)}, \dots, y^{(Q)}\}$. Moreover, since the additional pair is indexed by $q = 0$, we have $x^{(0)} = x^*$ and $y^{(0)} = y^*$ in the ideal experiment.

In the real experiment, π is a random permutation. Since x^* is distinct from $x^{(1)}, \dots, x^{(Q)}$, the values $\pi(x^*), \pi(x^{(1)}), \dots, \pi(x^{(Q)})$ are distributed as sampling without replacement from $\{0,1\}^\lambda$. Therefore, we have

$$\Pr[\Theta_{\mathrm{real}} = \omega] = \frac{1}{2^\lambda} \cdot \frac{1}{2^\lambda - 1} \cdot \dots \cdot \frac{1}{2^\lambda - Q}.$$

In the ideal experiment, ρ is a random function, and $y^{(0)} = y^*$ as well as $y^{(1)}, \dots, y^{(Q)}$ are sampled independently and uniformly from $\{0,1\}^\lambda$. Thus, we have

$$\Pr[\Theta_{\mathrm{ideal}} = \omega] = \left(\frac{1}{2^\lambda}\right)^{Q+1}.$$

Consequently, for any $\omega \notin \Omega_{\mathrm{bad}}$, we have

$$\frac{\Pr[\Theta_{\mathrm{real}} = \omega]}{\Pr[\Theta_{\mathrm{ideal}} = \omega]} = \prod_{i=0}^{Q} \frac{2^\lambda}{2^\lambda - i} \ge 1,$$

and therefore we can take $\varepsilon_{\mathrm{good}} = 0$.

Applying the H-coefficient technique (Theorem 6.1), we obtain that the following bound: There exists an efficient adversary $\mathcal{A}_{\mathrm{ow}}$ such that

$$\begin{aligned}
&|\Pr[\mathsf{Real} = 1] - \Pr[\mathsf{Ideal} = 1]| \\
&\quad \le \Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}] + 0 \\
&\quad \le \Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}_1] + \Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}_2] + \Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}_3] \\
&\quad \le Q(Q-1)/2^{\lambda+1} + \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda) + Q/2^\lambda.
\end{aligned}$$

□

D.2 Warmup 2: TCCR Hash with Onewayness

Let $E : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda$ be an ideal cipher. For a function $\sigma : \{0,1\}^\lambda \to \{0,1\}^\lambda$, we define $\bar{\sigma} := \{0,1\}^\lambda \to \{0,1\}^\lambda$ by setting $\bar{\sigma}(s) := \sigma(s) \oplus s$. We say that σ is an *orthomorphism* if both σ and $\bar{\sigma}$ are permutations. In what follows, we assume that both σ and $\bar{\sigma}$ are linear and efficiently invertible.

Let us consider the TCCR hash candidate $\mathsf{H} = \mathsf{EncFF}$ defined by

$$\mathsf{H}(s,t) = \mathsf{EncFF}(s,t) := E(t,s) \oplus \sigma(s)$$

as in [BBFR24], which is a variant of the Davies-Meyer transform. [7]

For $x \in \{0,1\}^\lambda$, we define an oracle $\mathcal{O}_x^{\mathrm{TCCR}}$ as

$$\mathcal{O}_x^{\mathrm{TCCR}}(s,t,b) := \mathsf{EncFF}(s \oplus x, t) \oplus b \cdot x = E(t, s \oplus x) \oplus \sigma(s \oplus x) \oplus b \cdot x.$$

We want to show that the following games are computationally indistinguishable:

$$\begin{aligned}
\mathsf{Real} &: (f, f(x), x) \leftarrow \mathsf{Gen}(1^\lambda), E \leftarrow \mathsf{IC}_\lambda; \\
&\quad b \leftarrow \mathcal{A}^{E^\pm, \mathcal{O}_x^{\mathrm{TCCR}}}(f, f(x)), \text{ output } b \\
\mathsf{Ideal} &: (f, f(x), x) \leftarrow \mathsf{Gen}(1^\lambda), E \leftarrow \mathsf{IC}_\lambda, \rho \leftarrow \mathsf{Func}(\{0,1\}^{2\lambda+1}, \{0,1\}^\lambda); \\
&\quad b \leftarrow \mathcal{A}^{E^\pm, \rho}(f, f(x)), \text{ output } b,
\end{aligned}$$

where the distinguisher is required *not* to query both $(s,t,0)$ and $(s,t,1)$ to its oracle for any $(s,t) \in \{0,1\}^{2\lambda}$.

We modify the security proof of TCCR hash for $\widehat{\mathsf{MMO}}^E(s,t) := E(t, \sigma(s)) \oplus \sigma(s)$ in Guo et al. [GKWY20]. While Guo et al. [GKW+20] showed the multi-instance TCCR (miTCCR) property, where an adversary is given to multiple oracles, we only need TCCR because the secret x is shared among all signatures. In other words, when we consider the multi-use EUF-CMA secuirty, we need to consider the miTCCR property of $\mathsf{EncFF} = \widehat{\mathsf{MMO}}$.

Theorem D.2 (TCCR with onewayness). *Let $\mathcal{A}$ be a PPT distinguisher that queries to the oracle $\mathcal{O}$ and $E^\pm$ at most $Q_\mathcal{O}$-times and Q_E-times. There exists an adversary $\mathcal{A}_{\mathrm{ow}}$ satisfying*

$$\left| \Pr_{E,\mathsf{Gen},\mathcal{A}}[\mathsf{Real} = 1] - \Pr_{E,\rho,\mathsf{Gen},\mathcal{A}}[\mathsf{Ideal} = 1] \right| \le \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda) + Q_\mathcal{O}^2/2^\lambda.$$

The running time of $\mathcal{A}_{\mathrm{ow}}$ is that of $\mathcal{A}$ plus $Q_\mathcal{O} Q_E \cdot \mathsf{poly}(\lambda)$.

Proof (proof of theorem). By fixing the random tape of $\mathcal{A}$ and averaging over it, it suffices to prove the claim for deterministic distinguishers. Hence, fix an arbitrary deterministic distinguisher $\mathcal{A}$ and show that

$$\left| \Pr_{E,\mathsf{Gen}}[\mathsf{Real} = 1] - \Pr_{E,\rho,\mathsf{Gen}}[\mathsf{Ideal} = 1] \right| \le \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda) + Q_\mathcal{O}^2 \cdot 2^{-\lambda}.$$

Transcripts: In the real experiment, the oracle $\mathcal{O}_x^{\mathrm{TCCR}}$ is defined by

$$\mathcal{O}_x^{\mathrm{TCCR}}(s,t,b) := \mathsf{EncFF}(s \oplus x, t) \oplus b \cdot x = E(t, s \oplus x) \oplus \sigma(s \oplus x) \oplus b \cdot x.$$

In the ideal experiment, $\mathcal{A}$ instead has access to a random function $\rho \leftarrow \mathsf{Func}(\{0,1\}^{2\lambda+1}, \{0,1\}^\lambda)$.

We record the interaction of $\mathcal{A}$ with its oracles by a transcript $\mathcal{Q} := (\mathcal{Q}_\mathcal{O}, \mathcal{Q}_E, x)$, where x is the secret sampled by Gen. Here, $\mathcal{Q}_\mathcal{O}$ stores all query–answer tuples (s,t,b,z) made to the $\mathcal{O}$-oracle, and $\mathcal{Q}_E$ stores all ideal-cipher query–answer tuples under $E^\pm$. For convenience, we represent $\mathcal{Q}_E$ as a set of triples (t,u,v) meaning that $E(t,u) = v$. We only consider attainable transcripts induced by $\mathcal{A}$.

For each $t \in \{0,1\}^\lambda$, let $\mathcal{Q}_E[t] := \{(u,v) : (t,u,v) \in \mathcal{Q}_E\}$ denote the set of cipher relations under key t. We also define $\mathcal{Q}_\mathcal{O}[t] := \{(s,t,b) : (s,t,b,z) \in \mathcal{Q}_\mathcal{O}\}$.

[7] The original Davies-Meyer transform converts $E : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda$ to $\mathsf{DM}^E(s,t) = E(t,s) \oplus s$.

Bad transcripts: We define the following subsets of bad transcripts:

- Bad_1: There exists a query $(s,t,b,z) \in \mathcal{Q}_{\mathcal{O}}$ and a query $(t, s \oplus x, *)$ or $(t, *, \sigma(s \oplus x) \oplus b \cdot x \oplus z)$ in $\mathcal{Q}_E$.
- Bad_2: There exist two distinct tuples $(s_i, t_i, b_i, z_i) \neq (s_j, t_j, b_j, z_j)$ in $\mathcal{Q}_{\mathcal{O}}$ such that

$$t_i = t_j \quad \text{and} \quad \sigma(s_i) \oplus b_i \cdot x \oplus z_i = \sigma(s_j) \oplus b_j \cdot x \oplus z_j.$$

Note that Bad_2 forces a collision among the (same-key) outputs of the permutation $E_{t_i}(\cdot)$ on the internal inputs $s_i \oplus x$ and $s_j \oplus x$.

Following [GKWY20, GKW+20], let us bound ϵ_{bad}, the probabilities that a transcript in the ideal world is in those two bad sets.

Bounding $\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}_1]$ *(one-wayness reduction):* In [GKWY20, GKW+20], Guo et al. fixed some $(s,t,b,z) \in \mathcal{Q}_{\mathcal{O}}$ and showed that $\Pr[(t, s \oplus x, *) \in \mathcal{Q}_E] \leq |\mathcal{Q}_E[t]|/2^{\rho}$, where ρ is the min-entropy of x. They conclude that $\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}_1] \leq 2\mu p/2^{\rho}$ in [GKW+20, Theorem 2] or $2qp/2^{\rho}$ in [GKWY20, Theorem 5] (we skip the definition of p, q, μ, etc). Unfortunately, our setting degrades the min-entropy of x to 0 in the worst case since the distinguisher is given $(f, f(x))$. Instead, we show that if this event occurs, then one can find x' violating one-wayness.

Concretely, we can write a reduction algorithm $\mathcal{A}_{\mathrm{ow}}$ as follows:

1. Receive $(f, f(x))$. Initialize two tables $\mathcal{Q}_{\mathcal{O}}$ and $\mathcal{Q}_E$.
2. Run $\mathcal{A}$ and simulate the oracles as follows:
   - $\rho(s,t,b)$: This is implemented by lazy sampling with a table $\mathcal{Q}_{\mathcal{O}}$. If $(s,t,b,z) \in \mathcal{Q}_{\mathcal{O}}$, then answer z. If $(s,t,1-b,z) \in \mathcal{Q}_{\mathcal{O}}$, then abort. Otherwise, sample $z \leftarrow \{0,1\}^{\lambda}$, store (s,t,b,z) to $\mathcal{Q}_{\mathcal{O}}$. It then check Bad_1 as follows:
     - For every $(u,v) \in \mathcal{Q}_E[t]$, compute the candidate $x' := s \oplus u$. If $f(x') = f(x)$, then output x' and halt.
     - For every $(u,v) \in \mathcal{Q}_E[t]$, compute $w := v \oplus z \oplus \sigma(s)$. If $b = 0$ and $f(\sigma^{-1}(w)) = f(x)$, then output $x' := \sigma^{-1}(w)$ and halt. If $b = 1$ and $f(\bar{\sigma}^{-1}(w)) = f(x)$, then output $x' := \bar{\sigma}^{-1}(w)$ and halt.

     If not, return z to $\mathcal{A}$.
   - $E^{+}(t,u)$: This is implemented by lazy sampling with table $\mathcal{Q}_E$. If $(t,u,v) \in \mathcal{Q}_E$, then answer v. Otherwise, sample v consistency with $\mathcal{Q}_E[t]$ and store (t,u,v) to $\mathcal{Q}_E$. It then check Bad_1 as follows:
     - For every $(s,b,z) \in \mathcal{Q}_{\mathcal{O}}[t]$, compute $x' := s \oplus u$. If $f(x') = f(x)$, then output x' and halt.
     - For every $(s,b,z) \in \mathcal{Q}_{\mathcal{O}}[t]$, compute $w := v \oplus z \oplus \sigma(s)$. If $b = 0$ and $f(\sigma^{-1}(w)) = f(x)$, then output $x' := \sigma^{-1}(w)$ and halt. If $b = 1$ and $f(\bar{\sigma}^{-1}(w)) = f(x)$, then output $x' := \bar{\sigma}^{-1}(w)$ and halt.

     If not, return v to $\mathcal{A}$.
   - $E^{-}(t,v)$: This is implemented by lazy sampling with table $\mathcal{Q}_E$. If $(t,u,v) \in \mathcal{Q}_E$, then answer u. Otherwise, sample u consistency with $\mathcal{Q}_E[t]$ and store (t,u,v) to $\mathcal{Q}_E$. It then check Bad_1 and if it happens it output x' and halt as in the above. It not, return u to $\mathcal{A}$.

It is easy to check if a transcript is in Bad_1, then the reduction algorithm $\mathcal{A}_{\mathrm{ow}}$ catches the event and output x', the preimage of $f(x)$. Suppose that there exist $(s,t,b,z) \in \mathcal{Q}_{\mathcal{O}}$ and $(t,u,v) \in \mathcal{Q}_E$ satisfying $u = s \oplus x$. By our definition, the first check correctly detects this event and outputs $x' = s \oplus u$ after checking $f(x') = f(x)$. Next, suppose that there exist $(s,t,b,z) \in \mathcal{Q}_{\mathcal{O}}$ and $(t,u,v) \in \mathcal{Q}_E$ satisfying $v = \sigma(s \oplus x) \oplus b \cdot x \oplus z$, which implies $\sigma(x) \oplus b \cdot x = v \oplus z \oplus \sigma(s) =: w$. Then, the second check correctly detects this event and outputs $x' = \sigma^{-1}(w)$ or $\bar{\sigma}^{-1}(w)$, depending on the value of b, after checking $f(x') = f(x)$. Hence, we have

$$\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}_1] \leq \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(1^{\lambda}).$$

Bounding $\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}_2]$ *(collision probability):* This part is the same as that in [GKWY20]. Let us fix $t \in \{0,1\}^{\lambda}$. We consider two distinct queries (s,t,b,z) and $(s',t,b',z') \in \mathcal{Q}_{\mathcal{O}}$. We first claim that $\sigma(s \oplus x) \neq \sigma(s' \oplus x)$. To show it, we assume that $\sigma(s \oplus x) = \sigma(s' \oplus x)$. If so, then $s = s'$ since σ is a linear orthomorphism. However, if $s = s'$, then the two queries are (s,t,b,z) and (s,t,b',z'). Since we restrict the distinguisher's query, b should be equal to b'. But, this means $z = z'$, and this contradicts our assumption, two *distinct* queries. We then have

$$\begin{aligned}&\Pr[\sigma(s \oplus x) \oplus b \cdot x \oplus z = \sigma(s' \oplus x) \oplus b' \cdot x \oplus z'] \\ &= \Pr[\sigma(s) \oplus \sigma(s') \oplus (b \oplus b')x = z \oplus z'] \leq 2^{-\lambda},\end{aligned}$$

since z and z' are chosen uniformly at random from $\{0,1\}^{\lambda}$. Thus, fixing t, for any pair of distinct queries in $\mathcal{Q}_{\mathcal{O}}$, the probability that a transcript is in Bad_2 is at most $1/2^{\lambda}$. Considering all queries, we have

$$\Pr[\Theta_{\mathrm{ideal}} \in \mathsf{Bad}_2] \leq \sum_t \binom{|\mathcal{Q}_{\mathcal{O}}[t]|}{2} \cdot 2^{-\lambda} \leq Q_{\mathcal{O}}^2/2^{\lambda}.$$

Likelihood ratio on good transcripts: We then fix good transcripts $\mathsf{Good} := \Omega \setminus (\mathsf{Bad}_1 \cup \mathsf{Bad}_2)$. Following the arguments in [GKWY20, GKW+20], we have that $\epsilon_{\mathrm{bad}} = 0$.

Conclusion via the H-coefficient technique: Applying the H-coefficient technique (Theorem 6.1), we obtain that there exists a PPT machine $\mathcal{A}_{\mathrm{ow}}$ satisying

$$\begin{aligned}|\Pr[\mathsf{Real}=1] - \Pr[\mathsf{Ideal}=1]| &\leq \epsilon_{\mathrm{good}} + \epsilon_{\mathrm{bad}} \\ &\leq 0 + \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda) + Q_{\mathcal{O}}^2/2^{\lambda}\end{aligned}$$

as we wanted. □

E Proof of the EUF-NMA Security of MQOM

To show the EUF-NMA security, we consider a 3-round underlying protocol $\mathsf{ID3} = (\mathsf{Gen}, \mathsf{P}_1, C, \mathsf{P}_2, \mathsf{V})$, where $\eta = \hat{m}$, and a variant of $\widetilde{\mathsf{MQOM}}$.

Definition E.1. *The definition of* $\mathsf{ID3} = (\mathsf{Gen}, \mathsf{P}_1, C, \mathsf{P}_2, \mathsf{V})$ *follows:*

- $\mathsf{P}_1(\mathtt{sk}; \mathtt{salt}, \mathtt{mseed}) \to (a = (\mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2, \Delta_x^{(1)}), \mathtt{state})$*: The first prover takes* $x \in \mathbb{F}^n$*, salt* $\mathtt{salt} \in \{0,1\}^{\lambda}$*, and randomness* $\mathtt{mseed} \in \{0,1\}^{\lambda}$*. For each* $e \in [\tau]$*, the prover commits polynomials* $P_x[e](X) = x_0[e] + x \cdot X \in (\mathbb{K}[X])^n$ *and* $P_u[e](X) = u_0[e] + u_1[e] \cdot X \in (\mathbb{K}[X])^{\eta}$ *into* $h_{\mathtt{com}}[e]$ *and computes* $\mathtt{com}_1 := \mathsf{Hash}_7(h_{\mathtt{com}}, \Delta_x^{(1)})$*. It then computes polynomials* $P_\alpha[e](X) = \alpha_0[e] + \alpha_1[e] \cdot X \in (\mathbb{K}[X])^n$ *from* $\mathtt{vk}$*,* $P_x[e]$*, and* $P_u[e]$*, for each* $e \in [\tau]$*; specifically, it computes*

$$\begin{aligned} P_\alpha[e](X) &:= P_u[e](X) + P_z[e](X) \in (\mathbb{K}[X])^{\eta}, \\ \text{where } P_z[e](X)_i &:= P_x[e](X)^{\top} \cdot \hat{A}_i \cdot P_x[e](X) + \hat{b}_i^{\top} \cdot P_x[e](X) \cdot X - \hat{y}_i \cdot X^2 \in \mathbb{K}[X]. \end{aligned} \tag{14}$$

  It then computes $\mathtt{com}_2 := \mathsf{Hash}_3(\alpha_0, \alpha_1)$*. It outputs* $a = (\mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2)$*.*
- *The challenge space is* $C = \{[N] \setminus \{i\} \mid i \in [N]\}^{\tau}$*. The challenge is* $c = ([N] \setminus \mathbf{i}^*[e])_{e \in [\tau]}$*, which is represented as* $\mathbf{i}^* \in [N]^{\tau}$*.*
- $\mathsf{P}_2(c, \mathtt{state}) \to z = (\mathtt{seed}_c, \mathtt{com}_{-c}, \alpha_0, \alpha_1, \Delta_x^{(1)})$*: For each* $e \in [\tau]$*, the second prover reveals* $x_{\mathtt{eval}}[e] := P_x[e](\omega_{\mathbf{i}^*[e]})$ *and* $u_{\mathtt{eval}}[e] := P_u[e](\omega_{\mathbf{i}^*[e]})$*. This is done by sending* $(\mathtt{seed}_c, \mathtt{com}_{-c})$*, where* $\mathtt{seed}_c$ *and* $\mathtt{com}_{-c}$ *is defined as*

$$\mathtt{seed}_c := \left((\mathtt{seed}[e][i])_{i \in c[e]}\right)_{e \in [\tau]} \text{ and } \mathtt{com}_{-c} := (\mathtt{com}[e][\mathbf{i}^*[e]])_{e \in [\tau]},$$

  i.e., an expanded form of $\mathtt{pdecom}$*.*
  Note that we can consider this protocol as a commit-and-open *protocol.*
- $\mathsf{V}(\mathtt{vk}, a, c, z) \to 0/1$*: It first verifies the validity of* $\mathtt{com}_1$ *by reconstructing it from* $\mathtt{seed}_c$*,* $\mathtt{com}_{-c}$*, and* $\Delta_x^{(1)}$ *and the validity of* $\mathtt{com}_2$ *from* α_0 *and* α_1*. The verifier then checks if, for each e,* $P_\alpha(\omega_{\mathbf{i}^*[e]})$ *is equivalent to*

$$\begin{aligned} \alpha_{\mathtt{eval}}[e] &:= u_{\mathtt{eval}}[e] + z_{\mathtt{eval}}[e], \\ \text{where } z_{\mathtt{eval},i}[e] &:= x_{\mathtt{eval}}[e]^{\top} \cdot \hat{A}_i \cdot x_{\mathtt{eval}}[e] + \omega_{\mathbf{i}^*[e]} \cdot \hat{b}_i^{\top} \cdot x_{\mathtt{eval}}[e] - \omega_{\mathbf{i}^*[e]}^2 \cdot \hat{y}_i. \end{aligned} \tag{15}$$

  We then define the variant of MQOM as $\widetilde{\mathsf{MQOM}} := \mathsf{FS}_g[\mathsf{ID3}, \mathsf{Hash}_4, \mathsf{XOF}_5]$.

Definition E.2. $\widetilde{\mathsf{MQOM}} = (\mathsf{Gen}, \widetilde{\mathsf{Sign}}, \widetilde{\mathsf{Vrfy}})$ *is defined as follows:*

- $\widetilde{\mathsf{Sign}}$*: On input* $\mathtt{sk}$ *and* $h_{\mathtt{msg}}$*, the signing algorithm runs* P_1 *and obtains* $a = (\mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2)$ *and* $\mathtt{state}$*. It then computes* $h_2 := \mathsf{Hash}_4(\mathtt{vk}, \mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2, h_{\mathtt{msg}})$*. Let* $\mathtt{nonce} := 0$*. It computes* $(\mathbf{i}^*, v_{\mathrm{grinding}}) := \mathsf{XOF}_5(h_2, \mathtt{nonce})$ *and computes* $z = (\mathtt{seed}_c, \mathtt{com}_{-c}, \alpha_0, \alpha_1, \Delta_x^{(1)}) := \mathsf{P}_2(\mathbf{i}^*, \mathtt{state})$*. If* $v_{\mathrm{grinding}} = 0^w$*, then it outputs* $\tilde{\sigma} := (a, \mathtt{nonce}, z)$ *as a signature. If not, it increments* $\mathtt{nonce}$ *and retries to obtain a signature.*
- $\widetilde{\mathsf{Vrfy}}$*: On input* $\mathtt{vk}$*,* $h_{\mathtt{msg}}$*, and* $\tilde{\sigma} = (a, \mathtt{nonce}, z)$*, it recomputes* $h_2 := \mathsf{Hash}_4(\mathtt{vk}, \mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2, h_{\mathtt{msg}})$ *and* $(\mathbf{i}^*, v_{\mathrm{grinding}}) := \mathsf{XOF}_5(h_2, \mathtt{nonce})$*. It then outputs* 1 *if* $v_{\mathrm{grinding}} = 0^w$ *and* $\mathsf{V}(\mathtt{vk}, a, c, z) = 1$*.*

It is easy to show that if $\widetilde{\mathsf{MQOM}}$ is EUF-NMA-secure, then MQOM is EUF-NMA-secure.

Theorem E.1. *If* $\widetilde{\mathsf{MQOM}}$ *is EUF-NMA-secure, then* MQOM *is EUF-NMA-secure. In other words, if there exists* $\mathcal{A}$ *against* MQOM *who makes* Q_X *queries to the oracle* X*, then there exists* $\tilde{\mathcal{A}}$ *against* $\widetilde{\mathsf{MQOM}}$ *who makes* $\tilde{Q}_X$ *queries to the oracle* X *satisfying*

$$\mathsf{Adv}^{\text{euf-nma}}_{\mathsf{MQOM},\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\text{euf-nma}}_{\widetilde{\mathsf{MQOM}},\tilde{\mathcal{A}}}(\lambda).$$

The running time of $\tilde{\mathcal{A}}$ *is about that of* $\mathcal{A}$ *plus a verification time.*

- *For* $\widetilde{\mathsf{MQOM}}_1$*, we have* $\tilde{Q}_{\mathsf{H}_{\mathsf{avc},0}} := Q_{\mathsf{H}_{\mathsf{avc},0}} + \tau(N-1)$, $\tilde{Q}_{\mathsf{H}_{\mathsf{avc},1}} := Q_{\mathsf{H}_{\mathsf{avc},1}} + \tau(N-1)$, $\tilde{Q}_{\mathsf{Hash}_3} := Q_{\mathsf{Hash}_3}$, $\tilde{Q}_{\mathsf{Hash}_4} := Q_{\mathsf{Hash}_4} + 1$, $\tilde{Q}_{\mathsf{XOF}_5} := Q_{\mathsf{XOF}_5} + 1$, $\tilde{Q}_{\mathsf{Hash}_6} := Q_{\mathsf{Hash}_6}$*, and* $\tilde{Q}_{\mathsf{Hash}_7} := Q_{\mathsf{Hash}_7}$*, where we separate* $\mathsf{H}_{\mathsf{avc}}$*'s query depending on* $i \in [4]$ *in* $t_{\mathtt{salt},i,e,j}$ *computed by* TweakSalt′ *and we only treat two random oracles corresponding to* $i = 0$ *and* 1*.*
- *For* $\widetilde{\mathsf{MQOM}}_2$*, we have* $\tilde{Q}_E := Q_E + 4\tau(N-1)$, $\tilde{Q}_{\mathsf{Hash}_3} := Q_{\mathsf{Hash}_3}$, $\tilde{Q}_{\mathsf{Hash}_4} := Q_{\mathsf{Hash}_4} + 1$, $\tilde{Q}_{\mathsf{XOF}_5} := Q_{\mathsf{XOF}_5} + 1$, $\tilde{Q}_{\mathsf{Hash}_6} := Q_{\mathsf{Hash}_6}$*, and* $\tilde{Q}_{\mathsf{Hash}_7} := Q_{\mathsf{Hash}_7}$*.*

Hence, we consider the EUF-NMA security of $\widetilde{\mathsf{MQOM}}$.

Proof. Let $\mathcal{A}$ be an EUF-NMA adversary against MQOM. We consider the following reduction adversary $\tilde{\mathcal{A}}$.

1. Receive input $\mathtt{vk}$, run $\mathcal{A}$ on input $\mathtt{vk}$.
2. Receive $\mathtt{msg}$ and $\sigma = (\mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2, \alpha_1, \mathtt{pdecom}, \Delta_x^{(1)}, \mathtt{nonce})$ from $\mathcal{A}$.
3. Compute $h_{\mathtt{msg}} := \mathsf{Hash}_2(\mathtt{msg})$.
4. Compute $h_2 := \mathsf{Hash}_4(\mathtt{vk}, \mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2, h_{\mathtt{msg}})$, $(\mathbf{i}^*, v_{\text{grinding}}) := \mathsf{XOF}_5(h_2, \mathtt{nonce})$, and expand $\mathtt{pdecom}$ into $\mathtt{seed}_c$ and $\mathtt{com}_{-c}$. Compute α_0 by using $\alpha_{\mathtt{eval}}$ as in MQOM.Vrfy. Set $a := (\mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2)$ and $c = \mathbf{i}^*$, and $z := (\mathtt{seed}_c, \mathtt{com}_{-c}, \alpha_0, \alpha_1, \Delta_x^{(1)})$. Output $\mathtt{msg}$ and $\tilde{\sigma} = (a, \mathtt{nonce}, z)$.

We show that, if $\mathcal{A}$ wins the game, then $\tilde{\mathcal{A}}$ also wins its own game. Suppose that $\mathcal{A}$ wins the game, that is, $(\mathtt{msg}, \sigma)$ is a valid signature. Since MQOM.Vrfy outputs 1, we have $v_{\text{grinding}} = 0^w$ and $(\mathtt{com}_1, \mathtt{com}_2) = (\mathtt{com}_1', \mathtt{com}_2')$. Recall that Vrfy verifies the signature as follows:

1. Compute $h_{\mathtt{msg}} := \mathsf{Hash}_2(\mathtt{msg})$.
2. Compute $h_2 := \mathsf{Hash}_4(\mathtt{vk}, \mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2, h_{\mathtt{msg}})$.
3. Compute $(\mathbf{i}^*, v_{\text{grinding}}) := \mathsf{XOF}_5(h_2, \mathtt{nonce})$.
4. Expand $\mathtt{pdecom}$ into $\mathtt{seed}_c$ and $\mathtt{com}$ as defined in cAVC.Recon.
5. For each $e \in [\tau]$, compute $h_{\mathtt{com}}[e] := \mathsf{Hash}_6(\mathtt{com}[e])$. Compute $\mathtt{com}_1' := \mathsf{Hash}_7(h_{\mathtt{com}}, \Delta_x^{(1)})$.
6. For each $e \in [\tau]$, compute $x_{\mathtt{eval}}[e]$, $u_{\mathtt{eval}}[e]$, and $\alpha_{\mathtt{eval}}[e]$, and compute $\alpha_0[e] := \alpha_{\mathtt{eval}}[e] - \alpha_1[e] \cdot \omega_{\mathbf{i}^*[e]}$.
7. Compute $\mathtt{com}_2' := \mathsf{Hash}_3(\alpha_0, \alpha_1)$.
8. If $v_{\text{grinding}} = 0^w$ and $(\mathtt{com}_1, \mathtt{com}_2) = (\mathtt{com}_1', \mathtt{com}_2')$, then return 1.

Notice that the condition that $\alpha_{\mathtt{eval}}[e](\omega_{\mathbf{i}^*[e]}) = \alpha_0[e] + \alpha_1[e] \cdot \omega_{\mathbf{i}^*[e]}$ in V is equivalent to the condition that $\alpha_0[e] := \alpha_{\mathtt{eval}}[e] - \alpha_1[e] \cdot \omega_{\mathbf{i}^*[e]}$, which always holds due to the definition of MQOM.Vrfy. Hence, $(h_{\mathtt{msg}}, \tilde{\sigma})$ is a valid signature for $\widetilde{\mathsf{MQOM}}$ and this completes the proof. □

E.1 Preliminaries

Preliminaries: We review a notion of special and k-soundness, $\mathfrak{S}$-soundness, and some definitions for our bounds.

$\mathfrak{S}$-soundness: We review $\mathfrak{S}$-soundness of the commit-and-open (C&O) Σ-protocol $\Pi = (\tilde{\mathsf{P}}_1, \mathcal{C}, \tilde{\mathsf{P}}_2, \tilde{\mathsf{V}})$ defined in Don et al. [DFMS22b]. We adapt the definition in [DFMS22a, Section 3.2]. Let H be a random oracle.

- $\tilde{\mathsf{P}}_1$ first sends $a_\circ$ and $\mathtt{com} = (\mathtt{com}_i)_{i \in [\ell]}$, where $\mathtt{com}_i := H(\mathtt{seed}_i)$.
- The verifier sends a random challenge $c \leftarrow \mathcal{C} \subseteq 2^\ell$.
- $\tilde{\mathsf{P}}_2$ opens $\mathtt{seed}_c = \{\mathtt{seed}_i\}_{i \in c}$.
- $\tilde{\mathsf{V}}$ takes $\mathtt{vk}$, $a_\circ$, $\mathtt{com}$, c, and $\mathtt{seed}_c$ as input and checks if $H(\mathtt{seed}_i) = \mathtt{com}_i$ for all $i \in c$ and $V(\mathtt{inst}, c, \mathtt{seed}_c, a_\circ) = 1$, where V is an algorithm to verify the relation.

Based on the above, Don et al. considered a Merkle-tree-based C&O Σ-protocol in [DFMS22a, Section 5]:

- $\tilde{\mathsf{P}}_1$ first sends $a_\circ$ and $y := \mathsf{MerkleCom}^H(\mathtt{seed})$.
- The verifier sends a random challenge $c \leftarrow \mathcal{C} \subseteq 2^\ell$.
- $\tilde{\mathsf{P}}_2$ opens $\mathtt{seed}_c = \{\mathtt{seed}_i\}_{i \in c}$ and $\pi := \mathsf{MerkleOpen}^H(\mathtt{seed}, c)$.

– $\tilde{\mathsf{V}}$ takes $\mathtt{vk}$, $a_\circ$, $\mathtt{com}$, c, and $\mathtt{seed}_c$ as input and checks if $\mathsf{MerkleVrfy}^H(c, y, \mathtt{seed}_c, \pi) = 1$ and $V(\mathtt{inst}, c, \mathtt{seed}_c, a_\circ) = 1$, where V is an algorithm to verify the relation.

Suppose that the challenge space is $\mathcal{C} \subseteq 2^{[\ell]}$. Let $\mathfrak{S} \subseteq 2^{\mathcal{C}}$ be a non-empty, monotone increasing set of subset $S \subseteq \mathcal{C}$.[8] We also define $\mathfrak{S}_{\min} := \{S \in \mathfrak{S} \mid S' \subset S \Rightarrow S' \notin \mathfrak{S}\}$, the minimal sets in $\mathfrak{S}$.

In our context of the underlying basic protocol, the adversary could win at most two challenges because $P_\alpha[e]$'s degree is at most 2 (as in Equation 14) and $P_\alpha[e](X) = 0$ can have at most two solutions in $\Phi = \{\omega_0, \dots, \omega_{N-1}\} \subseteq \mathbb{K}$. Thus, we consider 3-soundness and $\mathfrak{S}$ is $\{S \subseteq \mathcal{C} \mid |S| \geq 3\}$, and $\mathfrak{S}_{\min} = \{S \subseteq \mathcal{C} \mid |S| = 3\}$. Don et al. defined a notion of special soundness as follows:

Definition E.3 ([DFMS22b, Definition 5.1] and [DFMS22a, Definition 3.4]). *The C&O Σ protocol Π is said to be* $\mathfrak{S}$-sound *if there exists an efficient extractor that takes* $\mathtt{inst}$, $\mathtt{seed} \in (\{0,1\}^\lambda \cup \{\bot\})^\ell$, *a string* $a_\circ$, *and a set* $S \in \mathfrak{S}_{\min}$ *and outputs a witness for* $\mathtt{inst}$ *if* $V(\mathtt{inst}, c, \mathtt{seed}_c, a_\circ) = 1$ *for all* $c \in S$.

For $\mathfrak{S}$-sound Σ protocol, we define

$$p^{\mathfrak{S}}_{\mathrm{triv}} := \frac{1}{|C|} \max_{\hat{S} \subseteq \mathcal{C} : \hat{S} \notin \mathfrak{S}} |\hat{S}|.$$

This captures a trivial attack with a challenge set $\hat{S} = \{\hat{c}_1, \dots, \hat{c}_m\} \notin \mathfrak{S}$ that prepares $\mathtt{seed}$ and $a_\circ$ satisfying $V(\mathtt{inst}, c, \mathtt{seed}_c, a_\circ)$ holds if $c \in \hat{S}$. In our context, the challenge set is $\mathcal{C} := \{[N] \setminus \{i\} \mid i \in [N]\}$.

Now, let us consider MQOM's 3-round ID protocol, ID3 in Definition E.1, *without shallow Merkle-tree commitment.*[9] By eliminating the Merkele-tree commitment, this can be considered as a τ-parallel composition of the basic protocol. According to the discussion in [DFMS22b, Section 5.2], the τ-parallel repetition of an arbitrary $\mathfrak{S}$-sound protocol is $\mathfrak{S}^{\vee\tau}$-sound with

$$\mathfrak{S}^{\vee\tau} := \{S \subseteq \mathfrak{C}^\tau \mid \exists i \,:\, S_i \in \mathfrak{S}\},$$

where $S_i := \{c \in \mathfrak{C} \mid \exists (c_1, \dots, c_\tau) \in S \,:\, c_i = c\}$ is the i-th marginal of S. In our case, the basic protocol without Merkle-tree commitment is (computationally) 3-sound and $p_{\mathrm{triv}} \leq 2/N$. According to the discussion in [DFMS22b, Section 5], ID3 with and without Merkle-tree commitment has the same trivial attack advantage as

$$p^{\mathfrak{S}}_{\mathrm{triv}} \leq (2/N)^\tau. \tag{16}$$

Database: We next review the database and its properties [CFHL21].

Let $D : \mathcal{X} \to \mathcal{Y} \cup \{\bot\}$ be a function, which we call a database. Let $\mathfrak{D}$ be a set of databases $D : \mathcal{X} \to \mathcal{Y} \cup \{\bot\}$. For a database $D \in \mathfrak{D}$, we define a database $D[x \mapsto y]$ as $D[x \mapsto y](x) = y$ and $D[x \mapsto y](\bar{x}) = D(y)$ for $\bar{x} \neq x$. For $y \in \mathcal{Y}$, we define $D^{-1}(y)$ to be the smallest $x \in \mathcal{X}$ such that $D(x) = y$. For convinience, we define $D^{-1}(y) := \bot$ if there is no such x.

A database property P on $\mathfrak{D}$ is a subset of D. We will simulate random oracles and ideal ciphers by using a database in the classical setting. In the quantum setting, we will use the compressed-oracle technique to simulate random oracles.

Definition E.4 (Classical transition capacity [CFHL21]). *Let* P *and* P' *be two database properties. Then, the* classical transition capacity *between them is defined as*

$$[\mathsf{P} \to \mathsf{P}'] := \max_{x \in \mathcal{X}, D \in \mathsf{P}} \Pr_{y \leftarrow \mathcal{Y}}[D[x \mapsto y] \in \mathsf{P}'].$$

We denote the probability that $D \in \mathsf{P}$ after $\mathcal{A}$ makes Q queries to H implemented with the database D by $[\bot \Longrightarrow_Q \mathsf{P}]$.

We will consider multiple oracles, say, X_1 and X_2, and unify their database as $D := D_{X_1} \cup D_{X_2}$. If we put (x, y) to D_{X_1}, we will denote $D[x \mapsto_{X_1} y]$, which should be understood as $D_{X_1}[x \mapsto y] \cup D_{X_2}$.

[8] We say $\mathfrak{S}$ is *monotone increasing* if, for any S and S', $S \in \mathfrak{S}$ and $S \subseteq S'$ implies $S' \in \mathfrak{S}$.

[9] P_1 sends $\mathtt{com}$ and $\Delta^{(1)}_x$ directly instead of sending $\mathtt{com}_1 := \mathsf{Hash}_7(h_{\mathtt{com}}, \Delta^{(1)}_x)$ with $h_{\mathtt{com}}[e] := \mathsf{Hash}_6(\mathtt{com}[e])$.

E.2 Proof of the EUF-NMA Security of MQOM_1 in the ROM

Theorem E.2 (EUF-NMA security of $\widetilde{\mathsf{MQOM}}_1$ in the ROM). *Let* $\mathsf{H}_{\mathsf{com}}$, Hash_3, Hash_4, XOF_5, Hash_6, *and* Hash_7 *be random oracles. For a PPT adversary* $\tilde{\mathcal{A}}$ *that queries the oracles* X *at most* $\tilde{Q}_X$ *times, there exists a PPT adversary* $\mathcal{A}_{\mathrm{ow}}$ *such that*

$$\mathsf{Adv}^{\text{euf-nma}}_{\widetilde{\mathsf{MQOM}}_1,\tilde{\mathcal{A}}}(\lambda) \leq \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda) + \tilde{Q}' \cdot \max\{\tilde{Q}'\tau N \cdot 2^{-\lambda}, (2/N)^{\tau} \cdot 2^{-w}\},$$

where $\tilde{Q}' = \tilde{Q}_{\mathsf{H}_{\mathsf{ayc},0}} + \tilde{Q}_{\mathsf{H}_{\mathsf{avc},1}} + \tilde{Q}_{\mathsf{Hash}_3} + \tilde{Q}_{\mathsf{Hash}_4} + \tilde{Q}_{\mathsf{XOF}_5} + \tilde{Q}_{\mathsf{Hash}_6} + \tilde{Q}_{\mathsf{Hash}_7} + 2\tau(N-1) + \tau + 4$. *The running time of* $\mathcal{A}_{\mathrm{ow}}$ *is that of* $\tilde{\mathcal{A}}$ *plus* $\mathsf{poly}(\tilde{Q}', \lambda)$.

Proof: We follow the EUF-NMA security proof in [DFMS22a, KX25].

We use the *classical transition capacity* [CFHL21] to show the EUF-NMA security of $\widetilde{\mathsf{MQOM}}_1$. In what follows, each random oracle X is implemented with a database D_X, and we unify the databases into D. Before going to the main proof, we introduce additional algorithms:

Definition E.5 (Seed inverter Inv for $\widetilde{\mathsf{MQOM}}_1$).

1. *Receive input* $\mathtt{salt}$, $\mathtt{com}_1$, *and* D.
2. *Search* $(h_{\mathtt{com}}, \Delta_x^{(1)}) = D^{-1}_{\mathsf{Hash}_7}(\mathtt{com}_1)$*; if there is no entry, then return* $\perp$.
3. *For each* $e \in [\tau]$, *search* $\mathtt{com}[e] = D^{-1}_{\mathsf{Hash}_6}(h_{\mathtt{com}}[e])$*; if there is no entry, then return* $\perp$.
4. *For each* $(e,i) \in [\tau] \times [N]$, *parse* $\mathtt{com}[e][i] = c_0 \,\|\, c_1$ *and search* $(t_{\mathtt{salt},0,e,0}, s) = D^{-1}_{\mathsf{H}_{\mathsf{avc}},0}(c_0)$ *and* $(t_{\mathtt{salt},1,e,0}, s') = D^{-1}_{\mathsf{H}_{\mathsf{avc}},1}(c_1)$*; if there is no entry or* $s \neq s'$*, then set* $\mathtt{seed}[e][i] = \perp$. *Otherwise, set* $\mathtt{seed}[e][i] := s$.
5. *Return* $\mathtt{seed} \in (\{0,1\}^{\lambda} \cup \{\perp\})^{\tau N}$.

We then define the extractor Ext as follows:

Definition E.6 (Online extractor Ext for $\widetilde{\mathsf{MQOM}}_1$).

1. *Receive* $\mathsf{inst} = (\mathtt{vk}, \mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2)$, $\mathbf{i}^*$, $\mathtt{aux} = (\alpha_0, \alpha_1, \Delta_x^{(1)}, \mathtt{com}_{-c})$, *and* $\mathtt{seed} \in (\{0,1\}^{\lambda} \cup \{\perp\})^{\tau N}$.
2. *Find* $e^* \in [\tau]$ *such that* $\mathtt{seed}[e^*][i] \neq \perp$ *for all* $i \in [N]$*; if cannot find, then output* $\perp$.
3. *For each* $(e,i) \in [\tau] \times [N]$, *if* $\mathtt{seed}[e][i] \neq \perp$, *then compute* $\mathtt{com}[e][i] := D_{\mathsf{H}_{\mathsf{com}}}(\mathtt{salt}, e, i, \mathtt{seed}[e][i])$. *Else, set* $\mathtt{com}[e][i] := \perp$.
4. *For each* e^* *found in Step 2:*
   (a) *Find three distinct* $i_0^*, i_1^*, i_2^* \in [N]$ *such that, for all* $j \in [3]$, *we have*

$$\mathsf{V}^D(\mathtt{vk}, a, c_j = \mathbf{i}_j^*, z_j = (\mathtt{seed}_{c_j}, \mathtt{com}_{-c_j}, \alpha_0, \alpha_1, \Delta_x^{(1)})) = 1,$$

   where $\mathbf{i}_j^*$ *is obtained by replacing* $\mathbf{i}^*[e^*]$ *with* i_j^* *and* c_j *is a challenge set corresponding to* $\mathbf{i}_j^*$*; if those indices cannot be found, then output* $\perp$.
5. *For* $i \in [N]$, *compute*

$$m_i := \mathtt{seed}[e^*][i] \,\|\, \mathsf{H}_{\mathsf{avc}}(\mathtt{seed}[e^*][i], t_{\mathtt{salt},3,e^*,0}) \,\|\, \ldots \,\|\, \mathsf{H}_{\mathsf{avc}}(\mathtt{seed}[e^*][i], t_{\mathtt{salt},3,e^*,n_b-1})$$

   and parse $(\bar{x}[e^*][i], \bar{u}[e^*][i]) := \mathsf{Parse}(m_i)$. *Return*

$$x' := (0^{\lambda} \,\|\, \Delta_x^{(1)}[e^*]) + \sum_{i \in [N]} \bar{x}[e^*][i].$$

Following [KX25], we check the soundness thrice in Step 4-(a) to ensure that the extracted $\mathtt{seed}$ violates 3-soundness of the protocol, while we do not need to check it explicitly. Those checks ensure that x' satisfies $\hat{F}(x') = 0$ as discussed in [KX25, Appendix E.2].

By using those algorithms, we can adopt the EUF-NMA security proof of commit-and-open protocol in [DFMS22a]. We consider the experiment defined as follows:

1. Run $\tilde{\mathcal{A}}$ on input $\mathtt{vk}$ and receive $(h_{\mathtt{msg}}, \tilde{\sigma})$, a forgery against $\widetilde{\mathsf{MQOM}}_1$.
2. Run $\widetilde{\mathsf{Vrfy}}$ on input $\mathtt{vk}$, $h_{\mathtt{msg}}$, and $\tilde{\sigma}$. If its output is 0, then abort the experiment. Otherwise, parse $\tilde{\sigma} = (a, \mathtt{nonce}, z)$, $a = (\mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2)$, and $z = (\mathtt{seed}_c, \mathtt{com}_{-c}, \alpha_0, \alpha_1, \Delta_x^{(1)})$. Let D be the database for $\mathsf{H}_{\mathsf{com}}$, $\mathsf{H}_{\mathsf{tape}}$, Hash_3, Hash_4, XOF_5, Hash_6, and Hash_7 after $\tilde{\mathcal{A}}$ and $\widetilde{\mathsf{Vrfy}}$'s queries.
3. Run Inv on input $\mathtt{salt}$, $\mathtt{com}_1$, and D and receive $\mathtt{seed}$.

4. Run Ext on input $\mathsf{inst} = (\mathtt{vk}, \mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2)$, $\mathbf{i}^*$ (computed in $\widetilde{\mathsf{Vrfy}}$), and $\mathtt{aux} = (\alpha_0, \alpha_1, \Delta_x^{(1)}, \mathtt{com}_{-c})$, and receive x'.
5. Return x'.

For ease of notation, we denote this experiment as $x' \leftarrow \mathsf{Ext} \circ \mathcal{A}(\mathtt{vk})$. We then have

$$\begin{aligned}\mathsf{Adv}^{\text{euf-nma}}_{\widetilde{\mathsf{MQOM}},\mathcal{A}}(\lambda) &\le \Pr[(\mathtt{vk},\mathtt{sk}) \leftarrow \mathsf{Gen}(1^\lambda); x' \leftarrow \mathsf{Ext} \circ \mathcal{A}(\mathtt{vk}) \,:\, \hat{F}(x') = 0] \\ &\quad + \Pr[(\mathtt{vk},\mathtt{sk}) \leftarrow \mathsf{Gen}(1^\lambda); x' \leftarrow \mathsf{Ext} \circ \mathcal{A}(\mathtt{vk}) \,:\, \hat{F}(x') \neq 0] \\ &\le \mathsf{Adv}^{\text{ow}}_{\mathsf{Gen},\mathcal{A}_{\text{ow}}}(\lambda) + \epsilon_{\text{snd}},\end{aligned}$$

where we consider $\mathsf{Ext} \circ \mathcal{A}_{\text{nma}}$ as $\mathcal{A}_{\text{ow}}$ and we define

$$\epsilon_{\text{snd}} := \Pr[(\mathtt{vk},\mathtt{sk}) \leftarrow \mathsf{Gen}(1^\lambda); x' \leftarrow \mathsf{Ext} \circ \mathcal{A}(\mathtt{vk}) \,:\, \hat{F}(x') \neq 0].$$

We note that $\mathcal{A}_{\text{ow}}$ internally simulates random oracles (and an ideal cipher) and maintains the database D. The numbers of queries $\mathcal{A}$ and $\widetilde{\mathsf{Vrfy}}$ made are $\tilde{Q}'_{\mathsf{H}_{\mathsf{avc}},0} := \tilde{Q}'_{\mathsf{H}_{\mathsf{avc}},0} + \tau(N-1)$, $\tilde{Q}'_{\mathsf{H}_{\mathsf{avc}},1} := \tilde{Q}'_{\mathsf{H}_{\mathsf{avc}},1} + \tau(N-1)$, $\tilde{Q}'_{\mathsf{Hash}_3} := \tilde{Q}_{\mathsf{Hash}_3} + 1$, $\tilde{Q}'_{\mathsf{Hash}_4} := \tilde{Q}_{\mathsf{Hash}_4} + 1$, $\tilde{Q}'_{\mathsf{XOF}_5} := \tilde{Q}_{\mathsf{XOF}_5} + 1$, $\tilde{Q}'_{\mathsf{Hash}_6} := \tilde{Q}_{\mathsf{Hash}_6} + \tau$, and $\tilde{Q}'_{\mathsf{Hash}_7} := \tilde{Q}_{\mathsf{Hash}_7} + 1$.

Evaluating ϵ_{snd} via classical capacity: What remains is evaluating ϵ_{snd} formally. We consider the database properties defined as follows:

$$\begin{aligned}\mathsf{SZ}_{\le k} &:= \{D \mid |D| \le k\} \\ \mathsf{CL}_{\mathsf{Hash}} &:= \{D \mid \exists H, x \neq x' \,:\, D_H(x) = D_H(x') \neq \bot\} \\ \mathsf{SUC} &:= \left\{ D \;\middle|\; \begin{array}{c} \exists h_{\mathtt{msg}}, \mathtt{inst} = (\mathtt{vk}, \mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2), \mathtt{nonce}, \mathtt{aux} = (\alpha_0, \alpha_1, \Delta_x^{(1)}, \mathtt{com}_{-c}) \\ \text{so that } \mathtt{seed} := \mathsf{Inv}_D(\mathtt{com}_1) \text{ satisfies} \\ 1)\ \mathsf{V}^D(\mathtt{vk}, a, \mathbf{i}^*, z) = 1 \text{ and } v_{\text{grinding}} = 0^w \\ \text{for } a = (\mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2),\ z = (\mathtt{seed}_c, \mathtt{com}_{-c}, \alpha_0, \alpha_1, \Delta_x^{(1)}), \\ \text{and } (\mathbf{i}^*, v_{\text{grinding}}) = D_{\mathsf{XOF}_5}(D_{\mathsf{Hash}_4}(\mathtt{vk}, \mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2, h_{\mathtt{msg}}), \mathtt{nonce}) \\ 2)\ \hat{F}(\mathsf{Ext}(\mathtt{inst}, \mathbf{i}^*, \mathtt{aux}, \mathtt{seed})) \neq 0 \end{array} \right\}.\end{aligned}$$

We note that V only accessed to $D_{\mathsf{H}_{\mathsf{com}}}$, D_{Hash_3}, D_{Hash_6} and D_{Hash_7} to verify a transcript. We consider *classical transition capacity* [CFHL21] and show the following lemma.

Lemma 5.5. *For every $\tilde{Q}' \in \mathbb{N}$,*

$$[\bot \Longrightarrow_{\tilde{Q}'} \mathsf{SUC} \cup \mathsf{CL}] \le \tilde{Q}' \cdot \max\{\tilde{Q}'\tau N \cdot 2^{-\lambda}, p^{\mathfrak{S}}_{\text{triv}} \cdot 2^{-w}\}.$$

Proof. By routine calculations, we have

$$\begin{aligned}[\bot \Longrightarrow_{\tilde{Q}'} \mathsf{SUC} \cup \mathsf{CL}] &\le \sum_{k \in [\tilde{Q}']} [\mathsf{SZ}_{\le k} \setminus (\mathsf{SUC} \cup \mathsf{CL}) \to \mathsf{SUC} \cup \mathsf{CL}] \\ &\le \sum_{k \in [\tilde{Q}']} [\mathsf{SZ}_{\le k} \setminus \mathsf{CL} \to \mathsf{CL}] + \sum_{k \in [\tilde{Q}']} [\mathsf{SZ}_{\le k} \setminus \mathsf{SUC} \to \mathsf{SUC}].\end{aligned}$$

For a collision for $\mathsf{Hash}_6, \mathsf{Hash}_7$, etc, we have the bound $[\mathsf{SZ}_{\le k} \setminus \mathsf{CL} \to \mathsf{CL}] \le (k+1)/2^{2\lambda} \le \tilde{Q}'/2^{2\lambda}$. For a collision for hash functions inside $\mathsf{cAVC}_{\mathsf{RO}}$, we have $[\mathsf{SZ}_{\le k} \setminus \mathsf{CL} \to \mathsf{CL}] \le \max\{(k+1)/2^{2\lambda}, (k+1)/2^{n_b\lambda}\} \le \tilde{Q}'/2^{\lambda}$. Thus, for all $k \in [\tilde{Q}']$, we have

$$[\mathsf{SZ}_{\le k} \setminus \mathsf{CL} \to \mathsf{CL}] \le \tilde{Q}' \cdot 2^{-\lambda}.$$

We then consider $[\mathsf{SZ}_{\le k} \setminus \mathsf{SUC} \to \mathsf{SUC}]$. Let s and u be a query and an answer. We have the following cases depending on the queried functions:

- $\mathsf{H}_{\mathsf{avc}}$ with $b = 0$ or 1: In this case, $s = (t_{\mathtt{salt},b,e,0}, \mathtt{seed}[e][i])$ and randomly chosen $u \leftarrow \{0,1\}^{2\lambda}$ satisfies the condition in SUC. Since the number of the images $c_0 \| c_1 = \mathtt{com}[e][i]$ is at most $k\tau N$ when $D \in \mathsf{SZ}_{\le k}$, we have $\Pr_u[D[s \mapsto_{\mathsf{H}_{\mathsf{com}}} u] \in \mathsf{SUC}] \le k\tau N/2^{\lambda} \le \tilde{Q}'\tau N/2^{\lambda}$.
- Hash_3: In this case, $s = (\alpha_0, \alpha_1)$ and randomly chosen $u \leftarrow \{0,1\}^{2\lambda}$ satisfies the condition in SUC. Since the number of the image $\mathtt{com}_2$ is at most k, we have $\Pr_u[D[s \mapsto_{\mathsf{Hash}_3} u] \in \mathsf{SUC}] \le k/2^{2\lambda} \le \tilde{Q}'/2^{2\lambda}$.

– Hash_4: In this case, $s = (\mathtt{vk}, \mathtt{com}_1, \mathtt{com}_2, h_{\mathtt{msg}})$ and randomly chosen $u \leftarrow \{0,1\}^{2\lambda}$ satisfies the condition in SUC: Since the number of the image h_2 is at most k, we have $\Pr_u[D[s \mapsto_{\mathsf{Hash}_4} u] \in \mathsf{SUC}] \le k/2^{2\lambda} \le \tilde{Q}'/2^{2\lambda}$.
– XOF_5: In this case, $s = (h_2, \mathtt{nonce})$ and randomly chosen $(u, v) \leftarrow [N]^\tau \times \{0,1\}^w$ satisfies the condtion in SUC. For fixed, a and z, we define a boolean predicate $\mathsf{GoodV}(c) : [N]^\tau \to \{0,1\}$ that is 1 if and only if $\mathsf{V}^D(\mathtt{vk}, a, c, z) = 1$.

$$\begin{aligned}
&\Pr_{(u,v)}[D[s \mapsto_{\mathsf{XOF}_5} (u,v)] \in \mathsf{SUC}] \\
&\le \Pr_{(u,v)}[\mathsf{GoodV}(u) = 1 \wedge v = 0^w \wedge \hat{F}(\mathsf{Ext}(h_{\mathtt{msg}}, \mathtt{inst}, \mathtt{aux}, \mathtt{seed})) \neq 0] \\
&\le \Pr_{(u,v)}[\mathsf{GoodV}(u) = 1 \wedge v = 0^w \wedge S := \{c \mid \mathsf{GoodV}(c)\} \notin \mathfrak{S}] \\
&\le \Pr_u[u \in S := \{c \mid \mathsf{GoodV}(c)\} \notin \mathfrak{S}] \cdot \Pr_v[v = 0^w] \\
&\le \left(\max_{S \notin \mathfrak{S}} \Pr_{u \leftarrow [N]^\tau}[u \in S] \right) \cdot 2^{-w} = p^{\mathfrak{S}}_{\mathrm{triv}} \cdot 2^{-w}.
\end{aligned}$$

– Hash_6: In this case, s is of the form $\mathtt{com}[e]$ and randomly chosen $u \leftarrow \{0,1\}^{2\lambda}$ satisfies the condition in SUC: Since the number of the image $h_{\mathtt{com}}[e]$ is at most $k\tau$, we have $\Pr_u[D[s \mapsto_{\mathsf{Hash}_6} u] \in \mathsf{SUC}] \le k\tau/2^{2\lambda} \le \tilde{Q}'\tau/2^{2\lambda}$.
– Hash_7: In this case, $s = (h_{\mathtt{com}}, \Delta^{(1)}_x)$ and randomly chosen $u \leftarrow \{0,1\}^{2\lambda}$ satisfies the condition in SUC: Since the number of the image $\mathtt{com}_1$ is at most k, we have $\Pr_u[D[s \mapsto_{\mathsf{Hash}_7} u] \in \mathsf{SUC}] \le k/2^{2\lambda} \le \tilde{Q}'/2^{2\lambda}$.

Thus, wrapping up, we have

$$\begin{aligned}
[\mathsf{SZ}_{\le k} \setminus \mathsf{SUC} \to \mathsf{SUC}] &\le \max\{\tilde{Q}'\tau \cdot 2^{-2\lambda}, \tilde{Q}'\tau N \cdot 2^{-\lambda}, p^{\mathfrak{S}}_{\mathrm{triv}} \cdot 2^{-w}\} \\
&= \max\{\tilde{Q}'\tau N \cdot 2^{-\lambda}, p^{\mathfrak{S}}_{\mathrm{triv}} \cdot 2^{-w}\}.
\end{aligned}$$

□

In the classical case, the simulation by database D is perfect. Thus, we have

$$\begin{aligned}
\epsilon_{\mathrm{snd}} &= \Pr[\widetilde{\mathsf{Vrfy}}^D(\mathtt{vk}, h_{\mathtt{msg}}, \tilde{\sigma}) = 1 \wedge \hat{F}(x') \neq 0] \\
&\le \Pr[\widetilde{\mathsf{Vrfy}}^D(\mathtt{vk}, h_{\mathtt{msg}}, \tilde{\sigma}) = 1 \wedge \hat{F}(x') \neq 0 \mid D \notin \mathsf{SUC} \cup \mathsf{CL}] + \Pr[D \in \mathsf{SUC} \cup \mathsf{CL}] \\
&\le 0 + [\bot \Longrightarrow_{\tilde{Q}'} \mathsf{SUC} \cup \mathsf{CL}] \\
&\le 0 + \tilde{Q}' \cdot \max\{\tilde{Q}'\tau N \cdot 2^{-\lambda}, p^{\mathfrak{S}}_{\mathrm{triv}} \cdot 2^{-w}\} \\
&\le \tilde{Q}' \cdot \max\{\tilde{Q}'\tau N \cdot 2^{-\lambda}, (2/N)^\tau \cdot 2^{-w}\},
\end{aligned}$$

where we use Equation 16. □

E.3 Proof of the EUF-NMA Security of MQOM_1 in the QROM

The following theorem is obtained by adjusting the parameters in theorems [DFMS22b] as in [AHJ+23, KX25].

Theorem E.3 (EUF-NMA Security of $\widetilde{\mathsf{MQOM}}_1$ in the QROM). *Let* $\mathsf{H}_{\mathsf{com}}$, Hash_3, Hash_4, XOF_5, Hash_6, *and* Hash_7 *be random oracles. For a QPT adversary* $\mathcal{A}$ *that queries the oracles* X *at most* $\tilde{Q}_X$ *times, there exists a QPT adversary* $\mathcal{A}_{\mathrm{ow}}$ *such that*

$$\begin{aligned}
\mathsf{Adv}^{\text{euf-nma}}_{\widetilde{\mathsf{MQOM}}_1, \mathcal{A}}(\lambda) &\le \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen}, \mathcal{A}_{\mathrm{ow}}}(\lambda) \\
&\quad + (\tilde{Q}')^2 \left(\sqrt{10 \max\left\{ \frac{\tilde{Q}'(\tau N + \tau + 1)}{2^\lambda}, \frac{2^\tau}{N^\tau 2^w} \right\}} + 2e\sqrt{\frac{\tilde{Q}' + 1}{N^\tau 2^w}} \right)^2 \\
&\quad + 2(\tau N + 4) \cdot 2^{-\lambda}.
\end{aligned}$$

where $\tilde{Q}' = \tilde{Q}_{\mathsf{H}_{\mathsf{avc}},0} + \tilde{Q}_{\mathsf{H}_{\mathsf{avc}},1} + \tilde{Q}_{\mathsf{Hash}_3} + \tilde{Q}_{\mathsf{Hash}_4} + \tilde{Q}_{\mathsf{XOF}_5} + \tilde{Q}_{\mathsf{Hash}_6} + \tilde{Q}_{\mathsf{Hash}_7} + 2\tau N + 4$. *The running time of* $\mathcal{A}_{\mathrm{ow}}$ *is that of* $\tilde{\mathcal{A}}$ *plus* $\mathsf{poly}(\tilde{Q}', \lambda)$.

E.4 Proof of the EUF-NMA Security of MQOM_2 in the ROM + ICM

Theorem E.4 (EUF-NMA security of $\widetilde{\mathsf{MQOM}}_2$ in the ROM + ICM). *Let E be an ideal cipher and let Hash_3, Hash_4, XOF_5, Hash_6, and Hash_7 be random oracles. For a PPT adversary $\tilde{\mathcal{A}}$ that queries the oracles X at most $\tilde{Q}_X$ times, there exists a PPT adversary $\mathcal{A}_{\mathrm{ow}}$ such that*

$$\mathsf{Adv}^{\text{euf-nma}}_{\widetilde{\mathsf{MQOM}}_2,\tilde{\mathcal{A}}}(\lambda) \leq \mathsf{Adv}^{\mathrm{ow}}_{\mathsf{Gen},\mathcal{A}_{\mathrm{ow}}}(\lambda) + \tilde{Q}' \cdot \max\{4\tilde{Q}'\tau N \cdot 2^{-\lambda}, (2/N)^{\tau} \cdot 2^{-w}\},$$

where $\tilde{Q}' = \tilde{Q}_E + \tilde{Q}_{\mathsf{Hash}_3} + \tilde{Q}_{\mathsf{Hash}_4} + \tilde{Q}_{\mathsf{XOF}_5} + \tilde{Q}_{\mathsf{Hash}_6} + \tilde{Q}_{\mathsf{Hash}_7} + 3\tau(N-1) + \tau + 4$. The running time of $\mathcal{A}_{\mathrm{ow}}$ is that of $\tilde{\mathcal{A}}$ plus $\mathsf{poly}(\tilde{Q}', \lambda)$.

Proof: We follow the EUF-NMA security proof in [DFMS22a, KX25].

We again introduce additional algorithms:

Definition E.7 (Seed inverter Inv for $\widetilde{\mathsf{MQOM}}_2$).

1. *Receive input* $\mathtt{salt}$, $\mathtt{com}_1$, *and* D.
2. *Search* $h_{\mathtt{com}} = D^{-1}_{\mathsf{Hash}_7}(\mathtt{com}_1)$*; if there is no entry, then return* $\perp$.
3. *Foe each* $e \in [\tau]$, *search* $\mathtt{com}[e] = D^{-1}_{\mathsf{Hash}_6}(h_{\mathtt{com}}[e])$*; if there is no entry, then return* $\perp$.
4. *For each* $(e,i) \in [\tau] \times [N]$, *search* $(t_{\mathtt{salt},0,e,0}, s, y)$ *and* $(t_{\mathtt{salt},1,e,0}, s, y')$ *from* D_E *such that* $\mathtt{com}[e][i] = (y \oplus \sigma(s)) \| (y' \oplus \sigma(s))$*; if there are no such entries, then set* $\mathtt{seed}[e][i] = \perp$.
5. *Return* $\mathtt{seed} \in (\{0,1\}^{\lambda} \cup \{\perp\})^{\tau N}$.

We then define the extractor Ext as follows:

Definition E.8 (Online extractor Ext for $\widetilde{\mathsf{MQOM}}_2$).

1. *Receive* $\mathtt{inst} = (\mathtt{vk}, \mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2)$, $\mathbf{i}^*$, $\mathtt{aux} = (\alpha_0, \alpha_1, \mathtt{com}_{-c})$, *and* $\mathtt{seed} \in (\{0,1\}^{\lambda} \cup \{\perp\})^{\tau N}$.
2. *Find* $e^* \in [\tau]$ *such that* $\mathtt{seed}[e^*][i] \neq \perp$ *for all* $i \in [N]$*; if cannot find, then output* $\perp$.
3. *For each* $(e,i) \in [\tau] \times [N]$, *if* $\mathtt{seed}[e][i] \neq \perp$, *then compute* $\mathtt{com}[e][i] := c_0 \| c_1$, *where* $c_0 := D_E(t_{\mathtt{salt},0,e,0}, \mathtt{seed}[e][i]) \oplus \sigma(\mathtt{seed}[e][i])$ *and* $c_1 := D_E(t_{\mathtt{salt},1,e,0}, \mathtt{seed}[e][i]) \oplus \sigma(\mathtt{seed}[e][i])$. *Else, set* $\mathtt{com}[e][i] := \perp$.
4. *For each* e^* *found in Step 2:*
   (a) *Find three distinct* $i^*_0, i^*_1, i^*_2 \in [N]$ *such that, for all* $j \in [3]$, *we have*

   $$\mathsf{V}^D(\mathtt{vk}, a, c_j = \mathbf{i}^*_j, z_j = (\mathtt{seed}_{c_j}, \mathtt{com}_{-c_j}, \alpha_0, \alpha_1)) = 1,$$

   where $\mathbf{i}^*_j$ *is obtained by replacing* $\mathbf{i}^*[e^*]$ *with* i^*_j *and* c_j *is a challenge set corresponding to* $\mathbf{i}^*_j$*; if those indices cannot be found, then output* $\perp$.
5. *For* $i \in [N]$, *set* $\bar{x}[e^*][i] := \mathtt{seed}[e^*][i]$ *and* $\bar{u}[e^*][i] := D_E(t_{\mathtt{salt},3,e,0}, \mathtt{seed}[e^*][i]) \oplus \sigma(\mathtt{seed}[e^*][i])$. *Return*

$$x' := \sum_{i \in [N]} \bar{x}[e^*][i].$$

Following [KX25], we add the check in Step 4-(a) to ensure that extracted $\mathtt{seed}$ violates 3-soundness of the protocol while we do not need to check it explicitly. Those checks ensure that x' satisfies $\hat{F}(x') = 0$ as discussed in [KX25, Appendix E.2].

By using those algorithms, we can adopt the EUF-NMA security proof of commit-and-open protocol in [DFMS22a]. We consider the experiment defined as follows:

1. Run $\tilde{\mathcal{A}}$ on input $\mathtt{vk}$ and receive $(h_{\mathtt{msg}}, \tilde{\sigma})$, a forgery against $\widetilde{\mathsf{MQOM}}_2$.
2. Run $\widetilde{\mathsf{Vrfy}}$ on input $\mathtt{vk}$, $h_{\mathtt{msg}}$, and $\tilde{\sigma}$. If its output is 0, then abort the experiment. Otherwise, parse $\tilde{\sigma} = (a, \mathtt{nonce}, z)$, $a = (\mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2)$, and $z = (\mathtt{seed}_c, \mathtt{com}_{-c}, \alpha_0, \alpha_1)$. Let D be the database for E, Hash_3, Hash_4, XOF_5, Hash_6, and Hash_7 after $\tilde{\mathcal{A}}$ and $\widetilde{\mathsf{Vrfy}}$'s queries.
3. Run Inv on input $\mathtt{salt}$, $\mathtt{com}_1$, and D and receive $\mathtt{seed}$.
4. Run Ext on input $\mathtt{inst} = (\mathtt{vk}, \mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2)$, $\mathbf{i}^*$ (computed in $\widetilde{\mathsf{Vrfy}}$), and $\mathtt{aux} = (\alpha_0, \alpha_1, \mathtt{com}_{-c})$, and receive x'.
5. Return x'.

For ease of notation, we denote this experiment as $x' \leftarrow \mathsf{Ext} \circ \mathcal{A}(\mathtt{vk})$. We then have

$$\begin{aligned}\mathsf{Adv}^{\text{euf-nma}}_{\overline{\mathsf{MQOM}},\mathcal{A}}(\lambda) &\le \Pr[(\mathtt{vk},\mathtt{sk}) \leftarrow \mathsf{Gen}(1^\lambda); x' \leftarrow \mathsf{Ext} \circ \mathcal{A}(\mathtt{vk}) \,:\, \hat{F}(x') = 0] \\ &\quad + \Pr[(\mathtt{vk},\mathtt{sk}) \leftarrow \mathsf{Gen}(1^\lambda); x' \leftarrow \mathsf{Ext} \circ \mathcal{A}(\mathtt{vk}) \,:\, \hat{F}(x') \neq 0] \\ &\le \mathsf{Adv}^{\text{ow}}_{\mathsf{Gen},\mathcal{A}_{\text{ow}}}(\lambda) + \epsilon_{\text{snd}},\end{aligned}$$

where we consider $\mathsf{Ext} \circ \mathcal{A}_{\text{nma}}$ as $\mathcal{A}_{\text{ow}}$ and we define

$$\epsilon_{\text{snd}} := \Pr[(\mathtt{vk},\mathtt{sk}) \leftarrow \mathsf{Gen}(1^\lambda); x' \leftarrow \mathsf{Ext} \circ \mathcal{A}(\mathtt{vk}) \,:\, \hat{F}(x') \neq 0].$$

We note that $\mathcal{A}_{\text{ow}}$ internally simulates random oracles (and an ideal cipher) and maintains the database D. The numbers of queries $\mathcal{A}$ and $\widetilde{\mathsf{Vrfy}}$ made are $\tilde{Q}'_{\mathsf{Hash}_3} := \tilde{Q}_{\mathsf{Hash}_3} + 1$, $\tilde{Q}'_{\mathsf{Hash}_4} := \tilde{Q}_{\mathsf{Hash}_4} + 1$, $\tilde{Q}'_{\mathsf{XOF}_5} := \tilde{Q}_{\mathsf{XOF}_5} + 1$, $\tilde{Q}'_{\mathsf{Hash}_6} := \tilde{Q}_{\mathsf{Hash}_6} + \tau$, $\tilde{Q}'_{\mathsf{Hash}_7} := \tilde{Q}_{\mathsf{Hash}_7} + 1$, and $\tilde{Q}'_E := \tilde{Q}_E + 3\tau(N-1)$.

Evaluating ϵ_{snd} via classical capacity: What remains is evaluating ϵ_{snd} formally. We consider the database properties defined as follows:

$$\begin{aligned}\mathsf{SZ}_{\le k} &:= \{D \mid |D| \le k\} \\ \mathsf{CL}_{\mathsf{Hash}} &:= \{D \mid \exists H, x \neq x' \,:\, D_H(x) = D_H(x') \neq \bot\} \\ \mathsf{CL}_E &:= \{D \mid \exists t_0, t_1, s \neq s' \,:\, \forall b \in \{0,1\} \,:\, D_E(t_b, s) \oplus \sigma(s) = D_E(t_b, s') \oplus \sigma(s')\} \\ \mathsf{CL} &:= \mathsf{CL}_{\mathsf{Hash}} \cup \mathsf{CL}_E \\ \mathsf{SUC} &:= \left\{ D \;\middle|\; \begin{array}{c} \exists h_{\mathtt{msg}}, \mathtt{inst} = (\mathtt{vk}, \mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2), \mathtt{nonce}, \mathtt{aux} = (\alpha_0, \alpha_1, \mathtt{com}_{-c}) \\ \text{so that } \mathtt{seed} := \mathsf{Inv}_D(\mathtt{com}_1) \text{ satisfies} \\ 1)\ \mathsf{V}^D(\mathtt{vk}, a, \mathbf{i}^*, z) = 1 \text{ and } v_{\text{grinding}} = 0^w \\ \text{for } a = (\mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2),\ z = (\mathtt{seed}_c, \mathtt{com}_{-c}, \alpha_0, \alpha_1), \\ \text{and } (\mathbf{i}^*, v_{\text{grinding}}) = D_{\mathsf{XOF}_5}(D_{\mathsf{Hash}_4}(\mathtt{vk}, \mathtt{salt}, \mathtt{com}_1, \mathtt{com}_2, h_{\mathtt{msg}}), \mathtt{nonce}) \\ 2)\ \hat{F}(\mathsf{Ext}(\mathtt{inst}, \mathbf{i}^*, \mathtt{aux}, \mathtt{seed})) \neq 0 \end{array} \right\}.\end{aligned}$$

We note that V only accessed to D_{Hash_3}, D_{Hash_6}, D_{Hash_7}, and D_E to verify a transcript. We consider *classical transition capacity* [CFHL21] and show the following lemma.

Lemma 5.5. *For every $\tilde{Q}' \in \mathbb{N}$,*

$$[\bot \Longrightarrow_{\tilde{Q}'} \mathsf{SUC} \cup \mathsf{CL}] \le \tilde{Q}' \cdot \max\{4\tilde{Q}'\tau N \cdot 2^{-\lambda}, p^{\mathfrak{S}}_{\text{triv}} \cdot 2^{-w}\}.$$

Proof. By routine calculations, we have

$$\begin{aligned}[\bot \Longrightarrow_{\tilde{Q}'} \mathsf{SUC} \cup \mathsf{CL}] &\le \sum_{k \in [\tilde{Q}']} [\mathsf{SZ}_{\le k} \setminus \mathsf{SUC} \setminus \mathsf{CL} \to \mathsf{SUC} \setminus \mathsf{CL}] \\ &\le \sum_{k \in [\tilde{Q}']} [\mathsf{SZ}_{\le k} \setminus \mathsf{CL} \to \mathsf{CL}] + \sum_{k \in [\tilde{Q}']} [\mathsf{SZ}_{\le k} \setminus \mathsf{SUC} \to \mathsf{SUC}].\end{aligned}$$

For a collision for $\mathsf{Hash}_6, \mathsf{Hash}_7$, etc, we have the bound $[\mathsf{SZ}_{\le k} \setminus \mathsf{CL}_{\mathsf{Hash}} \to \mathsf{CL}_{\mathsf{Hash}}] \le (k+1)/2^{2\lambda} \le \tilde{Q}'/2^{2\lambda}$. For a collision for E inside $\mathsf{cAVC}_{\mathrm{IC}}$, we have $[\mathsf{SZ}_{\le k} \setminus \mathsf{CL}_E \to \mathsf{CL}_E] \le (k+1)/2^{\lambda}$. Thus, for all $k \in [\tilde{Q}']$, we have

$$[\mathsf{SZ}_{\le k} \setminus \mathsf{CL} \to \mathsf{CL}] \le \max\{\tilde{Q}' \cdot 2^{-2\lambda}, \tilde{Q}' \cdot 2^{-\lambda}\} \le \tilde{Q}' \cdot 2^{-\lambda}.$$

We then consider $[\mathsf{SZ}_{\le k} \setminus \mathsf{SUC} \to \mathsf{SUC}]$. Let s and u be a query and an answer. We have the following cases depending on the queried functions:

- $E^{\pm}$: In this case, a query is of the form (t, s) to E or (t, y) to E^-. In both cases, randomly chosen image $y \leftarrow \{0,1\}^\lambda \setminus D_{E,}[t]$ or preimage $s \leftarrow \{0,1\}^\lambda \setminus D_{E,}[t]$ hits the half of one of commitments $\mathtt{com}[e][i]$. Since then number of the commitments $\mathtt{com}$ is at most $k\tau N$, we have

$$\begin{aligned}\Pr_y[D[(t,s) \mapsto_E y] \in \mathsf{SUC}], \Pr_s[D[(t,s) \mapsto_E y] \in \mathsf{SUC}] &\le k(2\tau N)/(2^\lambda - i) \\ &\le 2\tilde{Q}'\tau N/(2^\lambda/2) \\ &= 4\tilde{Q}'\tau N \cdot 2^{-\lambda}.\end{aligned}$$

- Hash_3: In this case, $s = (\alpha_0, \alpha_1)$ and randomly chosen $u \leftarrow \{0,1\}^{2\lambda}$ satisfies the condition in SUC. Since the number of the image $\mathtt{com}_2$ is at most k, we have $\Pr_u[D[s \mapsto_{\mathsf{Hash}_3} u] \in \mathsf{SUC}] \le k/2^{2\lambda} \le \tilde{Q}'/2^{2\lambda}$.
- Hash_4: In this case, $s = (\mathtt{vk}, \mathtt{com}_1, \mathtt{com}_2, h_{\mathtt{msg}})$ and randomly chosen $u \leftarrow \{0,1\}^{2\lambda}$ satisfies the condition in SUC: Since the number of the image h_2 is at most k, we have $\Pr_u[D[s \mapsto_{\mathsf{Hash}_4} u] \in \mathsf{SUC}] \le k/2^{2\lambda} \le \tilde{Q}'/2^{2\lambda}$.
- XOF_5: In this case, $s = (h_2, \mathtt{nonce})$ and randomly chosen $(u, v) \leftarrow [N]^\tau \times \{0,1\}^w$ satisfies the condtion in SUC. For fixed, a and z, we define a boolean predicate $\mathsf{GoodV}(c) : [N]^\tau \to \{0,1\}$ that is 1 if and only if $\mathsf{V}^D(\mathtt{vk}, a, c, z) = 1$.

$$
\begin{aligned}
&\Pr_{(u,v)}[D[s \mapsto_{\mathsf{XOF}_5} (u,v)] \in \mathsf{SUC}] \\
&\quad \le \Pr_{(u,v)}[\mathsf{GoodV}(u) = 1 \wedge v = 0^w \wedge \hat{F}(\mathsf{Ext}(h_{\mathtt{msg}}, \mathtt{inst}, \mathtt{aux}, \mathtt{seed})) \neq 0] \\
&\quad \le \Pr_{(u,v)}[\mathsf{GoodV}(u) = 1 \wedge v = 0^w \wedge S := \{c \mid \mathsf{GoodV}(c)\} \notin \mathfrak{S}] \\
&\quad \le \Pr_u[u \in S := \{c \mid \mathsf{GoodV}(c)\} \notin \mathfrak{S}] \cdot \Pr_v[v = 0^w] \\
&\quad \le \left(\max_{S \notin \mathfrak{S}} \Pr_{u \leftarrow [N]^\tau}[u \in S] \right) \cdot 2^{-w} = p^{\mathfrak{S}}_{\mathrm{triv}} \cdot 2^{-w}.
\end{aligned}
$$

- Hash_6: In this case, s is of the form $\mathtt{com}[e]$ and randomly chosen $u \leftarrow \{0,1\}^{2\lambda}$ satisfies the condition in SUC: Since the number of the image $h_{\mathtt{com}}[e]$ is at most $k\tau$, we have $\Pr_u[D[s \mapsto_{\mathsf{Hash}_6} u] \in \mathsf{SUC}] \le k\tau/2^{2\lambda} \le \tilde{Q}'\tau/2^{2\lambda}$.
- Hash_7: In this case, $s = (h_{\mathtt{com}}, \Delta^{(1)}_x)$ and randomly chosen $u \leftarrow \{0,1\}^{2\lambda}$ satisfies the condition in SUC: Since the number of the image $\mathtt{com}_1$ is at most k, we have $\Pr_u[D[s \mapsto_{\mathsf{Hash}_7} u] \in \mathsf{SUC}] \le k/2^{2\lambda} \le \tilde{Q}'/2^{2\lambda}$.

Thus, wrapping up, we have

$$
\begin{aligned}
[\mathsf{SZ}_{\le k} \setminus \mathsf{SUC} \to \mathsf{SUC}] &\le \max\{\tilde{Q}' \cdot 2^{-2\lambda}, 4\tilde{Q}'\tau N \cdot 2^{-\lambda}, p^{\mathfrak{S}}_{\mathrm{triv}} \cdot 2^{-w}\} \\
&= \max\{4\tilde{Q}'\tau N \cdot 2^{-\lambda}, p^{\mathfrak{S}}_{\mathrm{triv}} \cdot 2^{-w}\}.
\end{aligned}
$$

□

Since the simulation by database D is perfect, we have

$$
\begin{aligned}
\epsilon_{\mathrm{snd}} &= \Pr[\widetilde{\mathsf{Vrfy}}^D(\mathtt{vk}, h_{\mathtt{msg}}, \tilde{\sigma}) = 1 \wedge \hat{F}(x') \neq 0] \\
&\le \Pr[\widetilde{\mathsf{Vrfy}}^D(\mathtt{vk}, h_{\mathtt{msg}}, \tilde{\sigma}) = 1 \wedge \hat{F}(x') \neq 0 \mid D \notin \mathsf{SUC} \cup \mathsf{CL}] + \Pr[D \in \mathsf{SUC} \cup \mathsf{CL}] \\
&\le 0 + [\bot \Longrightarrow_{\tilde{Q}'} \mathsf{SUC} \cup \mathsf{CL}] \\
&\le 0 + \tilde{Q}' \cdot \max\{4\tilde{Q}'\tau N \cdot 2^{-\lambda}, p^{\mathfrak{S}}_{\mathrm{triv}} \cdot 2^{-w}\} \\
&\le \tilde{Q}' \cdot \max\{4\tilde{Q}'\tau N \cdot 2^{-\lambda}, (2/N)^\tau \cdot 2^{-w}\}
\end{aligned}
$$

as we wanted, where we use Equation 16. □